scope-fw cookbook

A DIY oscilloscope on the STM32F103C6T6 Blue Pill

Bare-metal C. 18 issues. Recipes, not puzzles. Fully offline.

Every issue opens with a START HERE box. Every step ends with a CHECK. The six pattern cards come back again and again until you do not need the book.

Contents

How to use this cookbook **3**

Before you go offline: checklist **5**

Project spec sheet **7**

Toolchain cheat sheet **9**

Safety notes **11**

Pattern cards C1 to C6 **12**

Milestone M1: Foundation **14**

M1-01 Register header + LED (GPIO and RCC structs, field_set, pin_shift, gpio_cfg) . . . 15

M1-02 Linker script + startup code + cross Makefile . . . 21

M1-03 Flash with OpenOCD, blink, and the CI cross-build job . . . 27

M1-04 72 MHz clock and SysTick millis (plus wait_flag, elapsed, reached) . . . 31

M1-05 USART1 transmit, fmt_u32, and the UART banner . . . 37

Milestone M2: Sampling **42**

M2-01 ADC single read on PA2 (plus mul_div and smpr_field) . . . 43

M2-02 TIM3-triggered ADC with DMA1 channel 1: capture 256 samples . . . 48

M2-03 PWM test signal on PA1 (TIM2 channel 2) looped into the ADC . . . 53

Milestone M3: Display **57**

M3-01 I2C1 master with timeouts and a bus scan that finds 0x3C . . . 58

M3-02 SSD1306 init/flush and the framebuffer (host-tested with a fake I2C) . . . 63

M3-03 plot_wave, grid and 5x7 text . . . 67

Milestone M4: Trigger and controls **71**

M4-01 trig_find: the edge trigger . . . 72

M4-02 Time/div and volts/div tables, sample-rate math . . . 75

M4-03 Button debouncer and the table-driven UI state machine . . . 79

Milestone M5: Measurements **84**

M5-01 min, max, Vpp and average . . . 85

M5-02 Frequency from rising crossings, fmt_mv and fmt_hz . . . 88

Milestone M6: Ship **92**

M6-01 scope_process, scope_draw and the main super loop . . . 93

M6-02 Calibration, README, CI badge, tag v1.0 (stretch: hold/freeze) . . . 98

Print list (placeholder links) **103**

How to use this cookbook

This book teaches you to build a small oscilloscope from nothing. You write the register header, the linker script, the startup code and the Makefile yourself. There is no RTOS and no vendor library.

You will get stuck at the start of tasks. That is normal. This book is built so that you never have to ask "where do I begin?". Every issue opens with a **START HERE** box. It tells you the first command or the first file.

The loop (use it for every issue)

Part	Time	What you do
Warm-up	10 min	Delete three old drill functions and rewrite them from memory.
Learn	30 min	Read the short explanation. The analogies are there to be remembered.
Worked example	15 min	Read complete, working code for a similar but different part. Copy its shape.
Cookbook steps	1 to 3 h	Do the numbered STEPs in order. Each step ends with a CHECK.
Verify	15 min	Run the "Do this, Expect this" tables.
Self-check	5 min	Answer the questions aloud. Open hints one at a time only if you are stuck.

How a STEP looks

Every step has five lines. Read them top to bottom.

Line	Meaning
WHAT	One plain sentence: what this step does.
INGREDIENTS	The exact register, bit, value or offset you need. They are also in the cheat sheet.
HOW	The shape of the code, with placeholders, and a small table with this step's real numbers.
WHERE	The file and the function.
CHECK	A 10-second command and the exact thing you should see.

You type the code yourself. The book never gives the finished line. The last hint shows a template with every value filled in, and you still type the line.

Every TODO has a tag

In the code you will find comments like `@TODO(M1-01/6) ... [src: M1-01 step 6, card C1]`. The tag points to the step that answers it. Find the issue text with `TODO M1-01/6` and you land on it.

Repetition is the method

The same six patterns (cards C1 to C6) come back again and again with new numbers. A step often says "this is card C1 again, with shift 14 and width 2". That is on purpose. After a few rounds you will stop needing the card.

Daily commands

You want	Type
Run one issue's tests with full output	`make t T=m1_01`
Run every test CI will run	`make test-active`
Run every test file (red for issues you have not done)	`make test-all`
Build the firmware (from M1-02)	`make firmware`
Flash the board (from M1-03)	`make flash`
Save your work	`git add -A && git commit -m "M1-01 done"`

Commit when an issue is green. Commit before every warm-up. A commit is your undo button.

Files you edit

`include/stm32f103.h` (registers), `src/*.c` (code), `ld/stm32f103c6.ld` (linker script), `Makefile`, `.github/workflows/ci.yml`, `tests.active`. You never edit the files in `test/` or `third_party/`.

Rules of the road

1. One issue at a time, in order. Later issues assume earlier ones are green.
2. Add the issue's test name to `tests.active` only when it is green. CI is then green after every issue.
3. You go to the internet only to run `git push`. Everything else works offline.
4. The only wires that touch PA2 (the ADC pin) are PA1, GND, 3V3, or the pot wiper. Read the safety page first.

Before you go offline: checklist

Do this once, with the internet on. Tick every box. After this you only need the internet for `git push`.

1. NixOS packages

Add these to `environment.systemPackages` in `configuration.nix`, then run `sudo nixos-rebuild switch`.

```
gcc-arm-embedded   # arm-none-eabi-gcc, size, objdump, nm
gcc                # the host compiler for unit tests
gnumake
gdb                # multi-architecture
openocd
stlink             # st-info
git
picocom
```

Add the udev rules and the serial group, then log out and in:

```
services.udev.packages = [ pkgs.stlink pkgs.openocd ];
users.users.yourname.extraGroups = [ "dialout" ];
```

2. Check each tool (type these, read the answer)

Type	Expect
`arm-none-eabi-gcc --version`	a version line
`gcc --version`	a version line
`make --version`	GNU Make
`openocd --version`	a version line
`gdb --version`	a version line
`st-info --probe` (ST-Link plugged in, board wired)	chip info with `flash: 32768` for a real C6 (some clones report more)
`picocom --help`	usage text
`git --version`	a version line

3. Repo and CI

- [ ] Create an empty GitHub repository called `scope-fw`.
- [ ] Unpack `scope-fw.zip`, open a terminal inside the folder, then type: `git init`, `git add -A`, `git commit -m "skeleton"`.
- [ ] Add the remote: `git remote add origin <your repo address>` and `git push -u origin HEAD`.
- [ ] Open the Actions tab once and look at the first run. The host-tests job runs `make test-active`.

4. The test framework

`third_party/unity/` holds a small Unity-compatible test framework, so the repo builds with no internet. If you want the full upstream Unity (ThrowTheSwitch/Unity, MIT licence), copy its `unity.c`, `unity.h` and `unity_internals.h` over the three files in that folder now. No test needs changing.

5. Print these pages (Ctrl+P) before you disconnect

The book works without them. They are extra reading. Placeholder links are listed on the last page of this book ("Print list").

- [ ] This book (the PDF) on paper or on a second screen.
- [ ] The print list pages.

6. Dry run

- [ ] Turn the Wi-Fi off. Type `make test-all`. You see red tests with readable messages. That is correct.
- [ ] Turn the Wi-Fi on again only when you want to push.

7. Parts on the desk

Blue Pill, 0.96 inch SSD1306 OLED (I2C), 10k pot, ST-Link V2, USB-TTL adapter, 3 buttons, breadboard, jumper wires. You need no multimeter, resistor or capacitor.

Project spec sheet

DIY oscilloscope on an STM32F103C6T6 Blue Pill: 32 KB flash, 10 KB RAM, 8 MHz crystal, on-board LED on PC13 (active-low: pin LOW = LED on). Bare-metal C: one super loop plus interrupts. No RTOS. Output only on a 128x64 SSD1306 OLED.

Features: edge trigger (rising or falling, adjustable level), time/div and volts/div by buttons, readouts of frequency, Vpp and average. Stretch: hold/freeze.

Pin map

Function	Pin	Notes
UART TX	PA9	USART1, to the USB-TTL RX
UART RX	PA10	USART1, to the USB-TTL TX
ADC input	PA2	ADC channel 2. 0 to 3.3 V ONLY
PWM test signal	PA1	TIM2 channel 2, 1 kHz
Sampling trigger	(internal)	TIM3 TRGO starts each ADC conversion
I2C SCL	PB6	I2C1, to the OLED SCL
I2C SDA	PB7	I2C1, to the OLED SDA
Button MODE	PB12	to GND when pressed, internal pull-up
Button UP	PB13	same
Button DOWN	PB14	same
LED	PC13	active-low

Memory map

Region	Start	Size	End (first address after)
FLASH	0x08000000	32 KB (0x8000)	0x08008000
RAM	0x20000000	10 KB (0x2800)	0x20002800 (this is `_estack`)

Vector table word 0 = 0x20002800 (stack top). Word 1 = Reset_Handler address with the lowest bit set (odd number: the Thumb bit).

RAM budget: framebuffer 1024 bytes, capture buffer 256 x 2 = 512 bytes, the rest is variables and the stack.

Clock tree

Step	Value
Crystal (HSE)	8 MHz
PLL	x9 = 72 MHz = SYSCLK = AHB
Flash	2 wait states, prefetch on
APB2	72 MHz (no divider)
APB1	/2 = 36 MHz; timers on APB1 then run at 72 MHz
ADC clock	72 / 6 = 12 MHz (limit is 14 MHz)
I2C	400 kHz fast mode: CR2 FREQ 36, CCR 30 (F/S = 1, DUTY 0), TRISE 12. For 100 kHz: CCR 180, TRISE 37
USART1	BRR 625 for 115200 baud
SysTick	LOAD 71999 = 1 ms

Screen layout

128 px wide = 8 divisions x 16 px. Rows 0 to 15 are text (OLED pages 0 and 1): two lines of 5x7 characters, each 6 px wide. Plot rows 16 to 63 = 6 divisions x 8 px. Row 63 is 0 V.

Capture buffer: 256 uint16 samples. Displayed window: 128 samples. The trigger is scanned only in the first 128 samples (start index 1), so the window always fits. No trigger = auto: the window starts at sample 0.

Text rows (examples): `1ms 1V 1.00kHz R` and `T p3.30V a0.83V`. First row: time/div, volts/div, frequency, edge (R or F). Second row: mode letter (T, V, L or E), p = Vpp, a = average.

Scale tables

Index	Time/div (us)	Label	Sample rate (16 px per div / time)
0	100	100us	160000 Hz
1	200	200us	80000 Hz
2	500	500us	32000 Hz
3	1000	1ms	16000 Hz
4	2000	2ms	8000 Hz
5	5000	5ms	3200 Hz
6	10000	10ms	1600 Hz
7	20000	20ms	800 Hz

Index	Volts/div (mV)	Label	Full screen
0	1000	1V	6.0 V
1	500	500mV	3.0 V (3.3 V clips at the top: a teaching point)
2	250	250mV	1.5 V

Default UI: time/div index 3 (1 ms), volts/div index 0, trigger level 1650 mV, rising edge.

Slow rates and a 1 kHz signal: at 8 kHz or slower the test signal has fewer than 10 samples per period, and below 2 kHz it is aliased. The frequency readout is only trustworthy at 500us/div and faster.

Test-signal plan

PWM 1 kHz, 25 % duty on PA1, wired to PA2. Capture at 32 kSPS (256 samples = 8 periods).

Measurement	Expect
Samples above 2048	about 64 (allow 56 to 72)
Frequency readout	1.00 kHz within 2 %
Vpp	near 3.30 V
Average	near 0.82 V
PA2 wired to GND	0 to about 10 counts
PA2 wired to 3V3	about 4080 to 4095 counts

SSD1306 bytes

Init bytes, in order (25): `AE D5 80 A8 3F D3 00 40 8D 14 20 00 A1 C8 DA 12 81 CF D9 F1 DB 40 A4 A6 AF`. Sent in ONE I2C write with a control byte 0x00 in front (26 bytes). Address 0x3C.

Flush = one command write `00 21 00 7F 22 00 07`, then 8 data writes, each `40` followed by 128 bytes (one page).

Formulas used everywhere

Formula	Value
counts to mV	counts x 3300 / 4095
mV to counts	mV x 4095 / 3300
sample rate	16 x 1,000,000 / time-per-div (us)
timer ticks per sample	72,000,000 / sample rate
y row on screen	63 - (mV x 8 / volts-per-div), kept inside 16..63

Toolchain cheat sheet

Build and test (make)

Type	What happens
`make t T=m1_01`	builds and runs one test file on your PC; prints one line per test
`make test-active`	runs the tests listed in `tests.active` (what CI runs)
`make test-all`	runs every test file
`make firmware`	cross-builds `build/firmware.elf` and `build/firmware.bin`
`make size`	prints text, data, bss sizes
`make flash`	writes the firmware to the board with OpenOCD
`make clean`	deletes `build/`

Recipe lines in a Makefile start with a real TAB. Spaces do not work.

Reading the build

Type	Tells you
`arm-none-eabi-size build/firmware.elf`	text (code + constants, in flash), data (initialised variables, flash and RAM), bss (zeroed variables, RAM). Flash used = text + data. RAM used = data + bss.
`arm-none-eabi-nm build/firmware.elf`	every symbol with its address
`arm-none-eabi-objdump -h build/firmware.elf`	the sections with their addresses (VMA) and load addresses (LMA)
`arm-none-eabi-objdump -s -j .isr_vector build/firmware.elf`	the vector table bytes. Little-endian: `00280020` means 0x20002800
`arm-none-eabi-objdump -d build/firmware.elf`	the disassembly

Flash and debug

Wire the ST-Link: SWDIO, SWCLK, GND, 3V3 to the board's 4-pin header. Power the board from the ST-Link only. Keep both BOOT jumpers at 0.

Type	What happens
`st-info --probe`	prints the chip id and flash size
`make flash`	flashes and resets (OpenOCD prints `Verified OK`)
`openocd -f interface/stlink.cfg -f target/stm32f1x.cfg`	starts a GDB server on port 3333 (leave it running)
`openocd -c "set CPUTAPID 0x2ba01477" -f interface/stlink.cfg -f target/stm32f1x.cfg`	the same, for clone chips with another id (the -c part goes first)

In a second terminal:

GDB command	Meaning
`gdb build/firmware.elf`	start GDB with symbols
`target extended-remote :3333`	connect to OpenOCD
`monitor reset halt`	reset and stop the chip
`load`	write the firmware
`break main` then `continue`	run to main
`info registers`	CPU registers
`x/4wx 0x08000000`	4 words in hex from flash start
`x/wx 0x4001100C`	one word: GPIOC ODR
`print g_ms`	a variable
`watch g_ms`	stop when it changes
`stepi` or `next`	one instruction or one line
`set architecture arm`	only if GDB complains about the architecture

Serial

`picocom -b 115200 /dev/ttyUSB0` opens the terminal. Leave with Ctrl-A then Ctrl-X. If it says permission denied, your user is not in the `dialout` group.

Wire: adapter RX to PA9, adapter TX to PA10, GND to GND. Leave the adapter's power pin unconnected, because the ST-Link powers the board.

Git

Type	Meaning
`git status`	what changed
`git add -A && git commit -m "message"`	save a snapshot
`git checkout src/drills.c`	throw away uncommitted changes to one file
`git push`	the one step that needs the internet; CI then runs
`git tag v1.0 && git push --tags`	mark the release

Safety notes

You have no multimeter. These rules protect the chip.

1. **ADC pins take 0 to 3.3 V only.** PA2 must only ever touch: PA1 (your own PWM), GND, 3V3, or the middle pin of the pot. Never connect PA2 to 5 V. Never connect it to an unknown signal.
2. **Check the logic level of the USB-TTL adapter before probing it.** Many adapters (CH340G, PL2303) idle at 5 V on TX. PA9 and PA10 tolerate 5 V, but PA2 does not. Never connect the adapter to PA2. Look for a 3.3V/5V jumper or switch on the adapter and set it to 3.3 V if it has one.
3. **Power the board from one source only:** USB or the ST-Link 3.3 V pin, never both. Leave the USB-TTL adapter's power pin unconnected.
4. **3V3 and GND must never touch each other.** Check the breadboard before you power up.
5. **Keep both BOOT jumpers at 0.**
6. **The OLED runs on 3.3 V.** Wire VCC to the 3V3 pin.
7. **Do not hot-plug wires into PA2** while the board runs a test. Unplug the USB first. A wire that is loose can touch a 5 V rail.
8. **Pot wiring:** outer pins to 3V3 and GND, the middle pin to PA2. Never put the middle pin on 3V3 while the outer pin is on GND with a wire loose elsewhere.
9. If something gets hot or smells: unplug at once.

Pattern cards C1 to C6

Six patterns carry the whole project. Each is introduced here once and then comes back in many issues with new numbers. Warm-ups make you rewrite the drill functions from memory.

C1 Change a field (clear with a mask, then set)

Drill function: `field_set`

```
REG &= ~(MASK << SHIFT);     /* 1. clear the field  */
REG |=  (VALUE << SHIFT);    /* 2. write the value  */
MASK = (1 << WIDTH) - 1      /* width 1 -> 0x1, 2 -> 0x3, 3 -> 0x7, 4 -> 0xF */
In code:  reg = field_set(reg, SHIFT, WIDTH, VALUE);
```

Erase one box on a paper form, then write in the box. If you skip the erase, old ink and new ink mix.

```
field_set(0x44444444, 20, 4, 2)
  clear bits 23..20 -> 0x44044444
  set 2 in them     -> 0x44244444
```

register value	shift	width	value	result
0x44444444	20	4	2	0x44244444
0xFFFFFFFF	4	4	0	0xFFFFFF0F
0x00000000	8	3	5	0x00000500

C2 GPIO pin nibble (CRL or CRH)

Drill function: `pin_shift / gpio_cfg`

```
shift = (pin % 8) * 4          /* each pin owns 4 bits */
register = (pin < 8) ? CRL : CRH
gpio_cfg(GPIOx, pin, NIBBLE);   /* = C1 with width 4 */
```

A hotel corridor: 8 rooms per floor, 4 bits per room. Pins 0-7 are on floor CRL, pins 8-15 on floor CRH.

```
PC13: 13 % 8 = 5, shift = 5 * 4 = 20, register CRH.
PA2 : shift = 2 * 4 = 8,  register CRL.
```

nibble	meaning
0x0	analog input
0x4	floating input
0x8	input with pull-up or pull-down (ODR bit picks which: 1 = pull-up)
0x2	output, push-pull, 2 MHz
0x3	output, push-pull, 50 MHz
0xA	alternate function, push-pull, 2 MHz
0xB	alternate function, push-pull, 50 MHz
0xF	alternate function, open-drain, 50 MHz

C3 Wait for a flag with a timeout

Drill function: `wait_flag`

```
while (tries--) {
    if ((*reg & MASK) is in the wanted state) return 0;   /* flag came */
}
return -1;                                                /* gave up   */
In code:  if (wait_flag(&REG, MASK, WANT_SET, TRIES)) { /* timed out */ }
```

Ring a doorbell up to N times. If nobody comes, walk away. Never ring forever.

```
wait_flag(&USART1->SR, USART_SR_TXE, 1, 100000u)   -> 0 when TXE became 1, -1 if not
wait_flag(&I2C1->SR2, I2C_SR2_BUSY, 0, 20000u)       -> waits for BUSY to become 0
```

call	returns
flag already in the wanted state	0
flag never arrives, tries = 5	-1 after 5 reads
tries = 0	-1 at once

C4 Peripheral bring-up recipe

Drill function: `(used in every driver)`

```
1. CLOCK      RCC->xxxENR |= bit        (a peripheral is OFF until its clock bit is set)
2. PIN        gpio_cfg(GPIOx, pin, NIBBLE)
3. CONFIGURE  write the setup registers (C1 for fields)
4. START      the enable bit (UE, ADON, CEN, PE, ...)
5. WAIT       wait_flag(...)  (C3)
6. READ       read DR / the result
```

Opening a shop: electricity on, door unlocked, shelves arranged, sign flipped to OPEN, wait for a customer, serve.

```
USART1: clock APB2ENR bits 2 and 14 -> pins PA9 0xB, PA10 0x4 -> BRR, CR1 -> UE|TE|RE
        -> wait TXE -> write DR
```

C5 Integer scaling: multiply first, divide last

Drill function: `mul_div`

```
result = (uint64_t)a * b / c;       /* multiply first, in 64 bits */
In code:  mul_div(a, b, c)
```

Converting money: multiply 2048 by 3300 first, then divide by 4095. Dividing 2048/4095 first throws the answer away (it becomes 0).

```
mul_div(2048, 3300, 4095) = 6758400 / 4095 = 1650
wrong order: (2048 / 4095) * 3300 = 0 * 3300 = 0
```

a	b	c	result
4095	3300	4095	3300
2048	3300	4095	1650
1	1	2	0

C6 Wrap-safe time compare

Drill function: `elapsed / reached`

```
elapsed = now - start;                    /* unsigned subtraction survives the wrap */
reached = (int32_t)(now - deadline) >= 0; /* sign of the difference                 */
NEVER write:  now >= deadline
```

A car odometer that rolls over from 999999 to 000000. The distance driven is still new minus old.

```
elapsed(5, 0xFFFFFFF0) = 21
reached(5, 0xFFFFFFF0) = 1      (the deadline was just before the wrap)
```

now	start / deadline	elapsed	reached
100	40	60	1
9	10	(wraps: huge)	0
5	0xFFFFFFF0	21	1

Milestone M1: Foundation

Register header, linker script, startup, flashing, clock, UART.

Title: M1-01 Register header + LED (GPIO and RCC structs, field_set, pin_shift, gpio_cfg)

Milestone: M1 Foundation

Labels: m1, blank-level-1, host-tests

M1-01 Register header + LED (GPIO and RCC structs, field_set, pin_shift, gpio_cfg)

START HERE

1. Open a terminal inside the repo folder. Type `git init`, then `git add -A`, then `git commit -m "skeleton"`.
2. Type `make t T=m1_01`. You will see 10 red tests. That is your starting point.
3. Open `include/stm32f103.h`. Find the comment `@TODO(M1-01/1)`. That is STEP 1 below.

Starts from / Ends with

Starts from	Ends with
The unpacked skeleton repo. Nothing earlier is needed.	GPIOC and RCC structs and addresses in your header.
	`field_set`, `pin_shift`, `gpio_cfg` written and tested (cards C1, C2).
	`led_init`, `led_on`, `led_off`, `led_toggle` tested against a fake chip.
	`test_m1_01` in `tests.active`, one git commit.

Warm-up (10 minutes, code only): the pointer mirror

There are no drills yet, so warm up on your weak spot. `&` asks "where does it live?" and gives an address. `*` asks "who is home at that address?" and gives the value.

1. Make a scratch file: type `mkdir -p scratch` then open `scratch/ptr.c` and copy this in:

```
/* Pointer mirror drill. & = "address of".  * = "value at that address". */
#include <stdio.h>
int main(void)
{
    int cookies = 5;
    int *jar = &cookies;       /* jar holds the ADDRESS of cookies          */
    *jar = *jar + 2;           /* read the value at jar, add 2, store back  */
    printf("cookies=%d\n", cookies);
    printf("same=%d\n", jar == &cookies);
    return 0;
}
```

2. BEFORE you run it, write down what you think it prints.
3. Run: `gcc scratch/ptr.c -o scratch/ptr && ./scratch/ptr`. Expect: `cookies=7` then `same=1`.
4. Change `*jar = *jar + 2;` to `*jar = *jar * 3;`. Predict, then run. Expect `cookies=15`.
5. Put the original line back and add two lines after it: `int *other = jar;` and `*other = 1;`. Predict, then run. Expect `cookies=1`.

Symbol	Read it as
`&cookies`	the address of the variable `cookies`
`int *jar`	`jar` holds an address of an int
`*jar`	the int that lives at that address

Learn (about 30 minutes)

A peripheral is a row of mailboxes. Each mailbox is a 32-bit register at a fixed street address. Writing a number into it flips switches inside the chip. Reading it shows what the chip is doing.

A struct lays mailboxes side by side. If a struct holds 32-bit members, the first sits at offset 0x00, the next at 0x04, then 0x08 and so on. So the struct IS the register map. Cast the base address to a pointer to the struct and `GPIOC->ODR` means "the mailbox at base + 0x0C".

`volatile` tells the compiler: "this value can change behind your back. Do not skip reads or writes." Every register member needs it.

Pointers, your weak spot. `&x` asks "where does x live?" and gives an address. `*p` asks "who is home at address p?" and gives the value. Read `*(volatile uint32_t *)0x40021014u` as: "treat this number as an address, then go to that address."

Clock gates. At power-on most peripherals have their clock switched off to save power. A register you write to an unclocked peripheral does nothing. So the first line of every driver turns the clock on in RCC.

GPIO pins. Each pin has a 4-bit setup nibble. Eight nibbles fit in a 32-bit register: CRL holds pins 0 to 7, CRH holds pins 8 to 15. That is card C2.

BSRR is a set/clear mailbox. Writing a 1 to bit n (low half) makes pin n HIGH. Writing a 1 to bit n+16 (high half) makes pin n LOW. Writing 0 changes nothing. No read-modify-write needed.

The LED is active-low. The on-board LED on PC13 lights when the pin is LOW.

The fake chip. The tests build with `-DUNIT_TEST`. Then `GPIOC` and `RCC` point at two ordinary structs in RAM (`fake_gpioc`, `fake_rcc`) instead of real addresses. Your code runs on your PC and the tests read the fake registers afterwards.

Worked example: the CRC unit (a different peripheral, same recipe)

```
/* WORKED EXAMPLE (M1-01): the CRC unit. Same recipe as GPIO and RCC, different peripheral. */
#include <stdint.h>
#define BIT(n) (1u << (n))

typedef struct {
    volatile uint32_t DR;    /* 0x00  write a word in, read the result out */
    volatile uint32_t IDR;   /* 0x04  one spare byte                       */
    volatile uint32_t CR;    /* 0x08  bit 0 = RESET                        */
} CRC_TypeDef;

#define CRC_BASE   0x40023000u
#define CRC        ((CRC_TypeDef *)CRC_BASE)       /* number -> pointer to struct */
#define RCC_AHBENR (*(volatile uint32_t *)0x40021014u)
#define CRCEN      BIT(6)                          /* clock gate bit in AHBENR    */

void crc_init(void)
{
    RCC_AHBENR |= CRCEN;            /* 1. clock on                                */
    CRC->CR = 1u;                   /* 2. reset: result goes back to 0xFFFFFFFF */
}

uint32_t crc_words(const uint32_t *p, unsigned n)
{
    for (unsigned i = 0; i < n; i++) {
        CRC->DR = p[i];             /* 3. feed one word                           */
    }
    return CRC->DR;                 /* 4. read the result                         */
}
```

Read it like this: struct (offsets 0, 4, 8) -> base address -> a macro that casts it to a pointer -> clock bit -> functions that write and read members.

Cheat sheet (everything this issue needs)

Item	Value
GPIOC base	0x40011000
RCC base	0x40021000
GPIOA / GPIOB base (given)	0x40010800 / 0x40010C00

GPIO register offsets (each 32 bit):

Name	CRL	CRH	IDR	ODR	BSRR	BRR	LCKR
Offset	0x00	0x04	0x08	0x0C	0x10	0x14	0x18

RCC register offsets (each 32 bit):

Name	CR	CFGR	CIR	APB2RSTR	APB1RSTR	AHBENR	APB2ENR	APB1ENR	BDCR	CSR
Offset	0x00	0x04	0x08	0x0C	0x10	0x14	0x18	0x1C	0x20	0x24

Fact	Value
APB2ENR bit for GPIOA / GPIOB / GPIOC	bit 2 / bit 3 / bit 4 (`IOPCEN` = bit 4)
LED pin	PC13 (pin 13), active-low
Nibble for PC13	pin 13 -> CRH, shift (13 % 8) * 4 = 20
Nibble value: output push-pull 2 MHz	0x2
CRH after reset	0x44444444 (all inputs, floating)
BSRR to set pin n HIGH	bit n
BSRR to set pin n LOW	bit n + 16
LED ON = pin LOW	BSRR = BIT(13 + 16) = 0x20000000
LED OFF = pin HIGH	BSRR = BIT(13) = 0x00002000
`BIT(n)`	`1u << n` (given in the header)

Cards C1 and C2 (full text in the book's Pattern Cards chapter):

```
C1:  REG &= ~(MASK << SHIFT);  REG |= (VALUE << SHIFT);     MASK = (1 << WIDTH) - 1
C2:  shift = (pin % 8) * 4;    register = (pin < 8) ? CRL : CRH
```

Cookbook steps

STEP 0 · set up (no TODO)

- **WHAT:** Make sure you can build and run the tests.
- **INGREDIENTS:** a terminal in the repo folder.
- **HOW:** `git init`, `git add -A`, `git commit -m "skeleton"`, then `make t T=m1_01`.
- **WHERE:** repo root.
- **CHECK:** the last lines read `10 Tests 10 Failures 0 Ignored` and `FAIL`. Each failing line ends with a hint such as `(M1-01 step 1)`.

STEP 1 · TODO M1-01/1 · base addresses · [src: M1-01 step 1]

- **WHAT:** Tell the header where GPIOC and RCC live.
- **INGREDIENTS:** GPIOC 0x40011000, RCC 0x40021000. See `GPIOA_BASE` right below the TODO: copy its style.
- **HOW:** `#define NAME_BASE 0x........u`

name	value	why
`GPIOC_BASE`	`0x40011000u`	port C registers
`RCC_BASE`	`0x40021000u`	clock control registers

- **WHERE:** `include/stm32f103.h`, under `/* ---------- base addresses ---------- */`. Delete the two placeholder lines that say `0u`.
- **CHECK:** `make t T=m1_01` -> a line ending `test_base_addresses:PASS`.

STEP 2 · TODO M1-01/2 · GPIO struct, 2 missing lines · [src: M1-01 step 2]

- **WHAT:** Finish the GPIO register struct. Two members are missing.
- **INGREDIENTS:** BRR at 0x14, LCKR at 0x18. The struct already has CRL..BSRR (0x00..0x10).
- **HOW:** one line per member, same style as `BSRR`: `volatile uint32_t NAME; /* 0x.. */`

member	offset	why
`BRR`	0x14	bit reset register (old way to clear pins)
`LCKR`	0x18	lock register (unused, but it fills the gap)

- **WHERE:** `include/stm32f103.h`, struct `GPIO_TypeDef`, after `BSRR`.
- **CHECK:** `make t T=m1_01` -> `test_gpio_layout:PASS`. The struct must be 0x1C bytes (7 x 4).

STEP 3 · TODO M1-01/3 · RCC struct, 3 missing lines · [src: M1-01 step 3]

- **WHAT:** Add the three RCC members that sit between CFGR and AHBENR.
- **INGREDIENTS:** CIR 0x08, APB2RSTR 0x0C, APB1RSTR 0x10 (cheat sheet).
- **HOW:** one `volatile uint32_t NAME;` per line, in this order.

member	offset	after
`CIR`	0x08	`CFGR`

member	offset	after
APB2RSTR	0x0C	CIR
APB1RSTR	0x10	APB2RSTR

- **WHERE:** include/stm32f103.h, struct RCC_TypeDef, between CFGR and AHBENR. Order matters: the offset is just the position.
- **CHECK:** make t T=m1_01 -> test_rcc_layout still FAILS, but now the message talks about CSR (that is STEP 4).

STEP 4 · TODO M1-01/4 · RCC struct, BDCR · [src: M1-01 step 4]

- **WHAT:** Add BDCR between APB1ENR and CSR.
- **INGREDIENTS:** BDCR at 0x20.
- **HOW:** volatile uint32_t BDCR; /* 0x20 */

member	offset	after
BDCR	0x20	APB1ENR

- **WHERE:** include/stm32f103.h, RCC_TypeDef.
- **CHECK:** make t T=m1_01 -> test_rcc_layout:PASS.

STEP 5 · TODO M1-01/5 · two bit names · [src: M1-01 step 5]

- **WHAT:** Name the GPIOC clock bit and the LED pin.
- **INGREDIENTS:** IOPCEN = bit 4, LED_PIN = 13. BIT(n) is given.
- **HOW:** #define NAME value

name	value	why
RCC_APB2ENR_IOPCEN	BIT(4)	clock gate for GPIOC
LED_PIN	13u	the LED is on PC13

- **WHERE:** include/stm32f103.h, under /* ---------- bit names ---------- */. Delete the two 0u placeholders.
- **CHECK:** make t T=m1_01 -> test_bit_names:PASS.

STEP 6 · TODO M1-01/6 · field_set · card C1 · [src: M1-01 step 6, card C1]

- **WHAT:** Write the function that changes one field of a value without touching the other bits.
- **INGREDIENTS:** mask = (1u << width) - 1u.
- **HOW:** (the C1 pattern, on a local variable)

```
mask = (1u << WIDTH) - 1u;
reg &= ~(mask << SHIFT);
reg |= (VALUE & mask) << SHIFT;
return reg;
```

reg	shift	width	value	result
0x44444444	20	4	2	0x44244444
0xFFFFFFFF	4	4	0	0xFFFFFF0F
0x000000F0	4	2	1	0x000000D0

- **WHERE:** src/drills.c, function field_set. The stub body (return reg;) is yours to replace.
- **CHECK:** make t T=m1_01 -> test_field_set:PASS.

STEP 7 · TODO M1-01/7 · pin_shift · card C2 · [src: M1-01 step 7, card C2]

- **WHAT:** Find where a pin's nibble starts inside CRL or CRH.
- **INGREDIENTS:** 8 pins per register, 4 bits per pin.
- **HOW:** return (PIN % 8u) * 4u;

pin	shift
0	0
7	28
8	0
13	20

- **WHERE:** src/drills.c, pin_shift.
- **CHECK:** make t T=m1_01 -> test_pin_shift:PASS.

STEP 8 · TODO M1-01/8 · `gpio_cfg` · cards C1 + C2 · [src: M1-01 step 8, cards C1+C2]

- **WHAT:** Write one nibble into the right setup register of a port.
- **INGREDIENTS:** `pin_shift` (step 7), `field_set` (step 6), CRL for pins 0 to 7, CRH for pins 8 to 15.
- **HOW:**

```
volatile uint32_t *r = (PIN < 8u) ? &g->CRL : &g->CRH;     /* & = address of the register */
*r = field_set(*r, pin_shift(PIN), 4u, NIBBLE);             /* * = the value stored there */
```

call	register	shift	width	value
`gpio_cfg(&g, 13, 2)`	CRH	20	4	2
`gpio_cfg(&g, 0, 3)`	CRL	0	4	3
`gpio_cfg(&g, 9, 0xB)`	CRH	4	4	0xB

This is card C1 again, with width 4 every time. This is also the pointer drill: `r` holds an address (`&`), `*r` reaches the register.

- **WHERE:** `src/drills.c`, `gpio_cfg`.
- **CHECK:** `make t T=m1_01` -> `test_gpio_cfg:PASS`.

STEP 9 · TODO M1-01/9 · `led_init` · card C4 · [src: M1-01 step 9, cards C2+C4]

- **WHAT:** Set up PC13 as an output and leave the LED off.
- **INGREDIENTS:** clock `RCC_APB2ENR_IOPCEN` in `RCC->APB2ENR`; pin `LED_PIN` nibble 0x2 on `GPIOC`.
- **HOW:** (card C4: clock, pin)

```
RCC->APB2ENR |= <CLOCK BIT>;
led_off();
gpio_cfg(<PORT>, <PIN>, <NIBBLE>);
```

step	value
clock bit	`RCC_APB2ENR_IOPCEN`
port, pin, nibble	`GPIOC`, `LED_PIN`, `0x2u`

`led_off()` first, so the pin is already HIGH (LED off) when it becomes an output. No flash of light.

- **WHERE:** `src/led.c`, `led_init`.
- **CHECK:** `make t T=m1_01` -> `test_led_init` still fails, now on the line `after init the LED must be OFF`. That is correct until STEP 10.

STEP 10 · TODO M1-01/10 · `led_on`, `led_off` · [src: M1-01 step 10]

- **WHAT:** Switch the LED with one BSRR write each.
- **INGREDIENTS:** on = pin LOW = high half of BSRR = `BIT(LED_PIN + 16u)`. off = pin HIGH = `BIT(LED_PIN)`.
- **HOW:** `GPIOC->BSRR = <BIT>;`

function	BSRR value	result
`led_on`	`BIT(LED_PIN + 16u)`	0x20000000
`led_off`	`BIT(LED_PIN)`	0x00002000

- **WHERE:** `src/led.c`.
- **CHECK:** `make t T=m1_01` -> `test_led_on_off:PASS` and `test_led_init:PASS`.

STEP 11 · TODO M1-01/11 · `led_toggle` · [src: M1-01 step 11]

- **WHAT:** Flip the LED.
- **INGREDIENTS:** ODR bit 13 shows the level you wrote last. If it is 1 the pin is HIGH (LED off).
- **HOW:** `if (GPIOC->ODR & BIT(LED_PIN)) led_on(); else led_off();`

ODR bit 13	LED now	call
1	off	`led_on()`
0	on	`led_off()`

- **WHERE:** `src/led.c`.
- **CHECK:** `make t T=m1_01` -> `test_led_toggle:PASS`, and the last lines `10 Tests 0 Failures 0 Ignored` / `OK`.

STEP 12 · finish (no TODO)

- Add the line `test_m1_01` to `tests.active`. Run `make test-active`. Commit: `git add -A && git commit -m "M1-01 done"`. Push when you next connect to the internet.

Verify

Do this	Expect
`make t T=m1_01`	10 lines ending `:PASS`, then `10 Tests 0 Failures 0 Ignored` and `OK`
`make test-active`	the same, nothing red
`git log --oneline`	two commits: skeleton and M1-01 done

If it fails, look here	Likely cause
`test_gpio_layout` or `test_rcc_layout` red	a member is missing, misspelled, or in the wrong order. Offsets are positions.
`test_gpio_cfg` red, value close	you forgot to clear the field first (card C1 step 1)
`test_led_*` red with wrong BSRR value	`BIT(13)` vs `BIT(13 + 16)` swapped. High half clears the pin = LED on.
build error `GPIOC undeclared`	`GPIOC` comes from the header: did STEP 1 or 2 break a bracket?
`make: *** No rule to make target build/test_m1_01`	you are not in the repo root

Self-check (answer aloud)

1. What does `&x` give you? What does `*p` give you?
2. Why is `volatile` on every register member?
3. Which register holds the setup nibble of pin 9? At what shift?
4. Why is writing BIT(13 + 16) to BSRR the way to turn the LED ON?

Hint ladder (open one at a time)

1. For STEP 8, think of card C2: choose the register, then card C1 with width 4.
2. Pattern: `volatile uint32_t *r = (pin < 8u) ? &g->CRL : &g->CRH;` then `*r = field_set(*r, <SHIFT FUNCTION>(pin), <WIDTH>, nibble);`
3. Template with the values: `*r = field_set(*r, <A>(pin), <B>, nibble);` where `<A>` = `pin_shift` and `<B>` = `4u`.

Title: M1-02 Linker script + startup code + cross Makefile

Milestone: M1 Foundation

Labels: m1, blank-level-1, cross-build

M1-02 Linker script + startup code + cross Makefile

START HERE

1. Type `make t T=m1_02`. You see red tests.
2. Open `src/memutil.c`. Find `@TODO(M1-02/1)`. It is a 4-line loop with walking pointers: STEP 1.
3. Later you will open `ld/stm32f103c6.ld` (STEP 5) and `Makefile` (STEP 10).

Starts from / Ends with

Starts from	Ends with
M1-01 green: `field_set`, `gpio_cfg`, `led_*`, `tests.active` has `test_m1_01`.	`copy_words`, `zero_words` tested on your PC.
	`Reset_Handler` and a vector table that you wrote.
	A linker script with FLASH 32K and RAM 10K.
	`make firmware` produces `build/firmware.elf` and `.bin`.
	`test_m1_02` in `tests.active`.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `field_set`, `pin_shift`, `gpio_cfg`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Card C1 Change a field (clear with a mask, then set)

```
REG &= ~(MASK << SHIFT);     /* 1. clear the field  */
REG |=  (VALUE << SHIFT);    /* 2. write the value  */
MASK = (1 << WIDTH) - 1      /* width 1 -> 0x1, 2 -> 0x3, 3 -> 0x7, 4 -> 0xF */
In code:  reg = field_set(reg, SHIFT, WIDTH, VALUE);
```

Erase one box on a paper form, then write in the box. If you skip the erase, old ink and new ink mix.

```
field_set(0x44444444, 20, 4, 2)
  clear bits 23..20 -> 0x44044444
  set 2 in them     -> 0x44244444
```

register value	shift	width	value	result
0x44444444	20	4	2	0x44244444
0xFFFFFFFF	4	4	0	0xFFFFFF0F
0x00000000	8	3	5	0x00000500

Card C2 GPIO pin nibble (CRL or CRH)

```
shift = (pin % 8) * 4          /* each pin owns 4 bits */
register = (pin < 8) ? CRL : CRH
gpio_cfg(GPIOx, pin, NIBBLE);   /* = C1 with width 4 */
```

A hotel corridor: 8 rooms per floor, 4 bits per room. Pins 0-7 are on floor CRL, pins 8-15 on floor CRH.

```
PC13: 13 % 8 = 5, shift = 5 * 4 = 20, register CRH.
PA2 : shift = 2 * 4 = 8,  register CRL.
```

nibble	meaning
0x0	analog input
0x4	floating input
0x8	input with pull-up or pull-down (ODR bit picks which: 1 = pull-up)

nibble	meaning
0x2	output, push-pull, 2 MHz
0x3	output, push-pull, 50 MHz
0xA	alternate function, push-pull, 2 MHz
0xB	alternate function, push-pull, 50 MHz
0xF	alternate function, open-drain, 50 MHz

Learn (about 30 minutes)

What happens at power-on. The CPU reads two words from the start of flash. Word 0 becomes the stack pointer. Word 1 is the address of the first function to run: `Reset_Handler`. That list of addresses is the **vector table**.

Why Reset_Handler exists. C expects two things before `main`: variables with start values (like `int x = 42;`) already in RAM, and zeroed variables set to 0. RAM is empty at power-on. So `Reset_Handler` is the moving crew: it copies the start values from flash into RAM (`.data`), zeroes the rest (`.bss`), then calls `main`.

The linker script is a floor plan. It says which room (FLASH or RAM) each piece lives in. `.data` is special: it lives in RAM while the program runs, but its start values are STORED in flash. Two addresses: where it runs (VMA, in RAM) and where it is stored (LMA, in flash). `> RAM AT> FLASH` says exactly that.

Linker symbols are addresses, not variables. `_sdata` marks a place. In C you write `&_sdata` to get that place. This is your `&` practice again.

No libc. You build with `-nostdlib`, so there is no `memcpy`. You write your own word copy and zero loops. `-lgcc` at the end of the link line provides helper code such as 64-bit divide. `-fno-tree-loop-distribute-patterns` stops GCC turning your loops back into `memcpy` calls.

Thumb bit. Cortex-M runs Thumb code only. Function addresses in the vector table must have bit 0 set. The compiler and linker do this for you, so word 1 looks odd (like ...45).

Worked example: walking pointers and a custom section

```
/* WORKED EXAMPLE (M1-02): walking pointers + a custom linker section. */
#include <stdint.h>

/* Add up every word from p up to (not including) end. */
uint32_t sum_words(const uint32_t *p, const uint32_t *end)
{
    uint32_t s = 0;
    while (p < end) {
        s += *p;        /* read the word p points at */
        p++;            /* move to the next word     */
    }
    return s;
}

/* A table the linker puts in its own section called .cfg (in Flash). */
__attribute__((section(".cfg"), used))
const uint32_t cfg_words[3] = { 10, 20, 30 };

extern const uint32_t _scfg, _ecfg;      /* symbols the linker script creates */

uint32_t cfg_total(void)
{
    return sum_words(&_scfg, &_ecfg);    /* & turns a symbol into its address */
}
```

The linker fragment that creates `_scfg` and `_ecfg` (a section called `.cfg`):

```
  .cfg :
  {
    . = ALIGN(4);
    _scfg = .;            /* start of the section */
    KEEP(*(.cfg))
    _ecfg = .;            /* end of the section   */
  } > FLASH
```

`sum_words` is the shape of both `copy_words` and `zero_words`: `while (p < end) { ... p++; }`.

Cheat sheet

Item	Value
FLASH	`FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 32K`
RAM	`RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 10K`
`_estack`	`ORIGIN(RAM) + LENGTH(RAM)` = 0x20002800
Symbols the startup code needs	`_sidata` (where .data is stored in flash), `_sdata`, `_edata` (where it runs in RAM), `_sbss`, `_ebss`
`_sidata`	`LOADADDR(.data)`
.text goes	`> FLASH`; contains `*(.text*)` and `*(.rodata*)`
.data goes	`> RAM AT> FLASH`
.bss goes	`> RAM`; contains `*(.bss*)` and `*(COMMON)`
`.isr_vector`	first in FLASH, `KEEP(*(.isr_vector))` (given)
Vector table (16 slots)	0 stack pointer, 1 Reset, 2 NMI, 3 HardFault, 4 MemManage, 5 BusFault, 6 UsageFault, 7-10 reserved, 11 SVCall, 12 DebugMon, 13 reserved, 14 PendSV, 15 SysTick
Reset order	copy .data, zero .bss, call main
Compiler flags	`-mcpu=cortex-m3 -mthumb -Os -g -std=gnu11 -Wall -Wextra -ffreestanding -fno-tree-loop-distribute-patterns -ffunction-sections -fdata-sections -Iinclude`
Linker flags	`-mcpu=cortex-m3 -mthumb -nostdlib -T ld/stm32f103c6.ld -Wl,-Map=build/firmware.map -Wl,--gc-sections`
`objcopy`	`arm-none-eabi-objcopy -O binary in.elf out.bin`

Cookbook steps

STEP 0 · set up (no TODO)

- Make sure M1-01 is committed. Type `make t T=m1_02`.
- **CHECK:** red tests; the linker, startup and Makefile tests show hints like `(M1-02 step 5)`.

STEP 1 · TODO M1-02/1 · `copy_words` · [src: M1-02 step 1, worked example sum_words]

- **WHAT:** Copy words from flash to RAM.
- **INGREDIENTS:** `dst` (where to write), `dst_end` (stop here), `src` (where to read).
- **HOW:** the sum_words shape:

```
while (<DST> < <DST_END>) {
    *<DST> = *<SRC>;     /* copy one word */
    <DST>++;             /* move both pointers on */
    <SRC>++;
}
```

name	role
`dst`, `dst_end`	`dst` walks up until it reaches `dst_end`
`src`	walks up in step with `dst`

- **WHERE:** `src/memutil.c`, `copy_words`. Delete the `(void)...` stub line.
- **CHECK:** `make t T=m1_02` -> `test_copy_words:PASS` and `test_copy_zero_length:PASS`.

STEP 2 · TODO M1-02/2 · `zero_words` · [src: M1-02 step 2]

- **WHAT:** Set a range of words to 0.
- **INGREDIENTS:** `start`, `end`.
- **HOW:** `while (<START> < <END>) { *<START> = 0u; <START>++; }`
- **WHERE:** `src/memutil.c`, `zero_words`.
- **CHECK:** `make t T=m1_02` -> `test_zero_words:PASS`.

STEP 3 · TODO M1-02/3 · `Reset_Handler` · [src: M1-02 step 3]

- **WHAT:** Prepare RAM, then start the program.
- **INGREDIENTS:** `copy_words(&_sdata, &_edata, &_sidata)`, `zero_words(&_sbss, &_ebss)`, `main()`.
- **HOW:** three calls, in this order.

order	call	meaning
1	`copy_words(&_sdata, &_edata, &_sidata);`	flash copy of .data -> RAM
2	`zero_words(&_sbss, &_ebss);`	clear .bss
3	`main();`	run the program

The `&` gives the address of each linker symbol.

- **WHERE:** `src/startup.c`, `Reset_Handler`, above the `for (;;)`.
- **CHECK:** `make t T=m1_02` -> `test_startup` still fails (vector slots come next).

STEP 4 · TODO M1-02/4 · three vector slots · [src: M1-02 step 4]

- **WHAT:** Put the stack address, the reset handler and the SysTick handler into the table.
- **INGREDIENTS:** slot 0 = `(isr_t)&_estack`, slot 1 = `Reset_Handler`, slot 15 = `SysTick_Handler`.
- **HOW:** replace each `0,` placeholder with the entry (keep the comma).

slot	replace `0,` with
0	`(isr_t)&_estack,`
1	`Reset_Handler,`
15	`SysTick_Handler,`

`(isr_t)` is a cast: it makes an address fit in a table of function pointers.

- **WHERE:** `src/startup.c`, array `vectors`.
- **CHECK:** `make t T=m1_02` -> `test_startup:PASS`.

STEP 5 · TODO M1-02/5 · ENTRY and MEMORY · [src: M1-02 step 5]

- **WHAT:** Start the linker script: the entry point and the two memory regions.
- **INGREDIENTS:** cheat sheet rows FLASH and RAM.
- **HOW:**

```
ENTRY(Reset_Handler)
MEMORY
{
  FLASH (rx)  : ORIGIN = <ADDR>, LENGTH = <SIZE>
  RAM   (rwx) : ORIGIN = <ADDR>, LENGTH = <SIZE>
}
```

region	ORIGIN	LENGTH
FLASH	`0x08000000`	`32K`
RAM	`0x20000000`	`10K`

- **WHERE:** `ld/stm32f103c6.ld`, at the top (the TODO comment sits there).
- **CHECK:** `make t T=m1_02` -> `test_linker_script` still fails, now at the `_estack` line.

STEP 6 · TODO M1-02/6 · `_estack` · [src: M1-02 step 6]

- **WHAT:** Create the symbol for the end of RAM, the first stack address.
- **INGREDIENTS:** `ORIGIN(RAM) + LENGTH(RAM)`.
- **HOW:** `_estack = ORIGIN(RAM) + LENGTH(RAM);` (a symbol assignment ends with `;`)
- **WHERE:** `ld/stm32f103c6.ld`, after MEMORY, before SECTIONS.
- **CHECK:** `make t T=m1_02` -> failure moves on to the `.text` line.

STEP 7 · TODO M1-02/7 · `.text` · [src: M1-02 step 7]

- **WHAT:** Say where code and read-only data go.
- **INGREDIENTS:** `*(.text*)`, `*(.rodata*)`, `> FLASH`.
- **HOW:** copy the shape of the given `.isr_vector` block:

```
.text :
{
  *(.text*)
  *(.rodata*)
  . = ALIGN(4);
} > FLASH
```

- **WHERE:** `ld/stm32f103c6.ld`, inside SECTIONS after `.isr_vector`.
- **CHECK:** `make t T=m1_02` -> failure moves on to `_sidata`.

STEP 8 · TODO M1-02/8 · `.data` (RAM, stored in FLASH) · [src: M1-02 step 8]

- **WHAT:** Say that `.data` runs in RAM but is stored in flash, and mark its edges.
- **INGREDIENTS:** `_sidata = LOADADDR(.data);`, `_sdata`, `_edata`, `> RAM AT> FLASH`.
- **HOW:**

```
_sidata = LOADADDR(.data);
.data :
{
  . = ALIGN(4);
  _sdata = .;        /* "." means "the address right here" */
  *(.data*)
  . = ALIGN(4);
  _edata = .;
} > RAM AT> FLASH
```

symbol	placed	startup code uses it as
`_sidata`	flash copy	the source
`_sdata`	start in RAM	`dst`
`_edata`	end in RAM	`dst_end`

- **WHERE:** `ld/stm32f103c6.ld`, after `.text`.
- **CHECK:** `make t T=m1_02` -> failure moves on to `_sbss`.

STEP 9 · TODO M1-02/9 · `.bss` · [src: M1-02 step 9]

- **WHAT:** Say where zeroed variables go and mark both edges.
- **INGREDIENTS:** `_sbss`, `_ebss`, `*(.bss*)`, `*(COMMON)`, `> RAM`.
- **HOW:** the same shape as `.data`, but only `> RAM`.

symbol	startup code uses it as
`_sbss`	`start` of `zero_words`
`_ebss`	`end` of `zero_words`

- **WHERE:** `ld/stm32f103c6.ld`, after `.data`.
- **CHECK:** `make t T=m1_02` -> `test_linker_script:PASS`.

STEP 10 · TODO M1-02/10 · Makefile variables · [src: M1-02 step 10]

- **WHAT:** Name the compiler and the flags once.
- **INGREDIENTS:** cheat sheet rows "Compiler flags" and "Linker flags".
- **HOW:** `NAME = value`, one per line.

variable	value
`CROSS`	`arm-none-eabi-`
`CPU`	`-mcpu=cortex-m3 -mthumb`
`CFLAGS`	`$(CPU) -Os -g -std=gnu11 -Wall -Wextra -ffreestanding -fno-tree-loop-distribute-patterns -ffunction-sections -fdata-sections -Iinclude`
`LDFLAGS`	`$(CPU) -nostdlib -T ld/stm32f103c6.ld -Wl,-Map=build/firmware.map -Wl,--gc-sections`
`FW_SRC`	`$(wildcard src/*.c)`

- **WHERE:** `Makefile`, section "firmware".
- **CHECK:** `make t T=m1_02` -> `test_makefile` still fails (rules come next).

STEP 11 · TODO M1-02/11 · Makefile rules · [src: M1-02 step 11; worked example in M1-03 style]

- **WHAT:** Teach make to build the ELF and the BIN.
- **INGREDIENTS:** `$(CROSS)gcc`, `-lgcc` last, `$(CROSS)objcopy -O binary`, `$(CROSS)size`.
- **HOW:** a recipe line starts with a real TAB.

```
build/firmware.elf: $(FW_SRC) $(wildcard include/*.h) ld/stm32f103c6.ld
<TAB>@mkdir -p build
<TAB>$(CROSS)gcc $(CFLAGS) $(FW_SRC) $(LDFLAGS) -lgcc -o $@

build/firmware.bin: build/firmware.elf
<TAB>$(CROSS)objcopy -O binary $< $@

firmware: build/firmware.elf build/firmware.bin
```

```
size: build/firmware.elf
<TAB>$(CROSS)size $<
```

(<TAB> means press the Tab key. $@ is the target, $< the first dependency.)

- **WHERE:** Makefile, after the variables.
- **CHECK:** make t T=m1_02 -> 6 Tests 0 Failures and OK. Then make firmware builds with no errors (the blink code comes in M1-03).

STEP 12 · finish (no TODO)

- Add test_m1_02 to tests.active, make test-active, commit M1-02 done.

Verify

Do this	Expect
make t T=m1_02	6 Tests 0 Failures 0 Ignored, OK
make firmware	no errors; files build/firmware.elf and build/firmware.bin appear
arm-none-eabi-objdump -s -j .isr_vector build/firmware.elf	first word reads 00280020 (= 0x20002800, little-endian). Second word: its first byte is ODD (for example 45), because of the Thumb bit. The 4th byte of every handler word is 08 (flash).
arm-none-eabi-nm build/firmware.elf | grep -E " _estack| _sdata| _edata| _sbss| _ebss| Reset_Handler"	_estack is 20002800; _sdata and _edata start with 2000; Reset_Handler starts with 0800
make size	text a few hundred bytes up to about 1 KB, data 0 to a few bytes, bss 0 (nothing uses the big buffers yet). These sizes are rough estimates: I could not run the ARM size tool while writing this kit. The hard limits are: text + data under 32768 and data + bss under 10240.

If it fails, look here	Likely cause
arm-none-eabi-gcc: not found	the NixOS package is not installed (see the checklist)
undefined reference to __aeabi_uldivmod	-lgcc missing, or not at the END of the link line
undefined reference to memcpy	-fno-tree-loop-distribute-patterns missing
*** missing separator	the recipe line starts with spaces, not a TAB
region FLASH overflowed	not expected here; check LENGTH = 32K
word 1 of the vector table is even	slot 1 is not the function name Reset_Handler

Self-check (answer aloud)

1. What are the first two words in flash, and who reads them?
2. Why must .data be copied, and what stays in flash?
3. What does &_sdata mean when _sdata is a linker symbol?
4. What does > RAM AT> FLASH say?

Hint ladder (open one at a time)

1. STEP 1 is the sum_words loop with a write instead of an add.
2. Pattern: while (A < B) { *A = *C; A++; C++; }
3. Template with the values: while (<DST> < <DST_END>) { *<DST> = *<SRC>; <DST>++; <SRC>++; } with <DST> = dst, <DST_END> = dst_end, <SRC> = src.

Title: M1-03 Flash with OpenOCD, blink, and the CI cross-build job

Milestone: M1 Foundation

Labels: m1, blank-level-1, hardware, ci

M1-03 Flash with OpenOCD, blink, and the CI cross-build job

START HERE

1. Open `src/main.c`. Find `@TODO(M1-03/1)`. You write a blink loop of about 6 lines: STEP 1.
2. Then open `Makefile` (STEP 2) and `.github/workflows/ci.yml` (STEP 4).
3. First wire the ST-Link (STEP 0).

Starts from / Ends with

Starts from	Ends with
M1-01 (`led_init`, `led_toggle`) and M1-02 (`make firmware` works).	The on-board LED blinks on real silicon.
	`make flash` works.
	`tests.active` lists `test_m1_01`, `test_m1_02`, `test_m1_03`.
	CI has a second job `firmware` that cross-builds.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `field_set`, `gpio_cfg`, `pin_shift`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Card C1 Change a field (clear with a mask, then set)

```
REG &= ~(MASK << SHIFT);     /* 1. clear the field  */
REG |=  (VALUE << SHIFT);    /* 2. write the value  */
MASK = (1 << WIDTH) - 1      /* width 1 -> 0x1, 2 -> 0x3, 3 -> 0x7, 4 -> 0xF */
In code:  reg = field_set(reg, SHIFT, WIDTH, VALUE);
```

Erase one box on a paper form, then write in the box. If you skip the erase, old ink and new ink mix.

```
field_set(0x44444444, 20, 4, 2)
  clear bits 23..20 -> 0x44044444
  set 2 in them     -> 0x44244444
```

register value	shift	width	value	result
0x44444444	20	4	2	0x44244444
0xFFFFFFFF	4	4	0	0xFFFFFF0F
0x00000000	8	3	5	0x00000500

Card C2 GPIO pin nibble (CRL or CRH)

```
shift = (pin % 8) * 4          /* each pin owns 4 bits */
register = (pin < 8) ? CRL : CRH
gpio_cfg(GPIOx, pin, NIBBLE);   /* = C1 with width 4 */
```

A hotel corridor: 8 rooms per floor, 4 bits per room. Pins 0-7 are on floor CRL, pins 8-15 on floor CRH.

```
PC13: 13 % 8 = 5, shift = 5 * 4 = 20, register CRH.
PA2 : shift = 2 * 4 = 8,  register CRL.
```

nibble	meaning
0x0	analog input
0x4	floating input
0x8	input with pull-up or pull-down (ODR bit picks which: 1 = pull-up)

nibble	meaning
0x2	output, push-pull, 2 MHz
0x3	output, push-pull, 50 MHz
0xA	alternate function, push-pull, 2 MHz
0xB	alternate function, push-pull, 50 MHz
0xF	alternate function, open-drain, 50 MHz

Learn (about 30 minutes)

The ST-Link is your key to the chip. It talks SWD (two wires, SWDIO and SWCLK) to the debug port inside the Cortex-M3. **OpenOCD** is the program on your PC that drives the ST-Link. Its command `program file.elf verify reset exit` writes flash, reads it back to compare, resets the chip, and quits.

Why `verify`? It reads the flash back and compares it. If a byte differs you hear about it.

Crude delay. Right now the chip runs on its internal 8 MHz oscillator. A loop that counts to a big number with a `volatile` counter wastes time. `volatile` stops the compiler deleting the empty loop. It is crude and temporary: in M1-04 you replace it with a real clock.

Two CI jobs. CI runs on GitHub every time you push. The `host-tests` job (given) runs your unit tests. You add a `firmware` job that installs the ARM compiler and runs `make firmware`. If either job turns red you know which half broke.

Power rule. The board gets power from the ST-Link 3.3 V pin. Never also plug USB.

Worked example: a small cross Makefile with a flash recipe

```
# A small cross Makefile for a program called "demo". Recipe lines start with a TAB.
CROSS  = arm-none-eabi-
CFLAGS = -mcpu=cortex-m3 -mthumb -Os -g -ffreestanding -Iinclude
LDFLAGS = -nostdlib -T link.ld

build/demo.elf: demo.c start.c link.ld
    @mkdir -p build
    $(CROSS)gcc $(CFLAGS) demo.c start.c $(LDFLAGS) -lgcc -o $@

size: build/demo.elf
    $(CROSS)size $<

flash: build/demo.elf
    openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c "program build/demo.elf verify reset exit"
```

Read it: one rule builds the ELF; `size` prints sizes; `flash` depends on the ELF and runs OpenOCD with the `program ... verify reset exit` command.

Cheat sheet

Item	Value
ST-Link wires	SWDIO, SWCLK, GND, 3V3 to the 4-pin header on the board
BOOT jumpers	both at 0
Probe the chip	`st-info --probe`
Flash command	`openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c "program build/firmware.elf verify reset exit"`
OpenOCD messages when all is well	`** Programming Finished **`, `** Verified OK **`, `** Resetting Target **`
Clone chips: unexpected id error	put `-c "set CPUTAPID 0x2ba01477"` FIRST on the command line
Blink loop shape	`for (;;) { led_toggle(); for (volatile uint32_t i = 0; i < N; i++) { } }`
Starting N	500000 (tune until about one toggle per second; not measured)
CI file: indent	job names sit 2 spaces in; `runs-on` 4; list items `-` 6
CI job shape	`name: runs-on: steps: - uses: actions/checkout@v4 - name: ... run: ...`
Install ARM compiler on the CI machine	`sudo apt-get update && sudo apt-get install -y gcc-arm-none-eabi`

Cookbook steps

STEP 0 · wire and probe (no TODO)

- Wire SWDIO, SWCLK, GND, 3V3 from the ST-Link to the board. Plug the ST-Link into the PC. Do NOT plug the board's USB.
- **CHECK:** `st-info --probe` prints a chip id and `flash: 32768` for a real C6 (a clone may print a bigger number: the plan still uses 32 KB).

STEP 1 · TODO M1-03/1 · blink in `main` · [src: M1-03 step 1]

- **WHAT:** Make the LED blink.
- **INGREDIENTS:** `led_init()`, `led_toggle()`, a `volatile` counter, the loop shape from the cheat sheet.
- **HOW:**

```
led_init();
for (;;) {
    led_toggle();
    for (volatile uint32_t i = 0; i < <N>; i++) { }
}
```

name	value
`<N>`	`500000` to start, then tune

- **WHERE:** `src/main.c`, inside `main`. Replace the stub line `for (;;) { }`. Leave the other TODO comments for later issues.
- **CHECK:** `make firmware` builds with no errors.

STEP 2 · TODO M1-03/2 · `flash` target · [src: M1-03 step 2; worked example above]

- **WHAT:** Add `make flash`.
- **INGREDIENTS:** the OpenOCD line from the cheat sheet. A recipe starts with a TAB.
- **HOW:**

```
flash: build/firmware.elf
<TAB>openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c "program build/firmware.elf verify reset exit"
```

- **WHERE:** `Makefile`, section "firmware", after the `size` target.
- **CHECK:** with the board wired, `make flash` ends with `** Verified OK **` and `** Resetting Target **`. The LED blinks.

STEP 3 · TODO M1-03/3 · `tests.active` · [src: M1-03 step 3]

- **WHAT:** Tell CI which tests to run.
- **INGREDIENTS:** the test names, one per line. You probably added the first two at the end of M1-01 and M1-02 already: then add only the third.
- **HOW:**

line to add	issue
`test_m1_01`	M1-01
`test_m1_02`	M1-02
`test_m1_03`	M1-03

- **WHERE:** `tests.active`. Lines starting with `#` are comments.
- **CHECK:** `make test-active` runs three test files and ends green. (`test_m1_03` turns green only after STEP 4.)

STEP 4 · TODO M1-03/4 · CI firmware job · [src: M1-03 step 4]

- **WHAT:** Add a second CI job that cross-builds.
- **INGREDIENTS:** the CI job shape from the cheat sheet.
- **HOW:** copy the `host-tests` job and change three things.

field	host-tests (given)	your `firmware` job
job name	`host-tests:`	`firmware:`
install step	none	`run: sudo apt-get update && sudo apt-get install -y gcc-arm-none-eabi`
run step	`make test-active`	`make firmware && make size`

- **WHERE:** `.github/workflows/ci.yml`, after the `host-tests` job, same indent level. YAML cares about spaces: use spaces, never tabs.
- **CHECK:** `make t T=m1_03` -> `4 Tests 0 Failures`, `OK`.

STEP 5 · finish (no TODO)

- `git add -A`, `git commit -m "M1-03 done"`, then `git push`. Open the Actions tab on GitHub: both jobs turn green (this is the one step that needs the internet).

Verify

Do this	Expect
`make t T=m1_03`	`4 Tests 0 Failures 0 Ignored`, `OK`
`make flash`	`** Verified OK **` and `** Resetting Target **`
Look at the board	the green/blue LED near PC13 blinks about once per second (tune N)
GitHub Actions after `git push`	jobs `host-tests` and `firmware` both green

If it fails, look here	Likely cause
`Error: init mode failed` or no device	the ST-Link wires: SWDIO/SWCLK swapped, or no common GND
`UNEXPECTED idcode`	clone chip: add `-c "set CPUTAPID 0x2ba01477"` first
`Permission denied` / `LIBUSB_ERROR_ACCESS`	udev rules not applied: log out and in, replug
LED never changes but flash says Verified	check `led_init()` is called; the LED is active-low, so toggling is what you want
LED blinks too fast to see	raise `<N>`
CI firmware job red: `arm-none-eabi-gcc: not found`	the install step is missing in the YAML
CI: `mapping values are not allowed here`	YAML indentation is wrong

Self-check (answer aloud)

1. What does `verify` in the OpenOCD command do?
2. Why is the counter `volatile`?
3. Why can the same repo have two CI jobs?
4. What power sources must never be connected together?

Hint ladder (open one at a time)

1. STEP 4: the `firmware` job is the `host-tests` job with an extra install step.
2. Pattern: `firmware:` / `runs-on: ubuntu-latest` / `steps:` / checkout / install / build.
3. Template with the values (2 spaces per level): job name `firmware:`; under `steps:` three list items: `uses: actions/checkout@v4`; `name: Install` with `run: sudo apt-get update && sudo apt-get install -y gcc-arm-none-eabi`; `name: Build` with `run: make firmware && make size`.

Title: M1-04 72 MHz clock and SysTick millis (plus wait_flag, elapsed, reached)

Milestone: M1 Foundation

Labels: m1, blank-level-1, hardware, drills

M1-04 72 MHz clock and SysTick millis (plus wait_flag, elapsed, reached)

START HERE

1. Type `make t T=m1_04`. Red tests.
2. Open `src/clock.c`. Find `@TODO(M1-04/1)`: the function `clock_cfgr_word` is four short lines and uses card C1 three times.
3. The three drill functions `wait_flag`, `elapsed`, `reached` live in `src/drills.c` (STEPS 7, 8, 9). Do them first if you like: `clock_init` needs `wait_flag`.

Starts from / Ends with

Starts from	Ends with
M1-03: board flashes, LED blinks, `field_set` and `gpio_cfg` work.	CPU runs at 72 MHz from the 8 MHz crystal.
	`millis()` counts real milliseconds from SysTick.
	`wait_flag` (C3), `elapsed` and `reached` (C6) tested.
	LED blinks every 500 ms with no busy-wait loop.
	`test_m1_04` in `tests.active`.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `pin_shift`, `gpio_cfg`, `field_set`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Card C2 GPIO pin nibble (CRL or CRH)

```
shift = (pin % 8) * 4          /* each pin owns 4 bits */
register = (pin < 8) ? CRL : CRH
gpio_cfg(GPIOx, pin, NIBBLE);   /* = C1 with width 4 */
```

A hotel corridor: 8 rooms per floor, 4 bits per room. Pins 0-7 are on floor CRL, pins 8-15 on floor CRH.

```
PC13: 13 % 8 = 5, shift = 5 * 4 = 20, register CRH.
PA2 : shift = 2 * 4 = 8,  register CRL.
```

nibble	meaning
0x0	analog input
0x4	floating input
0x8	input with pull-up or pull-down (ODR bit picks which: 1 = pull-up)
0x2	output, push-pull, 2 MHz
0x3	output, push-pull, 50 MHz
0xA	alternate function, push-pull, 2 MHz
0xB	alternate function, push-pull, 50 MHz
0xF	alternate function, open-drain, 50 MHz

Card C1 Change a field (clear with a mask, then set)

```
REG &= ~(MASK << SHIFT);     /* 1. clear the field  */
REG |=  (VALUE << SHIFT);    /* 2. write the value  */
MASK = (1 << WIDTH) - 1      /* width 1 -> 0x1, 2 -> 0x3, 3 -> 0x7, 4 -> 0xF */
In code:  reg = field_set(reg, SHIFT, WIDTH, VALUE);
```

Erase one box on a paper form, then write in the box. If you skip the erase, old ink and new ink mix.

```
field_set(0x44444444, 20, 4, 2)
  clear bits 23..20 -> 0x44044444
  set 2 in them     -> 0x44244444
```

register value	shift	width	value	result
0x44444444	20	4	2	0x44244444
0xFFFFFFFF	4	4	0	0xFFFFFF0F
0x00000000	8	3	5	0x00000500

Learn (about 30 minutes)

Clock tree. The chip starts on the slow internal oscillator (HSI, 8 MHz). To go fast: turn on the crystal (HSE, 8 MHz), wait until it is stable, then feed it into the PLL (a clock multiplier: x9 = 72 MHz), wait until the PLL locks, then switch the system clock to it. Each "wait until" is a flag, so each uses card C3 with a timeout.

Flash is slow. Above 48 MHz the flash cannot keep up, so you must insert 2 wait states BEFORE raising the clock. Skip this and the chip crashes.

Bus dividers. APB1 (slow bus) is limited to 36 MHz, so it divides by 2. Timers on APB1 get double the clock: 72 MHz. The ADC clock must be 14 MHz or less: 72 / 6 = 12 MHz.

SysTick is a down-counter inside the CPU. It counts from LOAD to 0, fires an interrupt, reloads. With LOAD = 71999 it counts 72000 ticks = 1 ms at 72 MHz. The interrupt handler adds 1 to a counter `g_ms`. `millis()` returns it.

Time never stops wrapping. `g_ms` is 32 bits: it wraps after about 49 days. Compare times with card C6 (`elapsed`, `reached`), never with `now >= deadline`.

Why no busy-wait? A delay loop blocks everything. Instead the loop checks "has my deadline passed?" and does other work meanwhile.

Worked example: 32 MHz from the internal oscillator, then SysTick

```
/* WORKED EXAMPLE (M1-04): 32 MHz from the internal oscillator, then a 1 ms SysTick. */
#include "stm32f103.h"
#include "drills.h"

static volatile uint32_t ms;
void SysTick_Handler(void) { ms++; }

int clock32_init(void)
{
    /* HSI (8 MHz) is already running. With PLLSRC = 0 the PLL input is HSI/2 = 4 MHz. */
    FLASH->ACR = field_set(FLASH->ACR, 0, 3, 1);                 /* 1 wait state: 24..48 MHz  */
    RCC->CFGR  = field_set(RCC->CFGR, RCC_CFGR_PLLMUL_Pos, 4, 6);/* PLLMUL 6 = x8 -> 32 MHz   */
    RCC->CR   |= RCC_CR_PLLON;
    if (wait_flag(&RCC->CR, RCC_CR_PLLRDY, 1, 100000u)) return -1;
    RCC->CFGR  = field_set(RCC->CFGR, RCC_CFGR_SW_Pos, 2, 2);    /* SW 2 = PLL                */
    if (wait_flag(&RCC->CFGR, RCC_CFGR_SWS_PLL, 1, 100000u)) return -1;

    SYSTICK->LOAD = 32000000u / 1000u - 1u;                      /* 31999 ticks = 1 ms        */
    SYSTICK->VAL  = 0;
    SYSTICK->CTRL = SYSTICK_ENABLE | SYSTICK_TICKINT | SYSTICK_CLKSRC;
    return 0;
}
```

Same recipe as yours, with different numbers: PLL source is HSI/2 (4 MHz), multiplier x8 = 32 MHz, 1 flash wait state, LOAD = 31999.

Cheat sheet

Item	Value
RCC CR bits	HSEON bit 16, HSERDY bit 17, PLLON bit 24, PLLRDY bit 25
RCC CFGR fields	SW: shift 0, width 2 (value 2 = PLL). SWS (status): bit 3 set when PLL is in use. PPRE1: shift 8, width 3 (value 4 = divide by 2). ADCPRE: shift 14, width 2 (value 2 = divide by 6). PLLSRC: bit 16 (1 = HSE). PLLMUL: shift 18, width 4 (value 7 = x9)
FLASH ACR	LATENCY: shift 0, width 3 (value 2 = two wait states). PRFTBE: bit 4 (prefetch enable)
FLASH base / ACR offset	0x40022000 / 0x00
SysTick base	0xE000E010: CTRL 0x00, LOAD 0x04, VAL 0x08

Item	Value
SysTick CTRL bits	ENABLE bit 0, TICKINT bit 1, CLKSOURCE bit 2 (1 = CPU clock)
SysTick LOAD	clock / tick_hz - 1 = 72000000 / 1000 - 1 = 71999
CFGR word this issue builds	PLLSRC `RCC_CFGR_PLLSRC_HSE` + PLLMUL 7 + PPRE1 4 + ADCPRE 2 = 0x001D8400 (SW is NOT in it)
Order of `clock_init`	HSEON, wait HSERDY, FLASH latency 2, CFGR = word, PLLON, wait PLLRDY, SW = 2, wait SWS, SysTick
Names in the header (given)	`RCC_CR_HSEON`, `RCC_CR_HSERDY`, `RCC_CR_PLLON`, `RCC_CR_PLLRDY`, `RCC_CFGR_SW_Pos`, `RCC_CFGR_SWS_PLL`, `RCC_CFGR_PPRE1_Pos`, `RCC_CFGR_ADCPRE_Pos`, `RCC_CFGR_PLLSRC_HSE`, `RCC_CFGR_PLLMUL_Pos`, `FLASH_ACR_LATENCY_Pos`, `FLASH_ACR_PRFTBE`, `SYSTICK_ENABLE`, `SYSTICK_TICKINT`, `SYSTICK_CLKSRC`, `SYSCLK_HZ`

Cards C1, C3, C6:

```
C1: reg = field_set(reg, SHIFT, WIDTH, VALUE);
C3: while (tries--) { if (flag is in the wanted state) return 0; }  return -1;
C6: elapsed = now - start;   reached = (int32_t)(now - deadline) >= 0;
```

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M1-03. Type `make t T=m1_04`.
- **CHECK:** `9 Tests` with failures; messages mention `(M1-04 step N)`.

STEP 7 · TODO M1-04/7 · `wait_flag` · card C3 · [src: M1-04 step 7, card C3]

(Numbering follows the TODO ids; do this step first.)

- **WHAT:** Wait for a register bit to reach a state, with a limit.
- **INGREDIENTS:** `reg` (pointer), `mask`, `want_set` (1 = wait for the bit to be 1, 0 = wait for 0), `tries`.
- **HOW:**

```
while (<TRIES>--) {
    int is_set = ((*<REG> & <MASK>) != 0u);
    if (is_set == (<WANT_SET> != 0)) return 0;
}
return -1;
```

case	result
bit already as wanted	0
never arrives, tries = 5	-1
tries = 0	-1 at once

`*reg` is the value at the register address (your `*` practice).

- **WHERE:** `src/drills.c`, `wait_flag`.
- **CHECK:** `make t T=m1_04` -> `test_wait_flag_set:PASS` and `test_wait_flag_clear:PASS`.

STEP 8 · TODO M1-04/8 · `elapsed` · card C6 · [src: M1-04 step 8, card C6]

- **WHAT:** How many ticks since `start`?
- **INGREDIENTS:** unsigned subtraction.
- **HOW:** `return <NOW> - <START>;`

now	start	result
100	40	60
5	0xFFFFFFF0	21

- **WHERE:** `src/drills.c`, `elapsed`.
- **CHECK:** `make t T=m1_04` -> `test_elapsed:PASS`.

STEP 9 · TODO M1-04/9 · `reached` · card C6 · [src: M1-04 step 9, card C6]

- **WHAT:** Has the deadline passed?
- **INGREDIENTS:** subtract first, then look at the sign (`int32_t`).
- **HOW:** `return (int32_t)(<NOW> - <DEADLINE>) >= 0;`

now	deadline	result
10	10	1
9	10	0
5	0xFFFFFFF0	1
0xFFFFFFF0	5	0

- **WHERE:** `src/drills.c`, `reached`.
- **CHECK:** `make t T=m1_04` -> `test_reached:PASS`.

STEP 1 · TODO M1-04/1 · `clock_cfgr_word` · card C1 · [src: M1-04 step 1, card C1]

- **WHAT:** Build the CFGR value for 72 MHz, as a number, without touching the chip.
- **INGREDIENTS:** PLLSRC bit, three fields (cheat sheet).
- **HOW:** start with the PLLSRC bit, then three times `w = field_set(w, SHIFT, WIDTH, VALUE);`

field	shift (name)	width	value	why
PLLMUL	`RCC_CFGR_PLLMUL_Pos` (18)	4	7	x9
PPRE1	`RCC_CFGR_PPRE1_Pos` (8)	3	4	APB1 / 2
ADCPRE	`RCC_CFGR_ADCPRE_Pos` (14)	2	2	ADC / 6

This is card C1 again, three times, with shift 18 width 4, then shift 8 width 3, then shift 14 width 2. Do NOT set SW here.

- **WHERE:** `src/clock.c`, `clock_cfgr_word`. Replace the stub `return 0;`.
- **CHECK:** `make t T=m1_04` -> `test_cfgr_word:PASS` (the word is 0x001D8400).

STEP 5 · TODO M1-04/5 · `tick_reload` and SysTick setup · [src: M1-04 step 5]

- **WHAT:** Compute the SysTick LOAD value, then (later in `clock_init`) start SysTick.
- **INGREDIENTS:** `clk_hz / tick_hz - 1`. SysTick registers `SYSTICK->LOAD`, `VAL`, `CTRL`.
- **HOW (the function):** `return <CLK> / <TICK> - 1u;`

clk_hz	tick_hz	result
72000000	1000	71999
32000000	1000	31999

HOW (inside `clock_init`, last lines before `return 0;`):

```
SYSTICK->LOAD = tick_reload(SYSCLK_HZ, 1000u);
SYSTICK->VAL  = 0u;
SYSTICK->CTRL = SYSTICK_ENABLE | SYSTICK_TICKINT | SYSTICK_CLKSRC;
```

- **WHERE:** `src/clock.c`, `tick_reload` and the end of `clock_init`.
- **CHECK:** `make t T=m1_04` -> `test_tick_reload:PASS`.

STEP 2 · TODO M1-04/2 · crystal on · card C3 · [src: M1-04 step 2, card C3]

- **WHAT:** Turn the 8 MHz crystal on and wait until it is stable.
- **INGREDIENTS:** `RCC_CR_HSEON`, `RCC_CR_HSERDY`.
- **HOW:**

```
RCC->CR |= <ON BIT>;
if (wait_flag(&RCC->CR, <READY BIT>, 1, WAIT_TRIES) != 0) return -1;
```

name	value
ON BIT	`RCC_CR_HSEON`
READY BIT	`RCC_CR_HSERDY`, want 1

`&RCC->CR` is the address of the CR register, as `wait_flag` wants a pointer.

- **WHERE:** `src/clock.c`, `clock_init`, at the TODO comment.
- **CHECK:** `make t T=m1_04` -> `test_clock_init_code` fails later, at the FLASH line (STEP 3).

STEP 3 · TODO M1-04/3 · flash wait states · card C1 · [src: M1-04 step 3, card C1]

- **WHAT:** Give the flash 2 wait states and switch prefetch on, BEFORE the clock goes up.
- **INGREDIENTS:** LATENCY shift 0 width 3 value 2; `FLASH_ACR_PRFTBE`.
- **HOW:** `FLASH->ACR = field_set(FLASH->ACR, <SHIFT>, <WIDTH>, <VALUE>) | <PREFETCH>;`

shift	width	value	prefetch
FLASH_ACR_LATENCY_Pos (0)	3	2	FLASH_ACR_PRFTBE

This is card C1 again, with shift 0 and width 3.

- **WHERE:** src/clock.c, clock_init, after STEP 2.
- **CHECK:** compile only: make t T=m1_04 still builds.

STEP 4 · TODO M1-04/4 · PLL and switch · cards C1 + C3 · [src: M1-04 step 4, cards C1+C3]

- **WHAT:** Configure the PLL, start it, wait for lock, then switch the system clock to it.
- **INGREDIENTS:** clock_cfgr_word(), RCC_CR_PLLON, RCC_CR_PLLRDY, SW shift 0 width 2 value 2, RCC_CFGR_SWS_PLL.
- **HOW:**

```
RCC->CFGR = clock_cfgr_word();
RCC->CR |= <PLLON>;
if (wait_flag(&RCC->CR, <PLLRDY>, 1, WAIT_TRIES) != 0) return -1;
RCC->CFGR = field_set(RCC->CFGR, <SW SHIFT>, <WIDTH>, <VALUE>);
if (wait_flag(&RCC->CFGR, <SWS>, 1, WAIT_TRIES) != 0) return -1;
```

name	value
PLLON / PLLRDY	RCC_CR_PLLON / RCC_CR_PLLRDY
SW	shift RCC_CFGR_SW_Pos (0), width 2, value 2
SWS	RCC_CFGR_SWS_PLL

- **WHERE:** src/clock.c, clock_init, after STEP 3.
- **CHECK:** make t T=m1_04 -> test_clock_init_code passes when STEP 5 lines are also in.

STEP 6 · TODO M1-04/6 · SysTick_Handler and millis · [src: M1-04 step 6]

- **WHAT:** Count milliseconds.
- **INGREDIENTS:** the static volatile uint32_t g_ms at the top of the file.
- **HOW:** SysTick_Handler: g_ms++; and millis: return g_ms;

function	body
SysTick_Handler	g_ms++;
millis	return g_ms;

- **WHERE:** src/clock.c.
- **CHECK:** make t T=m1_04 -> test_millis:PASS, then 9 Tests 0 Failures, OK.

STEP 10 · TODO M1-04/10 · blink with millis · card C6 · [src: M1-04 step 10, card C6]

- **WHAT:** Blink the LED every 500 ms with no delay loop.
- **INGREDIENTS:** clock_init(), led_init(), millis(), reached().
- **HOW:**

```
clock_init();
led_init();
uint32_t next = millis() + <PERIOD>;
for (;;) {
    if (reached(millis(), next)) {
        led_toggle();
        next += <PERIOD>;
    }
}
```

name	value
<PERIOD>	500u

- **WHERE:** src/main.c, main. Replace the crude delay from M1-03.
- **CHECK:** make flash. The LED toggles every half second. Count 60 toggles against a phone stopwatch: about 30 s.

STEP 11 · finish (no TODO)

- Add test_m1_04 to tests.active, run make test-active, commit M1-04 done.

Verify

Do this	Expect
`make t T=m1_04`	`9 Tests 0 Failures 0 Ignored`, `OK`
`make flash`, count 60 LED toggles with a phone stopwatch	30 s, give or take 1 s
GDB (see toolchain page): `x/wx 0x40021004` after the program runs	`0x001d840a` (CFGR: your word 0x001D8400 plus SW = 2 and SWS = 2 in bits 3:2)
GDB: `x/wx 0x40022000`	`0x00000032` (latency 2, prefetch on, prefetch status bit set); `0x12` is also fine
GDB: `print g_ms` twice, a second apart	the second value is about 1000 bigger

If it fails, look here	Likely cause
chip hangs or hard-faults right after flash	wait states set AFTER raising the clock, or wrong LATENCY value
LED blinks at the old crude speed	`main` still uses the delay loop
LED about 9 times slower	PLL not used: SW switch or `SWS` wait missing (running on the 8 MHz crystal)
`test_cfgr_word` red with the wrong value	check shifts: PLLMUL 18, PPRE1 8, ADCPRE 14
`clock_init` returns -1	crystal not detected: check the board has an 8 MHz crystal; raise `WAIT_TRIES`

Self-check (answer aloud)

1. Why must flash wait states be set before raising the clock?
2. What does SysTick LOAD = 71999 give you, and why minus one?
3. Why is `now >= deadline` wrong near the wrap?
4. What would happen if `wait_flag` had no `tries`?

Hint ladder (open one at a time)

1. STEP 1 is card C1 three times. Start with `w = RCC_CFGR_PLLSRC_HSE;`.
2. Pattern: `w = field_set(w, SHIFT, WIDTH, VALUE);` repeated.
3. Template with the values: line 1 `<SHIFT>`=`RCC_CFGR_PLLMUL_Pos`, `<WIDTH>`=4, `<VALUE>`=7; line 2 `RCC_CFGR_PPRE1_Pos`, 3, 4; line 3 `RCC_CFGR_ADCPRE_Pos`, 2, 2. Each line reads `w = field_set(w, <SHIFT>, <WIDTH>, <VALUE>);`

Title: M1-05 USART1 transmit, fmt_u32, and the UART banner

Milestone: M1 Foundation

Labels: m1, blank-level-1, hardware, drills

M1-05 USART1 transmit, fmt_u32, and the UART banner

START HERE

1. Type `make t T=m1_05`. Red tests.
2. Open `src/drills.c`, find `@TODO(M1-05/1)`: `brr_for` is ONE line.
3. Wire the USB-TTL adapter (STEP 0) before you flash.

Starts from / Ends with

Starts from	Ends with
M1-04: 72 MHz clock, `millis`, `wait_flag`.	`brr_for` tested.
	`fmt_u32` and `banner_build` tested on your PC.
	`uart_init`, `uart_putc`, `uart_puts` working on PA9.
	`picocom` shows `SCOPE v0.1 clk=72000000`.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `wait_flag`, `elapsed`, `reached`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Card C3 Wait for a flag with a timeout

```
while (tries--) {
    if ((*reg & MASK) is in the wanted state) return 0;   /* flag came */
}
return -1;                                                /* gave up   */
In code:  if (wait_flag(&REG, MASK, WANT_SET, TRIES)) { /* timed out */ }
```

Ring a doorbell up to N times. If nobody comes, walk away. Never ring forever.

```
wait_flag(&USART1->SR, USART_SR_TXE, 1, 100000u)   -> 0 when TXE became 1, -1 if not
wait_flag(&I2C1->SR2, I2C_SR2_BUSY, 0, 20000u)       -> waits for BUSY to become 0
```

call	returns
flag already in the wanted state	0
flag never arrives, tries = 5	-1 after 5 reads
tries = 0	-1 at once

Card C6 Wrap-safe time compare

```
elapsed = now - start;                    /* unsigned subtraction survives the wrap */
reached = (int32_t)(now - deadline) >= 0; /* sign of the difference                 */
NEVER write:  now >= deadline
```

A car odometer that rolls over from 999999 to 000000. The distance driven is still new minus old.

```
elapsed(5, 0xFFFFFFF0) = 21
reached(5, 0xFFFFFFF0) = 1      (the deadline was just before the wrap)
```

now	start / deadline	elapsed	reached
100	40	60	1
9	10	(wraps: huge)	0
5	0xFFFFFFF0	21	1

Learn (about 30 minutes)

UART sends bytes one bit at a time on one wire. No clock wire: both sides agree on a speed (the baud rate). 115200 baud = 115200 bits per second. A byte takes 10 bits (start, 8 data, stop).

Baud register. The chip divides its bus clock to make the bit clock: `BRR = pclk / baud`. USART1 is on APB2 (72 MHz): 72,000,000 / 115,200 = 625. Round to the nearest whole number.

Bring-up (card C4): clock (GPIOA and USART1), pin (PA9 = TX, alternate-function push-pull; PA10 = RX, floating input), configure (BRR), start (UE, TE, RE).

Sending a byte: wait until TXE (transmit data register empty) is 1, then write the byte to DR. That wait is card C3.

No printf. There is no C library. You write `fmt_u32` yourself: take the number apart with `% 10` (the digits come out backwards), then copy them out in reverse order.

Wires. Adapter RX goes to board PA9 (a transmitter talks into a receiver, so TX meets RX). Adapter TX to board PA10. GND to GND. Adapter power pin NOT connected.

Worked example: USART2 on PA2 and PA3

```
/* WORKED EXAMPLE (M1-05): USART2 on PA2 (TX) / PA3 (RX). USART2 hangs on the SLOWER bus (APB1, 36 MHz). */
#include "stm32f103.h"
#include "drills.h"

#define RCC_APB1ENR_USART2EN BIT(17)

void usart2_init(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;                 /* 1. clocks: GPIOA ...        */
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;               /*    ... and USART2           */
    gpio_cfg(GPIOA, 2, 0xB);                            /* 2. PA2 alternate push-pull  */
    gpio_cfg(GPIOA, 3, 0x4);                            /*    PA3 floating input       */
    USART2->BRR = brr_for(36000000u, 115200u);          /* 3. 36 MHz / 115200 = 313    */
    USART2->CR1 = USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;   /* 4. start            */
}

void usart2_putc(char c)
{
    if (wait_flag(&USART2->SR, USART_SR_TXE, 1, 100000u)) return;   /* 5. wait */
    USART2->DR = (uint32_t)(uint8_t)c;
}
```

Differences from yours: USART2 sits on the slower APB1 bus (36 MHz, BRR 313), and uses pins PA2 and PA3.

Cheat sheet

Item	Value
USART1 base	0x40013800
USART offsets	SR 0x00, DR 0x04, BRR 0x08, CR1 0x0C, CR2 0x10, CR3 0x14
SR bits	RXNE bit 5, TC bit 6, TXE bit 7
CR1 bits	RE bit 2, TE bit 3, UE bit 13
Clock bits in APB2ENR	GPIOA bit 2, USART1 bit 14
Pins	PA9 TX: nibble 0xB (alternate push-pull 50 MHz). PA10 RX: nibble 0x4 (floating input)
BRR	`(pclk + baud / 2) / baud`. 72 MHz, 115200 -> 625. 36 MHz, 115200 -> 313 (312.5 rounds up)
Banner text	`SCOPE v0.1 clk=` + `72000000` + CR LF (`\r\n`)
Names in the header (given)	`RCC_APB2ENR_IOPAEN`, `RCC_APB2ENR_USART1EN`, `USART_SR_TXE`, `USART_CR1_UE`, `USART_CR1_TE`, `USART_CR1_RE`, `SYSCLK_HZ`
picocom	`picocom -b 115200 /dev/ttyUSB0`; leave with Ctrl-A, Ctrl-X
fmt_u32 shape	collect `v % 10` digits into `tmp[]` (backwards), `v /= 10`, then copy `tmp` out in reverse; end with a 0 byte
Return value	every `fmt_` function returns the length (not counting the 0 byte)

Cookbook steps

STEP 0 · wire (no TODO)

- Wire: adapter RX to PA9, adapter TX to PA10, GND to GND. Power pin of the adapter unconnected. Read the safety page about the adapter's logic level.
- **CHECK:** `make t T=m1_05` runs (red).

STEP 1 · TODO M1-05/1 · `brr_for` · [src: M1-05 step 1]

- **WHAT:** Compute the baud divisor, rounded to the nearest whole number.
- **INGREDIENTS:** `(pclk + baud / 2) / baud` (adding half the divisor before dividing rounds to nearest).
- **HOW:** `return (<PCLK> + <BAUD> / 2u) / <BAUD>;`

pclk	baud	result
72000000	115200	625
36000000	115200	313
72000000	9600	7500

- **WHERE:** `src/drills.c`, `brr_for`.
- **CHECK:** `make t T=m1_05` -> `test_brr_for:PASS`.

STEP 2 · TODO M1-05/2 · `fmt_u32` · [src: M1-05 step 2]

- **WHAT:** Turn a number into text.
- **INGREDIENTS:** `% 10`, `/ 10`, a 10-character array (a uint32 has at most 10 digits).
- **HOW:**

```
char tmp[10];
unsigned n = 0, len = 0;
do {
    tmp[n++] = (char)('0' + (v % 10u));   /* last digit as a letter */
    v /= 10u;                              /* drop the last digit    */
} while (v != 0u);                         /* do-while: 0 still prints "0" */
while (n > 0u) out[len++] = tmp[--n];      /* copy out, reversed     */
out[len] = 0;
return len;
```

v	tmp (backwards)	out
123	'3','2','1'	"123"
0	'0'	"0"

- **WHERE:** `src/fmt.c`, `fmt_u32`. Remove the stub lines.
- **CHECK:** `make t T=m1_05` -> `test_fmt_u32:PASS`.

STEP 3 · TODO M1-05/3 · `banner_build` · [src: M1-05 step 3]

- **WHAT:** Build the banner text in a buffer.
- **INGREDIENTS:** `put_str(out, "text")` (given, returns the length), `fmt_u32(out + n, number)`.
- **HOW:**

```
unsigned n = put_str(out, "<TEXT 1>");
n += fmt_u32(out + n, <NUMBER>);
put_str(out + n, "<TEXT 2>");
```

name	value
TEXT 1	`SCOPE v0.1 clk=`
NUMBER	`SYSCLK_HZ`
TEXT 2	`\r\n`

`out + n` is "the address n characters after out".

- **WHERE:** `src/fmt.c`, `banner_build`.
- **CHECK:** `make t T=m1_05` -> `test_banner:PASS`.

STEP 4 · TODO M1-05/4 · `uart_init` · card C4 · [src: M1-05 step 4, cards C2+C4]

- **WHAT:** Bring USART1 up.
- **INGREDIENTS:** cheat sheet. `gpio_cfg` (M1-01), `brr_for` (STEP 1).
- **HOW:** card C4 in order:

```
RCC->APB2ENR |= <CLOCK BITS>;                /* 1 clock     */
```

```
gpio_cfg(GPIOA, <TX PIN>, <TX NIBBLE>);        /* 2 pins      */
gpio_cfg(GPIOA, <RX PIN>, <RX NIBBLE>);
USART1->BRR = brr_for(SYSCLK_HZ, <BAUD>);      /* 3 configure */
USART1->CR1 = <ENABLE BITS>;                   /* 4 start     */
```

name	value
CLOCK BITS	`RCC_APB2ENR_IOPAEN | RCC_APB2ENR_USART1EN`
TX pin, nibble	`9u`, `0xBu`
RX pin, nibble	`10u`, `0x4u`
BAUD	`115200u`
ENABLE BITS	`USART_CR1_UE | USART_CR1_TE | USART_CR1_RE`

This is `gpio_cfg` (card C2) with pin 9 nibble 0xB and pin 10 nibble 0x4.

- **WHERE:** `src/uart.c`, `uart_init`.
- **CHECK:** `make t T=m1_05` -> `test_uart_code` still fails at the `TXE` line.

STEP 5 · TODO M1-05/5 · `uart_putc` · card C3 · [src: M1-05 step 5, card C3]

- **WHAT:** Send one byte.
- **INGREDIENTS:** `USART_SR_TXE`, `wait_flag`, `USART1->DR`.
- **HOW:**

```
if (wait_flag(&USART1->SR, <FLAG>, 1, 100000u) != 0) return;   /* give up quietly */
USART1->DR = (uint32_t)(uint8_t)c;
```

name	value
FLAG	`USART_SR_TXE`

This is card C3 again, with flag TXE (bit 7) and 100000 tries.

- **WHERE:** `src/uart.c`, `uart_putc`.
- **CHECK:** `make t T=m1_05` -> `test_uart_code:PASS`.

STEP 6 · TODO M1-05/6 · `uart_puts` · [src: M1-05 step 6]

- **WHAT:** Send a whole string.
- **INGREDIENTS:** a string ends with a 0 byte; `uart_putc`.
- **HOW:** `while (*s) { uart_putc(*s); s++; }` (`*s` = the character s points at; `s++` = next character)
- **WHERE:** `src/uart.c`, `uart_puts`.
- **CHECK:** `make t T=m1_05` -> `5 Tests 0 Failures`, `OK`.

STEP 7 · TODO M1-05/7 · banner in `main` · [src: M1-05 step 7]

- **WHAT:** Print the banner once at start-up.
- **INGREDIENTS:** `clock_init`, `uart_init`, `banner_build`, `uart_puts`, a `char` array of 40.
- **HOW:** after `clock_init();` and `led_init();` add:

```
uart_init();
char b[40];
banner_build(b);
uart_puts(b);
```

- **WHERE:** `src/main.c`, `main`, before the blink loop.
- **CHECK:** `make flash`, then `picocom -b 115200 /dev/ttyUSB0`, press the board's reset: you see `SCOPE v0.1 clk=72000000`.

STEP 8 · finish (no TODO)

- Add `test_m1_05` to `tests.active`, `make test-active`, commit `M1-05 done`.

Verify

Do this	Expect
`make t T=m1_05`	`5 Tests 0 Failures 0 Ignored`, `OK`
`make flash`, `picocom -b 115200 /dev/ttyUSB0`, press reset	the line `SCOPE v0.1 clk=72000000`
keep picocom open	the LED still blinks (the blink loop is untouched)

If it fails, look here	Likely cause
nothing appears	TX/RX swapped, or no common GND
garbage characters	wrong baud: is the clock really 72 MHz? (check `x/wx 0x40021004` = `0x001d840a`)
`Permission denied` on `/dev/ttyUSB0`	user not in `dialout`: log out and in
the banner prints only after a long wait	`wait_flag` tries too small, or TXE wait missing
first letter lost	send happens before the clock is stable: call `clock_init()` first

Self-check (answer aloud)

1. Why is BRR = 625 at 72 MHz and 115200 baud?
2. Why does adapter RX connect to board TX (PA9)?
3. What does `do { ... } while (v != 0)` do for v = 0 that a plain `while` would not?
4. Which flag do you wait for before writing DR, and why?

Hint ladder (open one at a time)

1. STEP 4 is card C4 on a different peripheral: clock, pin, configure, start.
2. Pattern: clocks `|=`, two `gpio_cfg` lines, `BRR`, `CR1 = UE|TE|RE`.
3. Template with the values: `gpio_cfg(GPIOA, <P1>, <N1>); gpio_cfg(GPIOA, <P2>, <N2>);` with `<P1>`=9 `<N1>`=0xB `<P2>`=10 `<N2>`=0x4; then `USART1->BRR = brr_for(SYSCLK_HZ, <BAUD>);` with `<BAUD>`=115200.

Milestone M2: Sampling

ADC, timer-triggered DMA capture, PWM test signal.

Title: M2-01 ADC single read on PA2 (plus mul_div and smpr_field)

Milestone: M2 Sampling

Labels: m2, blank-level-2, hardware, drills

M2-01 ADC single read on PA2 (plus mul_div and smpr_field)

START HERE

1. Type `make t T=m2_01`. Red tests.
2. Open `src/drills.c`, find `@TODO(M2-01/1)`: `mul_div` is ONE line (card C5).
3. Then `smpr_field` (STEP 2), then `src/adc.c` (STEPS 3 to 5).

Starts from / Ends with

Starts from	Ends with
M1-05: 72 MHz clock, `millis`, UART printing, `wait_flag`, `fmt_u32`.	`mul_div` (C5) and `smpr_field` tested.
	`adc_init`, `adc_read`, `adc_to_mv` written.
	The pot (or GND / 3V3) shows up as numbers on the serial terminal.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `brr_for`, `wait_flag`, `field_set`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

`brr_for`: `BRR = (pclk + baud / 2) / baud /* 72 MHz, 115200 -> 625 */`

Card C3 Wait for a flag with a timeout

```
while (tries--) {
    if ((*reg & MASK) is in the wanted state) return 0;   /* flag came */
}
return -1;                                                /* gave up   */
In code:  if (wait_flag(&REG, MASK, WANT_SET, TRIES)) { /* timed out */ }
```

Card C1 Change a field (clear with a mask, then set)

```
REG &= ~(MASK << SHIFT);     /* 1. clear the field  */
REG |=  (VALUE << SHIFT);    /* 2. write the value  */
MASK = (1 << WIDTH) - 1      /* width 1 -> 0x1, 2 -> 0x3, 3 -> 0x7, 4 -> 0xF */
In code:  reg = field_set(reg, SHIFT, WIDTH, VALUE);
```

Learn (about 30 minutes)

An ADC measures a voltage as a number. 12 bits = 0 to 4095. 0 counts = 0 V, 4095 counts = 3.3 V. One count is about 0.8 mV.

Sample time. The ADC charges a tiny capacitor from the pin, then converts. A longer sample time suits a source that pushes weakly. Your pot (10k) wants a long time: we pick 41.5 ADC cycles (code 4). The register field is 3 bits per channel: SMPR2 for channels 0 to 9, SMPR1 for channels 10 to 17.

Speed limit. The ADC clock must be 14 MHz or less. 72 MHz divided by 6 = 12 MHz. One conversion = sample time + 12.5 cycles = 41.5 + 12.5 = 54 cycles = 4.5 microseconds.

Calibration. After switching the ADC on, wait a moment, then set CAL and wait for it to clear. It removes the chip's own offset error.

Starting a conversion in software. Set EXTTRIG = 1 with EXTSEL = 7 ("software start"), then set SWSTART each time. When the conversion is done, EOC (end of conversion) becomes 1. Reading DR gives the result and clears EOC.

Card C5: multiply first, divide last. counts to millivolts: `counts * 3300 / 4095`. If you divide first (`counts / 4095 * 3300`) the answer is nearly always 0. `mul_div` multiplies in 64 bits so nothing overflows.

PA2 is analog. Its pin nibble is 0x0. Never put more than 3.3 V on it.

Worked example: TIM2 as a polled one-second timer

```
/* WORKED EXAMPLE (M2-01): TIM2 as a one-second timer, read by polling. Same shape as an ADC read. */
#include "stm32f103.h"
#include "drills.h"

void tim2_start_1s(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;     /* 1. clock                          */
    TIM2->PSC = 7199u;                      /* 2. 72 MHz / 7200 = 10 kHz         */
    TIM2->ARR = 9999u;                      /*    10000 ticks = 1 second         */
    TIM2->EGR = TIM_EGR_UG;                 /*    load PSC now                   */
    TIM2->SR  = 0u;                         /*    clear the update flag          */
    TIM2->CR1 |= TIM_CR1_CEN;               /* 3. start                          */
}

int tim2_wait_1s(void)                      /* 4. wait for the update flag       */
{
    if (wait_flag(&TIM2->SR, BIT(0), 1, 50000000u)) return -1;
    TIM2->SR = 0u;                          /* 5. clear the flag by writing 0    */
    return 0;
}
```

Same shape as your ADC read: clock, configure, start, wait (card C3), clear.

Cheat sheet

Item	Value
ADC1 base	0x40012400
ADC offsets	SR 0x00, CR1 0x04, CR2 0x08, SMPR1 0x0C, SMPR2 0x10, SQR1 0x2C, SQR2 0x30, SQR3 0x34, DR 0x4C
ADC clock	`RCC->CFGR` ADCPRE: shift 14, width 2, value 2 (= divide by 6)
Clock bits in APB2ENR	GPIOA bit 2, ADC1 bit 9
PA2	CRL nibble 0x0 (analog input), shift 8
SMPR2 field for channel 2	shift `smpr_field(2)` = 6, width 3, value 4 (41.5 cycles)
Sample-time codes	0 = 1.5, 1 = 7.5, 2 = 13.5, 3 = 28.5, 4 = 41.5, 5 = 55.5, 6 = 71.5, 7 = 239.5 cycles
SQR3	field SQ1: shift 0, width 5, value 2 (channel 2). SQR1 length field stays 0 (one conversion)
CR2 bits	ADON bit 0, CAL bit 2, EXTSEL shift 17 width 3, EXTTRIG bit 20, SWSTART bit 22
EXTSEL = 7	software start
SR bit	EOC = bit 1
Order	clocks, ADCPRE, pin, SMPR2, SQR3, ADON, short wait, CAL + wait for CAL = 0, EXTSEL = 7 + EXTTRIG
Short wait	`for (volatile int i = 0; i < 1000; i++) { }`
counts to mV	`mul_div(raw, 3300u, 4095u)`
smpr_field	channel < 10 ? channel * 3 : (channel - 10) * 3
Names in the header (given)	`RCC_APB2ENR_IOPAEN`, `RCC_APB2ENR_ADC1EN`, `RCC_CFGR_ADCPRE_Pos`, `ADC_CR2_ADON`, `ADC_CR2_CAL`, `ADC_CR2_EXTSEL_Pos`, `ADC_CR2_EXTTRIG`, `ADC_CR2_SWSTART`, `ADC_SR_EOC`
Expect from wires	PA2 on GND: 0 to about 10 counts. PA2 on 3V3: about 4080 to 4095

Cards C1, C3, C5:

```
C1: reg = field_set(reg, SHIFT, WIDTH, VALUE);
C3: if (wait_flag(&REG, MASK, WANT_SET, TRIES)) { /* timed out */ }
C5: mul_div(a, b, c) = a * b / c in 64 bits
```

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M1-05. Type `make t T=m2_01`.
- **CHECK:** `5 Tests` with failures.

STEP 1 · TODO M2-01/1 · `mul_div` · card C5 · [src: M2-01 step 1, card C5]

- **WHAT:** Compute `a * b / c` without losing digits.
- **INGREDIENTS:** `uint64_t` for the product.
- **HOW:** `return (uint32_t)(((uint64_t)<A> * <B>) / <C>);`

a	b	c	result
4095	3300	4095	3300
2048	3300	4095	1650
1	1	2	0
0xFFFFFFFF	100	100	0xFFFFFFFF

- **WHERE:** `src/drills.c`, `mul_div`.
- **CHECK:** `make t T=m2_01` -> `test_mul_div:PASS`.

STEP 2 · TODO M2-01/2 · `smpr_field` · card C2 in disguise · [src: M2-01 step 2, card C2]

- **WHAT:** Find the shift of channel N's 3-bit sample-time field.
- **INGREDIENTS:** 3 bits per channel; channels 0 to 9 in SMPR2, 10 to 17 in SMPR1 (restart at 0).
- **HOW:** `return (<CH> < 10u) ? <CH> * 3u : (<CH> - 10u) * 3u;`

channel	shift	register
0	0	SMPR2
2	6	SMPR2
9	27	SMPR2
10	0	SMPR1
16	18	SMPR1

This is card C2 again, but with 3 bits per pin and the register switch at 10, not 8.

- **WHERE:** `src/drills.c`, `smpr_field`.
- **CHECK:** `make t T=m2_01` -> `test_smpr_field:PASS`.

STEP 3 · TODO M2-01/3 · `adc_init` · card C4 · [src: M2-01 step 3, cards C1+C2+C3+C4]

- **WHAT:** Bring the ADC up and calibrate it.
- **INGREDIENTS:** cheat sheet. All earlier drills.
- **HOW:** card C4 in order. Each line is a repeat:

```
RCC->APB2ENR |= <CLOCKS>;                                          /* C4 clock    */
RCC->CFGR = field_set(RCC->CFGR, <ADCPRE SHIFT>, 2u, <ADCPRE>);    /* C1          */
gpio_cfg(GPIOA, <PIN>, <NIBBLE>);                                  /* C2          */
ADC1->SMPR2 = field_set(ADC1->SMPR2, smpr_field(<CH>), 3u, <CODE>); /* C1          */
ADC1->SQR3  = field_set(ADC1->SQR3, 0u, 5u, <CH>);                 /* C1          */
ADC1->CR2 = ADC_CR2_ADON;                                          /* start       */
for (volatile int i = 0; i < 1000; i++) { }                        /* short wait  */
ADC1->CR2 |= ADC_CR2_CAL;
wait_flag(&ADC1->CR2, ADC_CR2_CAL, 0, 100000u);                    /* C3: CAL -> 0 */
ADC1->CR2 = field_set(ADC1->CR2, ADC_CR2_EXTSEL_Pos, 3u, <EXTSEL>) | ADC_CR2_EXTTRIG;
```

name	value
CLOCKS	`RCC_APB2ENR_IOPAEN | RCC_APB2ENR_ADC1EN`
ADCPRE shift / value	`RCC_CFGR_ADCPRE_Pos` (14) / 2
PIN / NIBBLE	`2u` / `0x0u`
CH	`2u`
CODE (41.5 cycles)	`4u`
EXTSEL	`7u`

This is card C1 again three times (shifts 14, 6, 0 with widths 2, 3, 5) and the last line a fourth time (shift 17 width 3 value 7).

- **WHERE:** `src/adc.c`, `adc_init`.
- **CHECK:** `make t T=m2_01` -> `test_adc_code` still fails at the `ADC_CR2_SWSTART` line (STEP 4).

STEP 4 · TODO M2-01/4 · `adc_read` · card C3 · [src: M2-01 step 4, card C3]

- **WHAT:** Do one conversion and return the number.

- **INGREDIENTS:** `ADC_CR2_SWSTART`, `ADC_SR_EOC`, `ADC1->DR`.
- **HOW:**

```
ADC1->CR2 |= <START>;
if (wait_flag(&ADC1->SR, <DONE>, 1, 100000u) != 0) return -1;
*out = (uint16_t)ADC1->DR;
return 0;
```

name	value
START	`ADC_CR2_SWSTART`
DONE	`ADC_SR_EOC`

`out` is a pointer to the caller's variable: `*out = ...` stores the result there. Return 0 for success, -1 for a timeout.

- **WHERE:** `src/adc.c`, `adc_read`.
- **CHECK:** `make t T=m2_01` -> `test_adc_code:PASS`.

STEP 5 · TODO M2-01/5 · `adc_to_mv` · card C5 · [src: M2-01 step 5, card C5]

- **WHAT:** Convert counts to millivolts.
- **INGREDIENTS:** 3300 mV full scale, 4095 counts.
- **HOW:** `return mul_div(<RAW>, <A>, <B>);`

raw	result
0	0
1023	824
2048	1650
4095	3300

`<A>` = `3300u`, `<B>` = `4095u`.

- **WHERE:** `src/adc.c`, `adc_to_mv`.
- **CHECK:** `make t T=m2_01` -> `5 Tests 0 Failures`, `OK`.

STEP 6 · TODO M2-01/6 · print readings · [src: M2-01 step 6]

- **WHAT:** Every 500 ms print `raw=... mv=...`.
- **INGREDIENTS:** `adc_init`, `adc_read`, `adc_to_mv`, `fmt_u32`, `uart_puts`, `reached`.
- **HOW:** extend the blink loop in `main` (UART and clock already started):

```
adc_init();
...
if (reached(millis(), next)) {
    uint16_t raw;
    char t[12];
    if (adc_read(&raw) == 0) {
        uart_puts("raw=");  fmt_u32(t, raw);             uart_puts(t);
        uart_puts(" mv=");  fmt_u32(t, adc_to_mv(raw));  uart_puts(t);
        uart_puts("\r\n");
    }
    led_toggle();
    next += 500u;
}
```

`&raw` gives `adc_read` the address of `raw`.

- **WHERE:** `src/main.c`, `main`.
- **CHECK:** `make flash`, `picocom`: lines like `raw=0 mv=0` appear every half second.

STEP 7 · finish (no TODO)

- Add `test_m2_01` to `tests.active`, `make test-active`, commit `M2-01 done`.

Verify

Do this	Expect
`make t T=m2_01`	`5 Tests 0 Failures 0 Ignored`, `OK`
Wire PA2 to GND (unplug USB first), flash	`raw` is 0 to about 10, `mv` 0 to about 8
Wire PA2 to 3V3	`raw` about 4080 to 4095, `mv` about 3290 to 3300

Do this	Expect
Wire the pot: outer pins to 3V3 and GND, middle to PA2; turn the knob	`raw` sweeps smoothly from near 0 to near 4095
Leave PA2 unconnected	readings wander: a floating pin is not a voltage

If it fails, look here	Likely cause
`raw` always 0	the ADC1 clock bit (APB2ENR bit 9) is missing, or PA2 nibble is not 0x0
`raw` always 4095 or stuck	`CAL` not waited for, or channel not 2 (`SQR3`)
no output lines	`adc_read` timed out (-1): is ADON set? is `EXTTRIG` + `EXTSEL` = 7 in place?
readings jump by 30 or more	sample time too short: check you wrote code 4 in the field `smpr_field(2)`
`mv` near 0 even at 3V3	`adc_to_mv` divides first: use `mul_div`

Self-check (answer aloud)

1. Why `counts * 3300 / 4095` and not `counts / 4095 * 3300`?
2. What is the highest voltage PA2 may see?
3. Why is the ADC clock divided by 6?
4. Which flag tells you the result is ready, and what clears it?

Hint ladder (open one at a time)

1. STEP 3 is four C1 lines plus one C2 line plus one C3 wait: card C4 in order.
2. Pattern: `REG = field_set(REG, SHIFT, WIDTH, VALUE);`
3. Template with the values for the three SMPR2/SQR3/ADCPRE lines: `<REG>`=`ADC1->SMPR2`, `<SHIFT>`=`smpr_field(2u)`, `<WIDTH>`=3, `<VALUE>`=4; `ADC1->SQR3`, 0, 5, 2; `RCC->CFGR`, `RCC_CFGR_ADCPRE_Pos`, 2, 2. Each reads `<REG> = field_set(<REG>, <SHIFT>, <WIDTH>, <VALUE>);`

Title: M2-02 TIM3-triggered ADC with DMA1 channel 1: capture 256 samples

Milestone: M2 Sampling

Labels: m2, blank-level-2, hardware

M2-02 TIM3-triggered ADC with DMA1 channel 1: capture 256 samples

START HERE

1. Type `make t T=m2_02`. Red tests.
2. Open `src/sampler.c`. Find `@TODO(M2-02/1)`: `sampler_timing` is three lines of arithmetic.
3. Then STEPS 2 to 6 in the same file. Read the worked example (DMA copy) first.

Starts from / Ends with

Starts from	Ends with
M2-01: `adc_init` works, `mul_div`, `field_set`, `wait_flag`.	`sampler_timing` tested for all 8 sample rates.
	`sampler_init`, `sampler_start`, `sampler_wait`, `sampler_stop` written.
	256 samples land in `g_cap` at a steady rate, with no CPU help.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `mul_div`, `smpr_field`, `gpio_cfg`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Card C5 Integer scaling: multiply first, divide last

```
result = (uint64_t)a * b / c;       /* multiply first, in 64 bits */
In code:  mul_div(a, b, c)
```

`smpr_field`: `shift = (ch < 10) ? ch * 3 : (ch - 10) * 3 /* 3 bits per channel; SMPR2 for ch 0-9, SMPR1 for ch 10-17 */`

Card C2 GPIO pin nibble (CRL or CRH)

```
shift = (pin % 8) * 4          /* each pin owns 4 bits */
register = (pin < 8) ? CRL : CRH
gpio_cfg(GPIOx, pin, NIBBLE);   /* = C1 with width 4 */
```

Learn (about 30 minutes)

A scope needs samples at a steady rate. Loops with delays jitter. Hardware does it exactly: a timer ticks, each tick starts one ADC conversion (a "trigger"), and DMA moves each result into memory. The CPU does nothing during the capture.

The chain. TIM3 counts. Each time it overflows (update event) it sends a pulse out the "trigger output" (TRGO). The ADC is set to start a conversion on TIM3 TRGO (EXTSEL = 4). At the end of each conversion the ADC asks DMA for help (DMA bit). DMA channel 1 copies `ADC1->DR` into the next slot of `g_cap`, 256 times, and sets the transfer-complete flag.

DMA is a delivery driver. You give it: where from (CPAR), where to (CMAR), how many (CNDTR), how wide (16 bits), and "move to the next slot each time" (MINC). It works while you cook.

Sample rate = timer overflow rate. Timer clock 72 MHz. `ticks = 72,000,000 / fs`. The reload register ARR is only 16 bits (max 65535). Most rates fit with prescaler 0: ARR = ticks - 1. The slowest rate (800 Hz) needs 90000 ticks: too big, so the prescaler divides by 2 first.

Order matters. Configure the timer first. Setting the update-generate bit (UG) fires one update event: do that BEFORE the ADC is routed to the timer, or you capture one stray sample. Then arm DMA. Then route the ADC. Start the timer last.

One shot. DMA runs in normal mode: after 256 transfers it stops and sets TCIF1. You poll TCIF1 (card C3).

Worked example: DMA memory-to-memory copy

```
/* WORKED EXAMPLE (M2-02): DMA memory-to-memory copy on channel 2 (channel 1 is the ADC's). */
#include "stm32f103.h"
#include "drills.h"

#define DMA_CCR_PINC     BIT(6)
#define DMA_CCR_MEM2MEM  BIT(14)
#define DMA_ISR_TCIF2    BIT(5)            /* channel 2 transfer-complete flag */

int dma_copy_words(uint32_t *dst, const uint32_t *src, uint32_t n)
{
    RCC->AHBENR |= RCC_AHBENR_DMA1EN;                          /* 1. clock                    */
    DMA1->IFCR = 0xFu << 4;                                    /* 2. clear channel-2 flags    */
    DMA1->ch[1].CPAR  = (uint32_t)(uintptr_t)src;              /*    source (the "peripheral") */
    DMA1->ch[1].CMAR  = (uint32_t)(uintptr_t)dst;              /*    destination              */
    DMA1->ch[1].CNDTR = n;                                     /*    how many items           */
    uint32_t ccr = field_set(0u, DMA_CCR_PSIZE_Pos, 2, 2);     /* 3. 32-bit on both sides     */
    ccr = field_set(ccr, DMA_CCR_MSIZE_Pos, 2, 2);
    DMA1->ch[1].CCR = ccr | DMA_CCR_MEM2MEM | DMA_CCR_PINC | DMA_CCR_MINC | DMA_CCR_EN; /* 4. go */
    if (wait_flag(&DMA1->ISR, DMA_ISR_TCIF2, 1, 100000u)) return -1;  /* 5. wait for done     */
    DMA1->IFCR = DMA_ISR_TCIF2;                                /*    clear the flag           */
    DMA1->ch[1].CCR &= ~DMA_CCR_EN;
    return 0;
}
```

Same recipe: clock, clear flags, addresses, count, field setup (card C1), enable, wait on TCIF (card C3). Yours uses channel 1, a peripheral address (`&ADC1->DR`), and 16-bit sizes.

Cheat sheet

Item	Value
DMA1 base	0x40020000. ISR 0x00, IFCR 0x04. Channel 1 starts at 0x08: CCR 0x08, CNDTR 0x0C, CPAR 0x10, CMAR 0x14 (each channel is 0x14 bytes). In code: `DMA1->ch[0].CCR`
DMA clock	`RCC->AHBENR` bit 0 (`RCC_AHBENR_DMA1EN`)
DMA CCR bits	EN bit 0, MINC bit 7, PSIZE shift 8 width 2, MSIZE shift 10 width 2 (value 1 = 16 bit). DIR stays 0 (peripheral to memory); CIRC stays 0 (one shot)
DMA flags (channel 1)	ISR bit 1 = TCIF1 (transfer complete). IFCR: write 0xF to clear all four channel-1 flags
TIM3 base	0x40000400. CR1 0x00, CR2 0x04, EGR 0x14, SR 0x10, PSC 0x28, ARR 0x2C
TIM3 clock	`RCC->APB1ENR` bit 1 (`RCC_APB1ENR_TIM3EN`)
TIM bits	CR1 CEN bit 0. CR2 MMS shift 4 width 3 (value 2 = update event to TRGO). EGR UG bit 0
ADC CR2	EXTSEL shift 17 width 3 (value 4 = TIM3 TRGO). DMA bit 8. EXTTRIG bit 20
`sampler_timing`	`ticks = 72000000 / fs; psc = (ticks - 1) / 65536; arr = ticks / (psc + 1) - 1;`
Rates	160000 -> psc 0 arr 449; 32000 -> 0, 2249; 1600 -> 0, 44999; 800 -> psc 1, arr 44999
Address of a register	`(uint32_t)(uintptr_t)&ADC1->DR` (the cast turns an address into a number for CPAR)
Buffer	`g_cap` (256 x uint16), `SAMPLE_COUNT` = 256
Names in the header (given)	`DMA_CCR_EN`, `DMA_CCR_MINC`, `DMA_CCR_PSIZE_Pos`, `DMA_CCR_MSIZE_Pos`, `DMA_ISR_TCIF1`, `TIM_CR1_CEN`, `TIM_CR2_MMS_Pos`, `TIM_EGR_UG`, `ADC_CR2_DMA`, `ADC_CR2_EXTSEL_Pos`, `ADC_CR2_EXTTRIG`

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M2-01. `make t T=m2_02`. **CHECK:** `4 Tests`, failures.

STEP 1 · TODO M2-02/1 · `sampler_timing` · card C5 flavour · [src: M2-02 step 1]

- **WHAT:** Turn a sample rate into timer prescaler and reload values.
- **INGREDIENTS:** 72000000 (`SYSCLK_HZ`), 65536.
- **HOW:**

```
uint32_t ticks = SYSCLK_HZ / <FS>;
*<PSC> = (ticks - 1u) / 65536u;
*<ARR> = ticks / (*<PSC> + 1u) - 1u;
```

fs	ticks	psc	arr
160000	450	0	449
32000	2250	0	2249
1600	45000	0	44999
800	90000	1	44999

`psc` and `arr` are pointers: `*psc = ...` stores the answer where the caller wants it.

- **WHERE:** `src/sampler.c`, `sampler_timing`.
- **CHECK:** `make t T=m2_02` -> `test_timing_table:PASS` and `test_timing_exact:PASS`.

STEP 2 · TODO M2-02/2 · `sampler_init` (and `sampler_stop`) · card C4 · [src: M2-02 step 2, card C4]

- **WHAT:** Switch on the clocks of DMA1 and TIM3.
- **INGREDIENTS:** `RCC_AHBENR_DMA1EN` (AHBENR), `RCC_APB1ENR_TIM3EN` (APB1ENR).
- **HOW:**

```
RCC->AHBENR  |= <DMA BIT>;
RCC->APB1ENR |= <TIM BIT>;
```

name	value
DMA BIT	`RCC_AHBENR_DMA1EN`
TIM BIT	`RCC_APB1ENR_TIM3EN`

Also fill the empty `sampler_stop`: `TIM3->CR1 &= ~TIM_CR1_CEN;` then `DMA1->ch[0].CCR &= ~DMA_CCR_EN;` (clear a bit with `&= ~`).

- **WHERE:** `src/sampler.c`, `sampler_init`, `sampler_stop`.
- **CHECK:** `make t T=m2_02` -> `test_sampler_code` fails later, at the MMS line.

STEP 3 · TODO M2-02/3 · timer setup in `sampler_start` · card C1 · [src: M2-02 step 3, card C1]

- **WHAT:** Set TIM3 to overflow at the sample rate and to send TRGO on each overflow.
- **INGREDIENTS:** `sampler_timing`, PSC, ARR, CR2 MMS, EGR UG, SR.
- **HOW:** (first lines of `sampler_start`: declare `uint32_t psc, arr;` and call `sampler_stop();`)

```
sampler_timing(fs_hz, &psc, &arr);
TIM3->PSC = psc;
TIM3->ARR = arr;
TIM3->CR2 = field_set(TIM3->CR2, <MMS SHIFT>, <WIDTH>, <VALUE>);
TIM3->EGR = TIM_EGR_UG;      /* load PSC/ARR now */
TIM3->SR  = 0u;              /* clear the update flag the UG caused */
```

field	shift	width	value
MMS	`TIM_CR2_MMS_Pos` (4)	3	2

This is card C1 again, with shift 4 and width 3.

- **WHERE:** `src/sampler.c`, `sampler_start`, after `sampler_stop();`.
- **CHECK:** `make t T=m2_02` -> `test_sampler_code` fails later, at the CNDTR line.

STEP 4 · TODO M2-02/4 · DMA setup · cards C1 + C5 flavour · [src: M2-02 step 4, card C1]

- **WHAT:** Tell DMA channel 1 where to read, where to write, and how many.
- **INGREDIENTS:** cheat sheet DMA rows.
- **HOW:**

```
DMA1->IFCR = <CLEAR ALL>;
DMA1->ch[0].CPAR  = <WHERE FROM>;
DMA1->ch[0].CMAR  = <WHERE TO>;
DMA1->ch[0].CNDTR = <HOW MANY>;
uint32_t ccr = field_set(0u, <PSIZE SHIFT>, 2u, <SIZE>);
ccr = field_set(ccr, <MSIZE SHIFT>, 2u, <SIZE>);
DMA1->ch[0].CCR = ccr | <INCREMENT BIT> | <ENABLE BIT>;
```

placeholder	value
CLEAR ALL	`0xFu` (the four channel-1 flags)
WHERE FROM	the address of the ADC result register, as a number: `(uint32_t)(uintptr_t)&ADC1->DR` shape
WHERE TO	the address of the capture array `g_cap` as a number (same cast shape; an array name is already an address)
HOW MANY	`SAMPLE_COUNT`
INCREMENT BIT / ENABLE BIT	`DMA_CCR_MINC` / `DMA_CCR_EN`

field	shift	width	value
PSIZE	`DMA_CCR_PSIZE_Pos` (8)	2	1 (16 bit)
MSIZE	`DMA_CCR_MSIZE_Pos` (10)	2	1 (16 bit)

`&ADC1->DR` is the address of the ADC result register. This is card C1 again twice (shifts 8 and 10, width 2).

- **WHERE:** `src/sampler.c`, `sampler_start`, after STEP 3.
- **CHECK:** `make t T=m2_02` -> `test_sampler_code` fails later, at the EXTSEL line.

STEP 5 · TODO M2-02/5 · route the ADC and start · card C1 · [src: M2-02 step 5, card C1]

- **WHAT:** Make the ADC start on TIM3 TRGO and use DMA, then start the timer.
- **INGREDIENTS:** EXTSEL = 4, `ADC_CR2_DMA`, `ADC_CR2_EXTTRIG`, `TIM_CR1_CEN`.
- **HOW:**

```
ADC1->CR2 = field_set(ADC1->CR2, <EXTSEL SHIFT>, 3u, <VALUE>);
ADC1->CR2 |= ADC_CR2_DMA | ADC_CR2_EXTTRIG;
TIM3->CR1 |= TIM_CR1_CEN;
```

field	shift	width	value
EXTSEL	`ADC_CR2_EXTSEL_Pos` (17)	3	4 (TIM3 TRGO)

`adc_init` left EXTSEL = 7. Because `field_set` clears first, 7 becomes 4. Without the clear, 7 OR 4 = 7 and you would stay on software start. That is why card C1 clears first.

- **WHERE:** `src/sampler.c`, end of `sampler_start`.
- **CHECK:** `make t T=m2_02` -> `test_sampler_code` fails only at the `TCIF1` line.

STEP 6 · TODO M2-02/6 · `sampler_wait` · card C3 · [src: M2-02 step 6, card C3]

- **WHAT:** Wait until 256 samples are in, then clear the flag.
- **INGREDIENTS:** `DMA1->ISR`, `DMA_ISR_TCIF1`, `DMA1->IFCR`.
- **HOW:**

```
if (wait_flag(&DMA1->ISR, <FLAG>, 1, <TRIES>) != 0) return -1;
DMA1->IFCR = <FLAG>;
return 0;
```

name	value
FLAG	`DMA_ISR_TCIF1`
TRIES	the function argument `tries`

- **WHERE:** `src/sampler.c`, `sampler_wait`.
- **CHECK:** `make t T=m2_02` -> `4 Tests 0 Failures`, `OK`.

STEP 7 · TODO M2-02/7 · capture in `main` · [src: M2-02 step 7]

- **WHAT:** Capture once and print the first 8 samples and the capture time.
- **INGREDIENTS:** `sampler_init`, `sampler_start(32000u)`, `sampler_wait(20000000u)`, `g_cap`, `millis`, `elapsed`.
- **HOW:** after `adc_init();` add:

```
sampler_init();
uint32_t t0 = millis();
sampler_start(32000u);
int rc = sampler_wait(20000000u);          /* 0 = done */
uint32_t dt = elapsed(millis(), t0);
```

then print `rc`, `dt`, and `g_cap[0]..g_cap[7]` with `fmt_u32` and `uart_puts` (as in M2-01 step 6).

- **WHERE:** `src/main.c`, `main`.

- **CHECK:** `picocom`: `rc` is 0 and `dt` is 8 or 9 (256 / 32000 = 8 ms).

STEP 8 · finish (no TODO)

- Add `test_m2_02` to `tests.active`, `make test-active`, commit `M2-02 done`.

Verify

Do this	Expect
`make t T=m2_02`	`4 Tests 0 Failures 0 Ignored`, `OK`
PA2 on the pot (fixed position), capture at 32000	the 8 printed samples agree within about 20 counts
PA2 on GND	all 8 samples between 0 and about 10
PA2 on 3V3	all about 4080 to 4095
`dt` at `sampler_start(32000u)`	8 or 9 (milliseconds)
`sampler_start(800u)`	`dt` about 320 (256 / 800 = 0.32 s) and `rc` still 0

If it fails, look here	Likely cause
`rc` = -1	DMA never finished: ADC not routed to TIM3 (EXTSEL 4, EXTTRIG), DMA bit missing, or TIM3 not started (CEN)
`dt` is 0	the capture finished instantly: stale TCIF1 flag. Clear flags with `IFCR = 0xF` before arming
samples all 0	the ADC is not ON: did `adc_init` run first?
first sample looks odd	UG was written after the ADC was routed: keep the order of STEPS 3 to 5
every sample twice as old as expected	the sample rate is half: check `psc`/`arr` against the table
`dt` wrong at 800 Hz	prescaler missing: ARR cannot hold 90000

Self-check (answer aloud)

1. What starts each conversion? What moves each result?
2. Why does 800 Hz need a prescaler?
3. Why write EGR = UG before routing the ADC to the timer?
4. What does CNDTR count, and what happens when it reaches 0?

Hint ladder (open one at a time)

1. STEP 4 is card C1 twice, then plain register writes.
2. Pattern: `ccr = field_set(ccr, SHIFT, WIDTH, VALUE);` twice, then `CCR = ccr | MINC | EN;`
3. Template with the values: `uint32_t ccr = field_set(0u, <S1>, <W>, <V>); ccr = field_set(ccr, <S2>, <W>, <V>);` with `<S1>`=`DMA_CCR_PSIZE_Pos` (8), `<S2>`=`DMA_CCR_MSIZE_Pos` (10), `<W>`=2, `<V>`=1.

Title: M2-03 PWM test signal on PA1 (TIM2 channel 2) looped into the ADC

Milestone: M2 Sampling

Labels: m2, blank-level-2, hardware, stretch-cut-first

M2-03 PWM test signal on PA1 (TIM2 channel 2) looped into the ADC

START HERE

1. Type `make t T=m2_03`. Red tests.
2. Open `src/pwm.c`. Find `@TODO(M2-03/1)`: `pwm_ccr` is ONE line, the same `mul_div` you already wrote.
3. Wire a jumper from PA1 to PA2 only AFTER STEP 5 is done.

Starts from / Ends with

Starts from	Ends with
M2-02: `sampler_start` / `sampler_wait`, `mul_div`.	`pwm_ccr`, `pwm_init`, `pwm_set_duty`, `count_above`.
	A 1 kHz, 25 % signal on PA1.
	The scope's own signal, looped PA1 to PA2, captured: about 64 samples are high.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `mul_div`, `wait_flag`, `brr_for`.
3. Rewrite each body from memory.
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Card C5 Integer scaling: multiply first, divide last

```
result = (uint64_t)a * b / c;       /* multiply first, in 64 bits */
In code:  mul_div(a, b, c)
```

Card C3 Wait for a flag with a timeout

```
while (tries--) {
    if ((*reg & MASK) is in the wanted state) return 0;   /* flag came */
}
return -1;                                                /* gave up   */
In code:  if (wait_flag(&REG, MASK, WANT_SET, TRIES)) { /* timed out */ }
```

`brr_for`: `BRR = (pclk + baud / 2) / baud /* 72 MHz, 115200 -> 625 */`

Learn (about 30 minutes)

PWM = a pin that switches on and off very fast. The fraction of time it is ON is the duty cycle. 25 % duty at 3.3 V averages 0.825 V.

A timer makes it. The counter counts up from 0 to ARR, then restarts. The compare register CCR sets the switch point: the pin is HIGH while the counter is below CCR (in PWM mode 1). Period = (ARR + 1) ticks. Duty = CCR / (ARR + 1).

Numbers. Timer clock 72 MHz. PSC = 71 divides by 72: 1 MHz. ARR = 999: 1000 ticks = 1 kHz. CCR = 250: 25 % duty.

Pin. TIM2 channel 2 is on PA1 (alternate function, push-pull; nibble 0xA = 2 MHz). The pin belongs to the timer, not to GPIO.

Preload. Set OC2PE so a new CCR value only takes effect at the next period, with no glitch.

Why loop it into the ADC? It is your free signal generator. You have no function generator and no scope: this is the only way to check the scope. PA1 only ever goes between 0 V and 3.3 V, which is exactly what PA2 tolerates.

Worked example: TIM3 PWM on PA6

```
/* WORKED EXAMPLE (M2-03): PWM on TIM3 channel 1, pin PA6, 20 kHz, 30 % duty. */
#include "stm32f103.h"
#include "drills.h"

#define TIM_CCMR1_OC1PE     BIT(3)
#define TIM_CCMR1_OC1M_Pos  4u            /* 3 bits: 6 = PWM mode 1 */
#define TIM_CCER_CC1E       BIT(0)

void pwm_pa6_start(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;                 /* 1. clocks                       */
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    gpio_cfg(GPIOA, 6, 0xA);                            /* 2. PA6 alternate push-pull      */
    TIM3->PSC = 0u;                                     /* 3. 72 MHz / 3600 = 20 kHz       */
    TIM3->ARR = 3599u;
    TIM3->CCMR1 = field_set(TIM3->CCMR1, TIM_CCMR1_OC1M_Pos, 3, 6) | TIM_CCMR1_OC1PE;
    TIM3->CCR1 = mul_div(3600u, 30u, 100u);             /*    30 % of 3600 = 1080          */
    TIM3->CCER |= TIM_CCER_CC1E;
    TIM3->EGR = TIM_EGR_UG;                             /* 4. start                        */
    TIM3->CR1 |= TIM_CR1_CEN;
}
```

Same recipe with different numbers: channel 1 (OC1M at shift 4, OC1PE bit 3, CC1E bit 0), pin PA6, 20 kHz, 30 %.

Cheat sheet

Item	Value
TIM2 base	0x40000000. CR1 0x00, EGR 0x14, CCMR1 0x18, CCER 0x20, PSC 0x28, ARR 0x2C, CCR1 0x34, CCR2 0x38
TIM2 clock	`RCC->APB1ENR` bit 0 (`RCC_APB1ENR_TIM2EN`); GPIOA clock APB2ENR bit 2
Pin PA1	CRL, shift 4, nibble 0xA
CCMR1 (channel 2)	OC2M: shift 12 width 3 value 6 (PWM mode 1). OC2PE: bit 11
CCER	CC2E bit 4 (output enable for channel 2)
Start	EGR UG bit 0, CR1 CEN bit 0
Numbers	PSC 71, ARR 999
`pwm_ccr(arr, percent)`	`mul_div(arr + 1, percent, 100)` -> 250 for 25 %
Duty table	0 % -> 0, 25 % -> 250, 50 % -> 500, 75 % -> 750, 100 % -> 1000
Names in the header (given)	`RCC_APB2ENR_IOPAEN`, `RCC_APB1ENR_TIM2EN`, `TIM_CCMR1_OC2M_Pos`, `TIM_CCMR1_OC2PE`, `TIM_CCER_CC2E`, `TIM_EGR_UG`, `TIM_CR1_CEN`
Capture at 32 kSPS	256 samples = 8 periods of 1 kHz. 25 % duty: about 64 samples above 2048 (allow 56 to 72)
Wire	jumper from PA1 to PA2
`count_above(s, n, thr)`	how many of `s[0..n-1]` are greater than `thr`

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M2-02. `make t T=m2_03`. **CHECK:** `5 Tests`, failures.

STEP 1 · TODO M2-03/1 · `pwm_ccr` · card C5 · [src: M2-03 step 1, card C5]

- **WHAT:** Duty percent to the CCR number.
- **INGREDIENTS:** `mul_div`, ARR + 1 ticks per period.
- **HOW:** `return mul_div(<ARR> + 1u, <PERCENT>, <HUNDRED>);`

arr	percent	result
999	25	250
999	100	1000
449	10	45

`<HUNDRED>` = `100u`. This is card C5 again, with (1000, 25, 100).

- **WHERE:** `src/pwm.c`, `pwm_ccr`.

- **CHECK:** `make t T=m2_03` -> `test_pwm_ccr:PASS`.

STEP 2 · TODO M2-03/2 · `pwm_init` · card C4 · [src: M2-03 step 2, cards C1+C2+C4]

- **WHAT:** Set up TIM2 channel 2 as a 1 kHz PWM on PA1.
- **INGREDIENTS:** cheat sheet.
- **HOW:** card C4 in order:

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;                       /* clock       */
RCC->APB1ENR |= <TIM CLOCK>;
gpio_cfg(GPIOA, <PIN>, <NIBBLE>);                         /* pin  (C2)   */
TIM2->PSC = <PSC>;                                        /* configure   */
TIM2->ARR = <ARR>;
TIM2->CCMR1 = field_set(TIM2->CCMR1, <OC2M SHIFT>, 3u, <MODE>) | TIM_CCMR1_OC2PE;   /* C1 */
TIM2->CCER |= TIM_CCER_CC2E;
TIM2->CCR2 = pwm_ccr(999u, 50u);                          /* start at 50 % */
TIM2->EGR = TIM_EGR_UG;                                   /* start       */
TIM2->CR1 |= TIM_CR1_CEN;
```

name	value
TIM CLOCK	`RCC_APB1ENR_TIM2EN`
PIN / NIBBLE	`1u` / `0xAu`
PSC / ARR	`71u` / `999u`
OC2M	shift `TIM_CCMR1_OC2M_Pos` (12), width 3, value 6

This is card C1 again (shift 12, width 3), and card C2 with pin 1 nibble 0xA.

- **WHERE:** `src/pwm.c`, `pwm_init`.
- **CHECK:** `make t T=m2_03` -> `test_pwm_code` still fails at the `pwm_set_duty` line.

STEP 3 · TODO M2-03/3 · `pwm_set_duty` · [src: M2-03 step 3]

- **WHAT:** Change the duty while running.
- **INGREDIENTS:** `TIM2->ARR` (read it back), `pwm_ccr`.
- **HOW:** `TIM2->CCR2 = pwm_ccr(<ARR>, <PERCENT>);` where `<ARR>` is `TIM2->ARR` and `<PERCENT>` is the argument.
- **WHERE:** `src/pwm.c`, `pwm_set_duty`.
- **CHECK:** `make t T=m2_03` -> `test_pwm_code:PASS`.

STEP 4 · TODO M2-03/4 · `count_above` · [src: M2-03 step 4]

- **WHAT:** Count samples above a threshold.
- **INGREDIENTS:** a loop and a counter.
- **HOW:**

```
unsigned count = 0;
for (unsigned i = 0; i < <N>; i++) {
    if (<S>[i] > <THR>) count++;
}
return count;
```

s	thr	result
{0, 4095, 0, 4095, 100}	2048	2
same	4095	0 (strictly greater)
n = 0	anything	0

- **WHERE:** `src/pwm.c`, `count_above`.
- **CHECK:** `make t T=m2_03` -> `test_count_above:PASS`, `test_count_above_pwm:PASS`, `5 Tests 0 Failures`.

STEP 5 · TODO M2-03/5 · use it in `main` · [src: M2-03 step 5]

- **WHAT:** Run the PWM, capture it, count the high samples.
- **INGREDIENTS:** `pwm_init`, `pwm_set_duty(25u)`, `sampler_start(32000u)`, `sampler_wait`, `count_above(g_cap, 256, 2048)`.
- **HOW:** in `main`, after `sampler_init();`:

```
pwm_init();
pwm_set_duty(25u);
sampler_start(32000u);
if (sampler_wait(20000000u) == 0) {
    unsigned high = count_above(g_cap, 256u, 2048u);
    /* print "high=" and the number with fmt_u32 / uart_puts */
```

```
}
```

- **WHERE:** `src/main.c`, `main`.
- **CHECK:** with PA1 NOT yet wired to PA2: `high=` is small or random (a floating pin).

STEP 6 · wire and measure (no TODO)

- Unplug USB/ST-Link power. Add a jumper from PA1 to PA2. Re-power and flash.
- **CHECK:** `high=` between 56 and 72 (about 64). Change `pwm_set_duty(50u)`: about 128 (112 to 144). 75 %: about 192.

STEP 7 · finish (no TODO)

- Add `test_m2_03` to `tests.active`, `make test-active`, commit `M2-03 done`.

Verify

Do this	Expect
`make t T=m2_03`	`5 Tests 0 Failures 0 Ignored`, `OK`
PA1 wired to PA2, 25 %	`high` 56 to 72
PA1 wired to PA2, 50 %	`high` 112 to 144
PA2 wired to GND, any duty	`high` 0
Print `g_cap[0..31]` with the loop closed	a block of values near 0 and a block near 4095 (a square wave), repeating every 32 samples

If it fails, look here	Likely cause
`high` is 0 with the jumper in	no PWM: `CC2E` missing, or PA1 nibble is not 0xA
`high` is always 256	PWM stuck high: OC2M value not 6, or CCR2 not set
frequency wrong	PSC and ARR: 72 MHz / 72 / 1000 = 1 kHz
`high` about 128 at 25 %	CCR2 still 500 from `pwm_init`: call `pwm_set_duty(25u)`
analog values near 2000 with no jumper	a floating pin: wire it

Self-check (answer aloud)

1. What is the PWM frequency if PSC = 71 and ARR = 999 at 72 MHz?
2. What CCR gives 25 % duty?
3. Why does `field_set` clear OC2M before writing 6?
4. Why is the PA1 to PA2 loop safe for the ADC pin?

Hint ladder (open one at a time)

1. STEP 2 is card C4 once more. The only new register is CCMR1: card C1 with width 3.
2. Pattern: `TIM2->CCMR1 = field_set(TIM2->CCMR1, SHIFT, 3u, MODE) | PRELOAD_BIT;`
3. Template with the values: `<SHIFT>` = `TIM_CCMR1_OC2M_Pos` (12), `<MODE>` = 6, `<PRELOAD_BIT>` = `TIM_CCMR1_OC2PE`.

Milestone M3: Display

I2C, SSD1306 and framebuffer, plot, grid and text.

Title: M3-01 I2C1 master with timeouts and a bus scan that finds 0x3C

Milestone: M3 Display

Labels: m3, blank-level-2, hardware, drills

M3-01 I2C1 master with timeouts and a bus scan that finds 0x3C

START HERE

1. Type `make t T=m3_01`. Red tests.
2. Open `src/drills.c`, find `@TODO(M3-01/1)`: `i2c_ccr` is ONE line with two divisions.
3. Then `src/i2c.c`, STEPS 2 to 4. Read the SPI worked example first: your `i2c_write` is the same shape.
4. Wire the OLED only after STEP 4 (STEP 5).

Starts from / Ends with

Starts from	Ends with
M2-03: all earlier drills, UART printing.	`i2c_ccr` tested.
	`i2c_init`, `i2c_write` (with timeouts), `i2c_scan`.
	A scan on the UART that prints address 60 (= 0x3C), your OLED.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `smpr_field`, `mul_div`, `reached`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

I2C is two wires, many devices. SCL is the clock, SDA is the data. Both are open-drain: a device only pulls the line LOW, and pull-up resistors (already on your OLED module) pull it HIGH. Every device has a 7-bit address. Your OLED answers at 0x3C.

One write transaction:

1. START (SDA falls while SCL is high).
2. Send the address in the top 7 bits plus a 0 write bit: `addr << 1`.
3. The device pulls SDA low for one bit: ACK ("I am here"). No pull = NACK ("nobody home").
4. Send data bytes, each followed by an ACK.
5. STOP.

The peripheral does the wiggling. You set START in CR1 and wait for SB (start sent) in SR1. You write the address to DR and wait for ADDR (address sent and acknowledged) or AF (acknowledge failure = NACK). Reading SR1 then SR2 clears ADDR. You write bytes to DR when TXE is set. After the last byte wait for BTF (byte transfer finished), then set STOP.

Every wait needs a timeout (card C3). A missing device or a stuck bus must never hang your scope. On any timeout: send STOP and return an error code.

Speed. I2C clock = pclk / (3 x CCR) in fast mode with DUTY 0. 36 MHz / (3 x 30) = 400 kHz. In standard mode it is pclk / (2 x CCR): CCR 180 gives 100 kHz. CR2 FREQ tells the peripheral the bus clock in MHz (36). TRISE is the maximum rise time in clock ticks plus one: 12 for fast mode, 37 for standard mode.

Bus scan. Try every address from 0x08 to 0x77 with a zero-length write. An ACK means a device lives there.

Worked example: SPI1 transmit with timeouts

```
/* WORKED EXAMPLE (M3-01): SPI1 transmit, every wait has a timeout. Same shape as the I2C write. */
```

```
#include "stm32f103.h"
#include "drills.h"

typedef struct { volatile uint32_t CR1, CR2, SR, DR; } SPI_TypeDef;      /* 0x00 0x04 0x08 0x0C */
#define SPI1 ((SPI_TypeDef *)0x40013000u)
#define RCC_APB2ENR_SPI1EN BIT(12)
#define SPI_CR1_MSTR BIT(2)
#define SPI_CR1_SPE  BIT(6)
#define SPI_CR1_SSI  BIT(8)
#define SPI_CR1_SSM  BIT(9)
#define SPI_SR_TXE   BIT(1)
#define SPI_SR_BSY   BIT(7)

void spi1_init(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_SPI1EN;   /* 1. clocks               */
    gpio_cfg(GPIOA, 5, 0xB);                                   /* 2. PA5 SCK  alt push-pull */
    gpio_cfg(GPIOA, 7, 0xB);                                   /*    PA7 MOSI alt push-pull */
    SPI1->CR1 = field_set(0u, 3, 3, 4)                         /* 3. baud: BR field = 4 -> /32 */
              | SPI_CR1_MSTR | SPI_CR1_SSM | SPI_CR1_SSI;
    SPI1->CR1 |= SPI_CR1_SPE;                                  /* 4. start                */
}

int spi1_send(const uint8_t *buf, unsigned len)
{
    for (unsigned i = 0; i < len; i++) {
        if (wait_flag(&SPI1->SR, SPI_SR_TXE, 1, 20000u)) return -1;   /* 5. wait: room?  */
        SPI1->DR = buf[i];
    }
    if (wait_flag(&SPI1->SR, SPI_SR_BSY, 0, 20000u)) return -1;       /*    wait: all sent */
    return 0;
}
```

Same shape as your `i2c_write`: setup (card C4), then each wait guarded by `wait_flag` (card C3) that returns an error instead of hanging.

Cheat sheet

Item	Value
I2C1 base	0x40005400. CR1 0x00, CR2 0x04, DR 0x10, SR1 0x14, SR2 0x18, CCR 0x1C, TRISE 0x20
Clock bits	GPIOB: APB2ENR bit 3. I2C1: APB1ENR bit 21
Pins	PB6 (SCL) is CRL shift 24; PB7 (SDA) is CRL shift 28. Nibble 0xF = alternate open-drain 50 MHz
CR1 bits	PE bit 0, START bit 8, STOP bit 9
CR2	FREQ: shift 0, width 6, value 36
CCR	bit 15 = F/S (1 = fast), bit 14 = DUTY (0), bits 11:0 = CCR value. Fast 400 kHz: `I2C_CCR_FS | i2c_ccr(36000000, 400000, 1)` = 30. Standard 100 kHz: 180
TRISE	12 (400 kHz). 37 (100 kHz)
SR1 bits	SB bit 0, ADDR bit 1, BTF bit 2, TXE bit 7, AF bit 10
SR2 bit	BUSY bit 1
`i2c_ccr(pclk, scl, fast)`	`pclk / ((fast ? 3 : 2) * scl)` -> 30, 180
Error codes (given)	`I2C_OK` 0, `I2C_ERR_TIMEOUT` -1, `I2C_ERR_NACK` -2, `I2C_ERR_BUSY` -3
Tries per wait	`I2C_TRIES` (20000, given)
Write order	wait BUSY=0, START, wait SB, DR = addr << 1, wait ADDR or AF, if AF: clear AF, STOP, return NACK; read SR1, read SR2; each byte: wait TXE, DR = byte; wait BTF (only if len > 0); STOP
Clear AF	`I2C1->SR1 &= ~I2C_SR1_AF;`
Read to clear ADDR	`(void)I2C1->SR1; (void)I2C1->SR2;`
OLED wiring	SCL to PB6, SDA to PB7, VCC to 3V3, GND to GND
Scan range	0x08 to 0x77 inclusive
Names in the header (given)	`RCC_APB2ENR_IOPBEN`, `RCC_APB1ENR_I2C1EN`, `I2C_CR1_PE`, `I2C_CR1_START`, `I2C_CR1_STOP`, `I2C_SR1_SB`, `I2C_SR1_ADDR`, `I2C_SR1_BTF`, `I2C_SR1_TXE`, `I2C_SR1_AF`, `I2C_SR2_BUSY`, `I2C_CCR_FS`

Cards C1, C2, C3:

```
C1: reg = field_set(reg, SHIFT, WIDTH, VALUE);
C2: gpio_cfg(GPIOx, pin, NIBBLE);
```

```
C3: if (wait_flag(&REG, MASK, WANT_SET, TRIES)) { /* timed out */ }
```

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M2-03. `make t T=m3_01`. **CHECK:** `4 Tests`, failures.

STEP 1 · TODO M3-01/1 · `i2c_ccr` · [src: M3-01 step 1]

- **WHAT:** Compute the CCR number for a bus speed.
- **INGREDIENTS:** divide by 3 x scl in fast mode, by 2 x scl in standard mode.
- **HOW:** `return <PCLK> / ((<FAST> ? 3u : 2u) * <SCL>);`

pclk	scl	fast	result
36000000	400000	1	30
36000000	100000	0	180
36000000	100000	1	120

- **WHERE:** `src/drills.c`, `i2c_ccr`.
- **CHECK:** `make t T=m3_01` -> `test_i2c_ccr:PASS`.

STEP 2 · TODO M3-01/2 · `i2c_init` · card C4 · [src: M3-01 step 2, cards C1+C2+C4]

- **WHAT:** Bring I2C1 up at 400 kHz.
- **INGREDIENTS:** cheat sheet.
- **HOW:** card C4:

```
RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;                    /* clocks     */
RCC->APB1ENR |= <I2C CLOCK>;
gpio_cfg(GPIOB, <SCL PIN>, <NIBBLE>);                  /* pins (C2)  */
gpio_cfg(GPIOB, <SDA PIN>, <NIBBLE>);
I2C1->CR1 = 0u;                                        /* PE off while configuring */
I2C1->CR2 = field_set(I2C1->CR2, 0u, 6u, <FREQ>);      /* C1         */
I2C1->CCR = I2C_CCR_FS | i2c_ccr(<PCLK>, <SCL HZ>, 1);
I2C1->TRISE = <TRISE>;
I2C1->CR1 |= I2C_CR1_PE;                               /* start      */
```

name	value
I2C CLOCK	`RCC_APB1ENR_I2C1EN`
SCL pin, SDA pin, NIBBLE	`6u`, `7u`, `0xFu`
FREQ	`36u`
PCLK, SCL HZ	`36000000u`, `400000u`
TRISE	`12u`

Card C2 twice (pins 6 and 7, nibble 0xF) and card C1 once (shift 0, width 6, value 36).

- **WHERE:** `src/i2c.c`, `i2c_init`.
- **CHECK:** `make t T=m3_01` -> `test_i2c_code` fails later, at the `BUSY` line.

STEP 3 · TODO M3-01/3 · `i2c_write` · card C3 five times · [src: M3-01 step 3, card C3]

- **WHAT:** Send `len` bytes to a device, never hang.
- **INGREDIENTS:** cheat sheet "Write order".
- **HOW:** (the shape; fill each `<...>` from the table)

```
if (wait_flag(&I2C1->SR2, <BUSY>, 0, I2C_TRIES) != 0) return I2C_ERR_BUSY;
I2C1->CR1 |= <START>;
if (wait_flag(&I2C1->SR1, <SB>, 1, I2C_TRIES) != 0) goto timeout;
I2C1->DR = (uint32_t)(addr << 1);
if (wait_flag(&I2C1->SR1, <ADDR> | <AF>, 1, I2C_TRIES) != 0) goto timeout;
if (I2C1->SR1 & <AF>) {
    I2C1->SR1 &= ~<AF>;
    I2C1->CR1 |= <STOP>;
    return I2C_ERR_NACK;
}
(void)I2C1->SR1;  (void)I2C1->SR2;
for (uint16_t i = 0; i < len; i++) {
    if (wait_flag(&I2C1->SR1, <TXE>, 1, I2C_TRIES) != 0) goto timeout;
    I2C1->DR = buf[i];
}
if (len > 0u && wait_flag(&I2C1->SR1, <BTF>, 1, I2C_TRIES) != 0) goto timeout;
I2C1->CR1 |= <STOP>;
```

```
return I2C_OK;
timeout:
I2C1->CR1 |= <STOP>;
return I2C_ERR_TIMEOUT;
```

placeholder	value
BUSY	`I2C_SR2_BUSY` (wait for 0)
START / STOP	`I2C_CR1_START` / `I2C_CR1_STOP`
SB, ADDR, AF, TXE, BTF	`I2C_SR1_SB`, `I2C_SR1_ADDR`, `I2C_SR1_AF`, `I2C_SR1_TXE`, `I2C_SR1_BTF`

This is card C3 again five times, each with a different register and flag. Delete the two stub lines first.

- **WHERE:** `src/i2c.c`, `i2c_write`.
- **CHECK:** `make t T=m3_01` -> `test_i2c_code:PASS`.

STEP 4 · TODO M3-01/4 · `i2c_scan` · [src: M3-01 step 4]

- **WHAT:** Find every device on the bus.
- **INGREDIENTS:** `i2c_write(addr, 0, 0)` (a zero-length write); `I2C_OK`.
- **HOW:**

```
unsigned n = 0;
for (uint8_t a = <FIRST>; a <= <LAST>; a++) {
    if (i2c_write(a, 0, 0) == I2C_OK && n < max) found[n++] = a;
}
return (int)n;
```

name	value
FIRST / LAST	`0x08` / `0x77`

- **WHERE:** `src/i2c.c`, `i2c_scan`.
- **CHECK:** `make t T=m3_01` -> `4 Tests 0 Failures`, `OK`.

STEP 5 · TODO M3-01/5 · scan in `main` · [src: M3-01 step 5]

- **WHAT:** Print the addresses found.
- **INGREDIENTS:** `i2c_init`, `i2c_scan`, `fmt_u32`, `uart_puts`.
- **HOW:** after the UART banner:

```
i2c_init();
uint8_t found[8];
int n = i2c_scan(found, 8);
/* for each i < n: print "i2c: " and found[i] as a number, then "\r\n" */
/* if n == 0 print "i2c: nothing found\r\n" */
```

- **WHERE:** `src/main.c`, `main`.
- **CHECK:** with the OLED wired (SCL to PB6, SDA to PB7, VCC to 3V3, GND to GND): `i2c: 60`. 60 = 0x3C.

STEP 6 · finish (no TODO)

- Add `test_m3_01` to `tests.active`, `make test-active`, commit `M3-01 done`.

Verify

Do this	Expect
`make t T=m3_01`	`4 Tests 0 Failures 0 Ignored`, `OK`
OLED wired, flash, open picocom	`i2c: 60` (and nothing else)
Pull the SDA wire out, reset the board	`i2c: nothing found`, within a few seconds (no hang)
In `main`, call `i2c_write(0x3D, 0, 0)` and print the result	-2 (NACK: nobody at 0x3D). Print as `0 - result` or compare, since `fmt_u32` is unsigned

If it fails, look here	Likely cause
nothing found, OLED wired	SCL and SDA swapped, or VCC not on 3V3, or the module's pins are in a different order (read its silkscreen)
scan hangs	a wait without `wait_flag`, so no timeout
every address "found"	`AF` is not checked: ADDR is set, AF is not
`i2c: 61` instead of 60	address-shifted twice: only `addr << 1` goes to DR
works once then fails	STOP missing after a NACK (the bus stays busy)

If it fails, look here	Likely cause
always BUSY	the bus is stuck: power-cycle the board

Self-check (answer aloud)

1. Why are SCL and SDA open-drain with pull-ups?
2. What does the address byte on the wire contain for 0x3C writing?
3. What is the difference between ADDR and AF?
4. What does each `wait_flag` protect you from?

Hint ladder (open one at a time)

1. STEP 3 is one long chain of C3 waits. Write it top to bottom from the "Write order" row.
2. Pattern: `if (wait_flag(&REG, MASK, WANT, I2C_TRIES) != 0) goto timeout;`
3. Template with the values: first wait `<REG>`=`I2C1->SR2`, `<MASK>`=`I2C_SR2_BUSY`, `<WANT>`=0 and it returns BUSY; second `I2C1->SR1`, `I2C_SR1_SB`, 1; third `I2C1->SR1`, `I2C_SR1_ADDR | I2C_SR1_AF`, 1; inside the loop `I2C1->SR1`, `I2C_SR1_TXE`, 1; last `I2C1->SR1`, `I2C_SR1_BTF`, 1.

Title: M3-02 SSD1306 init/flush and the framebuffer (host-tested with a fake I2C)

Milestone: M3 Display

Labels: m3, blank-level-2, hardware, host-tests

M3-02 SSD1306 init/flush and the framebuffer (host-tested with a fake I2C)

START HERE

1. Type `make t T=m3_02`. Red tests.
2. Open `src/drills.c`, find `@TODO(M3-02/1)`: `fb_index` is ONE line.
3. Read the 8x8 LED-matrix worked example: your framebuffer is the same idea, bigger.

Starts from / Ends with

Starts from	Ends with
M3-01: `i2c_write` works, OLED answers at 0x3C.	`fb_index`, `fb_bit`, `fb_clear`, `fb_pixel`.
	`ssd1306_init` and `ssd1306_flush` (tested with a fake bus).
	Pixels on the real OLED.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `i2c_ccr`, `field_set`, `wait_flag`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

A framebuffer is a picture in RAM. You draw into it with ordinary C. Then you send it to the screen in one go (the flush). 128 x 64 pixels, 1 bit each = 8192 bits = 1024 bytes.

The SSD1306 memory layout is in pages. The screen is 8 pages (horizontal strips) of 8 pixel rows. Each page has 128 columns. One byte = one column of one page: bit 0 is the TOP pixel of that strip, bit 7 the bottom. So pixel (x, y) lives in byte `(y / 8) * 128 + x`, bit `y % 8`.

Setting and clearing a bit. `byte |= mask` sets, `byte &= ~mask` clears, where `mask = 1 << bit`.

Commands and data. Every I2C write to the OLED starts with a control byte. `0x00` means "the bytes after me are commands". `0x40` means "the bytes after me are pixel data".

Fake bus. `ssd1306_init(bus)` takes a function pointer. On the board you pass `i2c_write`. In the tests you pass `fake_i2c_write`, which only records what you send. So you can prove the exact bytes without an OLED.

Flush. Send one command write to set the window (columns 0 to 127, pages 0 to 7), then send the 8 pages as 8 data writes of `0x40` plus 128 bytes.

Worked example: an 8x8 LED-matrix framebuffer

```
/* WORKED EXAMPLE (M3-02): an 8x8 LED-matrix framebuffer. One byte per row, bit x = column x. */
#include <stdint.h>
uint8_t m[8];

unsigned m_index(unsigned x, unsigned y) { (void)x; return y; }      /* which byte?  */
unsigned m_bit(unsigned x)               { return x; }               /* which bit?   */

void m_clear(void) { for (unsigned i = 0; i < 8; i++) m[i] = 0; }

void m_pixel(unsigned x, unsigned y, int on)
{
    if (x >= 8 || y >= 8) return;                       /* ignore off-screen pixels */
    uint8_t mask = (uint8_t)(1u << m_bit(x));
    if (on) m[m_index(x, y)] |= mask;                   /* set the bit              */
```

```
    else    m[m_index(x, y)] &= (uint8_t)~mask;          /* clear the bit           */
}
```

Same idea with a smaller screen: one byte per row, bit x = column x. Yours uses `fb_index` and `fb_bit` instead of `m_index` and `m_bit`.

Cheat sheet

Item	Value
Screen	128 x 64, 8 pages x 128 columns, 1024 bytes in `g_fb`
Pixel (x, y)	byte `(y / 8) * 128 + x`, bit `y % 8` (bit 0 = top row of the page)
Examples	(0,0) byte 0 bit 0; (5,9) byte 133 bit 1; (127,63) byte 1023 bit 7
Ignore	x >= 128 or y >= 64
OLED address	`SSD1306_ADDR` = 0x3C
Init bytes (25, in order)	`AE D5 80 A8 3F D3 00 40 8D 14 20 00 A1 C8 DA 12 81 CF D9 F1 DB 40 A4 A6 AF`
Init transaction	ONE write of 26 bytes: `00` then the 25 bytes
Flush, part 1	one write of 7 bytes: `00 21 00 7F 22 00 07`
Flush, part 2	8 writes of 129 bytes: `40` then 128 bytes of `g_fb` starting at `page * 128`, for page 0 to 7
Bus function type	`int (*bus_write_fn)(uint8_t addr, const uint8_t *buf, uint16_t len)` returns 0 for OK
Use of errors	return the first non-zero result and stop
Test bus	`fake_i2c_write` (records each transaction)
Real bus	`i2c_write`
No libc	write copy loops yourself (`for`), no `memcpy`

Card C1 in disguise: set or clear one bit in a byte: `b |= m` / `b &= ~m`.

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M3-01. `make t T=m3_02`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M3-02/1 · `fb_index` · [src: M3-02 step 1]

- **WHAT:** Which byte holds pixel (x, y)?
- **INGREDIENTS:** 8 rows per page, 128 bytes per page.
- **HOW:** `return (<Y> / 8u) * 128u + <X>;`

x	y	result
0	0	0
5	9	133
0	8	128
127	63	1023

- **WHERE:** `src/drills.c`, `fb_index`.
- **CHECK:** `make t T=m3_02` -> `test_fb_index_bit` still fails (STEP 2 next).

STEP 2 · TODO M3-02/2 · `fb_bit` · [src: M3-02 step 2]

- **WHAT:** Which bit inside that byte?
- **INGREDIENTS:** `y % 8`.
- **HOW:** `return <Y> % 8u;`

y	result
0	0
9	1
63	7

- **WHERE:** `src/drills.c`, `fb_bit`.
- **CHECK:** `make t T=m3_02` -> `test_fb_index_bit:PASS`.

STEP 3 · TODO M3-02/3 · `fb_clear` · [src: M3-02 step 3]

- **WHAT:** Blank the picture.
- **INGREDIENTS:** `g_fb` (1024 bytes).
- **HOW:** `for (unsigned i = 0; i < sizeof g_fb; i++) g_fb[i] = 0;`
- **WHERE:** `src/ssd1306.c`, `fb_clear`.
- **CHECK:** `make t T=m3_02` -> `test_fb_clear:PASS`.

STEP 4 · TODO M3-02/4 · `fb_pixel` · [src: M3-02 step 4, card C1 flavour]

- **WHAT:** Set or clear one pixel.
- **INGREDIENTS:** `fb_index`, `fb_bit`, the bound 128 x 64.
- **HOW:**

```
if (<X> >= 128u || <Y> >= 64u) return;
unsigned i = fb_index(x, y);
uint8_t m = (uint8_t)(1u << fb_bit(y));
if (on) g_fb[i] |= m; else g_fb[i] &= (uint8_t)~m;
```

call	effect
`fb_pixel(0, 0, 1)`	`g_fb[0] = 0x01`
`fb_pixel(5, 9, 1)`	`g_fb[133] = 0x02`
`fb_pixel(127, 63, 1)`	`g_fb[1023] = 0x80`
`fb_pixel(128, 0, 1)`	nothing

- **WHERE:** `src/ssd1306.c`, `fb_pixel`.
- **CHECK:** `make t T=m3_02` -> `test_fb_pixel:PASS`.

STEP 5 · TODO M3-02/5 · `init_seq` and `ssd1306_init` · [src: M3-02 step 5]

- **WHAT:** Send the 25 init bytes in one write.
- **INGREDIENTS:** the 25 bytes (cheat sheet), control byte 0x00, `SSD1306_ADDR`.
- **HOW:**

```
static const uint8_t init_seq[] = { <THE 25 BYTES, 0x.. separated by commas> };
...
uint8_t buf[1 + sizeof init_seq];
s_bus = bus;
buf[0] = <CONTROL>;
for (unsigned i = 0; i < sizeof init_seq; i++) buf[1 + i] = init_seq[i];
return s_bus(SSD1306_ADDR, buf, (uint16_t)sizeof buf);
```

name	value
THE 25 BYTES	`0xAE, 0xD5, 0x80, 0xA8, 0x3F, 0xD3, 0x00, 0x40, 0x8D, 0x14, 0x20, 0x00, 0xA1, 0xC8, 0xDA, 0x12, 0x81, 0xCF, 0xD9, 0xF1, 0xDB, 0x40, 0xA4, 0xA6, 0xAF`
CONTROL	`0x00`

`s_bus` is a function pointer: calling it calls the bus function. Delete the stub `init_seq` line and the two stub lines in the function.

- **WHERE:** `src/ssd1306.c`, `init_seq` and `ssd1306_init`.
- **CHECK:** `make t T=m3_02` -> `test_init:PASS`.

STEP 6 · TODO M3-02/6 · `ssd1306_flush` · [src: M3-02 step 6]

- **WHAT:** Send the picture: one command write, 8 page writes.
- **INGREDIENTS:** cheat sheet "Flush".
- **HOW:**

```
static const uint8_t cmd[7] = { <7 BYTES> };
uint8_t buf[129];
int r = s_bus(SSD1306_ADDR, cmd, sizeof cmd);
if (r != 0) return r;
for (unsigned page = 0; page < 8u; page++) {
    buf[0] = <DATA CONTROL>;
    for (unsigned i = 0; i < 128u; i++) buf[1 + i] = g_fb[page * 128u + i];
    r = s_bus(SSD1306_ADDR, buf, sizeof buf);
    if (r != 0) return r;
}
return 0;
```

name	value
7 BYTES	`0x00, 0x21, 0x00, 0x7F, 0x22, 0x00, 0x07`
DATA CONTROL	`0x40`

- **WHERE:** `src/ssd1306.c`, `ssd1306_flush`. Delete the `return -1;` stub.
- **CHECK:** `make t T=m3_02` -> `6 Tests 0 Failures`, `OK`.

STEP 7 · TODO M3-02/7 · pixels on the real OLED · [src: M3-02 step 7]

- **WHAT:** See your first pixels.
- **INGREDIENTS:** `i2c_init`, `ssd1306_init(i2c_write)`, `fb_clear`, `fb_pixel`, `ssd1306_flush`.
- **HOW:** in `main`, after `i2c_init();`:

```
ssd1306_init(i2c_write);
fb_clear();
for (unsigned i = 0; i < 64u; i++) fb_pixel(i, i, 1);   /* a diagonal */
fb_pixel(0, 0, 1);  fb_pixel(127, 0, 1);  fb_pixel(0, 63, 1);  fb_pixel(127, 63, 1);
ssd1306_flush();
```

`i2c_write` here is a function name used as a value: it passes the function itself, not a call.

- **WHERE:** `src/main.c`, `main`.
- **CHECK:** `make flash`: a diagonal line from the top-left corner down to the bottom edge at column 63, plus a dot in each of the four corners.

STEP 8 · finish (no TODO)

- Add `test_m3_02` to `tests.active`, `make test-active`, commit `M3-02 done`.

Verify

Do this	Expect
`make t T=m3_02`	`6 Tests 0 Failures 0 Ignored`, `OK`
Flash with the OLED wired	diagonal line plus four corner dots
Change the loop to `fb_pixel(i, 20, 1)` for i up to 127	one horizontal line across row 20
Change it to `fb_pixel(10, i, 1)` for i up to 63	one vertical line at column 10

If it fails, look here	Likely cause
blank screen	`ssd1306_init` bytes wrong (count 25), or the init write NACKed: print its return value
screen full of noise	the init was never sent; or flush sends 128 bytes instead of 129 (control byte missing)
picture upside down or mirrored	bytes `A1` and `C8` in `init_seq` wrong
lines have gaps of 8 rows	`fb_bit` uses `y / 8` instead of `y % 8`
`test_flush` fails at length	each page write is 1 + 128 = 129 bytes
screen shifted by 2 columns	some modules use the SH1106 chip instead: not covered by this kit

Self-check (answer aloud)

1. Which byte and bit hold pixel (5, 9)?
2. What does the control byte 0x00 mean? 0x40?
3. Why is the bus passed as a function pointer?
4. How many bytes does one flush send in total?

Hint ladder (open one at a time)

1. STEP 4 is the matrix example `m_pixel` with `fb_index(x, y)` and `fb_bit(y)`.
2. Pattern: bound check, `i = index`, `m = 1 << bit`, then `|=` or `&= ~`.
3. Template with the values: `if (<X> >= <W> || <Y> >= <H>) return;` with `<W>` = 128u and `<H>` = 64u; `g_fb[i] |= m;` for on, `g_fb[i] &= (uint8_t)~m;` for off.

Title: M3-03 plot_wave, grid and 5x7 text

Milestone: M3 Display

Labels: m3, blank-level-3, hardware, host-tests

M3-03 plot_wave, grid and 5x7 text

START HERE

1. Type `make t T=m3_03`. Red tests.
2. Open `src/drills.c`, find `@TODO(M3-03/1)`: `clamp_i` is 3 lines.
3. Then `src/gfx.c`: `draw_grid` (STEP 2), `text_draw` (STEP 3), `plot_wave` (STEP 4).

Starts from / Ends with

Starts from	Ends with
M3-02: `fb_pixel`, `fb_clear`, `ssd1306_flush`, `mul_div`.	`clamp_i` tested.
	`draw_grid`, `text_draw`, `plot_wave` tested on your PC.
	Grid, text and a test waveform on the OLED.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `fb_index`, `fb_bit`, `mul_div`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

Everything is pixels. Grid, text and waveform are three loops that call `fb_pixel`.

The grid is dots, so it does not hide the trace. Eight divisions across (16 px each), six down (8 px each, rows 16 to 63).

Text from a table. The font is 5 columns by 7 rows per character, stored as 5 bytes (one per column). Bit 0 of a column byte is the TOP row. To draw: for each column 0 to 4, for each row 0 to 6, if the bit is set, plot a pixel at (x + col, y + row). Move 6 pixels right for the next character (5 + 1 gap). Characters 32 (space) to 126 are in the table `font5x7[c - 32]`.

The waveform. Each sample is 0 to 4095 counts. Steps:

1. counts to millivolts: `mul_div(sample, 3300, 4095)`.
2. millivolts to rows above the bottom: `mul_div(mv, 8, vdiv_mv)`. (8 pixel rows per division.)
3. y = 63 minus those rows. Row 63 is 0 V.
4. Keep y inside 16 to 63 with `clamp_i`: that is "clipping".
5. Draw the pixel at (x, y). Also draw the vertical gap to the previous y, so square-wave edges look like edges.

Clipping is a lesson. At 500 mV/div the screen holds only 3.0 V, so a 3.3 V signal is clipped to the top row.

Worked example: an 8-bar bar graph

```
/* WORKED EXAMPLE (M3-03): an 8-bar bar graph on the 8x8 matrix. vals[x] is 0..255; bar height 0..8 rows. */
#include <stdint.h>
extern void m_pixel(unsigned x, unsigned y, int on);
extern uint32_t mul_div(uint32_t a, uint32_t b, uint32_t c);

static int clampi(int v, int lo, int hi) { return v < lo ? lo : (v > hi ? hi : v); }

void bars(const uint8_t *vals, unsigned n)
{
    for (unsigned x = 0; x < n && x < 8; x++) {
        int h = clampi((int)mul_div(vals[x], 8, 255), 0, 8);     /* height in rows     */
```

```
        for (int r = 0; r < h; r++) {
            m_pixel(x, 7u - (unsigned)r, 1);                    /* grow upward from row 7 */
        }
    }
}
```

Same chain as your `plot_wave`: scale with `mul_div`, keep in range with a clamp, then plot pixels column by column.

Cheat sheet

Item	Value
Text rows	rows 0 to 15: line 1 at y = 0, line 2 at y = 8
Plot rows	16 to 63. Row 63 = 0 V
Grid	horizontal dotted lines at y = 16, 24, 32, 40, 48, 56, 63, a dot at every 4th x (0, 4, 8, ...). Vertical dotted lines at x = 0, 16, 32, 48, 64, 80, 96, 112 and 127, a dot at every even y from 16 to 62
Font	`font5x7[95][5]` given. Index = character - 32. Bit 0 of a column byte = top row
Glyph 'A'	columns `7E 11 11 11 7E`
Character width	5 columns + 1 gap = advance 6
Clip text	`fb_pixel` already ignores x >= 128
`plot_wave(s, n, vdiv_mv)`	n at most 128 samples. For x = 0..n-1: `mv = mul_div(s[x], 3300, 4095)`, `rows = mul_div(mv, 8, vdiv_mv)`, `y = clamp_i(63 - rows, 16, 63)`
Vertical join	draw all y from `min(prev, y)` to `max(prev, y)` in column x (just `y` for x = 0)
Examples	4095 counts at 1000 mV/div: mv 3300, rows 26, y 37. At 500 mV/div: rows 52, y = 11 -> clamped to 16. 0 counts: y 63
`clamp_i(v, lo, hi)`	`if (v < lo) return lo; if (v > hi) return hi; return v;`
Drawing functions	`fb_pixel(x, y, on)`, `fb_clear()` (M3-02)

Card C5: `mul_div(a, b, c)` = a * b / c in 64 bits.

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M3-02. `make t T=m3_03`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M3-03/1 · `clamp_i` · [src: M3-03 step 1]

- **WHAT:** Force a number into a range.
- **INGREDIENTS:** two comparisons.
- **HOW:**

```
if (v < <LO>) return <LO>;
if (v > <HI>) return <HI>;
return v;
```

v	lo	hi	result
5	0	10	5
-3	0	10	0
11	0	10	10
16	16	63	16

- **WHERE:** `src/drills.c`, `clamp_i`. Delete the stub `return v;`.
- **CHECK:** `make t T=m3_03` -> `test_clamp_i:PASS`.

STEP 2 · TODO M3-03/2 · `draw_grid` · [src: M3-03 step 2]

- **WHAT:** Draw the dotted grid.
- **INGREDIENTS:** the grid row from the cheat sheet; `fb_pixel`.
- **HOW:**

```
static const uint8_t rows[7] = { 16, 24, 32, 40, 48, 56, 63 };
for (unsigned r = 0; r < 7u; r++)
    for (unsigned x = 0; x < 128u; x += 4u) fb_pixel(x, rows[r], 1);
for (unsigned k = 0; k < 9u; k++) {
    unsigned x = (k == 8u) ? 127u : k * 16u;
```

```
    for (unsigned y = 16; y < 64u; y += 2u) fb_pixel(x, y, 1);
}
```

line type	positions	dot spacing
horizontal	y = 16, 24, ..., 56, 63	every 4th x
vertical	x = 0, 16, ..., 112, 127	every 2nd y from 16

- **WHERE:** `src/gfx.c`, `draw_grid`.
- **CHECK:** `make t T=m3_03` -> `test_grid:PASS`.

STEP 3 · TODO M3-03/3 · `text_draw` · [src: M3-03 step 3]

- **WHAT:** Draw a string with the 5x7 font.
- **INGREDIENTS:** `font5x7`, `fb_pixel`.
- **HOW:**

```
for (; *<S>; <S>++) {
    unsigned c = (unsigned char)*<S>;
    if (c >= 32u && c <= 126u) {
        const uint8_t *g = font5x7[c - 32u];          /* the 5 column bytes */
        for (unsigned col = 0; col < 5u; col++)
            for (unsigned row = 0; row < 7u; row++)
                if (g[col] & (1u << row)) fb_pixel(x + col, y + row, 1);
    }
    x += 6u;
}
```

name	meaning
`g[col]`	the byte for one column
`1u << row`	the bit for one row (bit 0 = top)
`x += 6u`	next character starts 6 px right

`g` is a pointer into the table: `g[col]` is the value at `g + col`. Delete the stub line.

- **WHERE:** `src/gfx.c`, `text_draw`.
- **CHECK:** `make t T=m3_03` -> `test_text:PASS`.

STEP 4 · TODO M3-03/4 · `plot_wave` · cards C5 · [src: M3-03 step 4, card C5, drill clamp_i]

- **WHAT:** Draw samples as a connected trace.
- **INGREDIENTS:** `mul_div`, `clamp_i`, `fb_pixel`.
- **HOW:**

```
int prev = 0;
if (n > 128u) n = 128u;
for (unsigned x = 0; x < n; x++) {
    uint32_t mv   = mul_div(s[x], <FULL MV>, <FULL COUNTS>);
    uint32_t rows = mul_div(mv, <ROWS PER DIV>, vdiv_mv);
    int y = clamp_i(63 - (int)rows, <TOP>, <BOTTOM>);
    int lo = y, hi = y;
    if (x > 0u) { if (prev < lo) lo = prev; if (prev > hi) hi = prev; }
    for (int yy = lo; yy <= hi; yy++) fb_pixel(x, (unsigned)yy, 1);
    prev = y;
}
```

name	value
FULL MV / FULL COUNTS	`3300u` / `4095u`
ROWS PER DIV	`8u`
TOP / BOTTOM	`16` / `63`

This is card C5 twice (with 3300/4095, then 8/vdiv).

- **WHERE:** `src/gfx.c`, `plot_wave`.
- **CHECK:** `make t T=m3_03` -> `6 Tests 0 Failures`, `OK`.

STEP 5 · demo on the OLED (no TODO)

- **WHAT:** See text, grid and a square wave.
- **HOW:** in `main`, replace the diagonal from M3-02 by:

```
uint16_t demo[128];
for (int i = 0; i < 128; i++) demo[i] = ((i % 32) < 16) ? 4095 : 0;
fb_clear();
```

```
text_draw(0, 0, "SCOPE TEST");
draw_grid();
plot_wave(demo, 128, 1000);
ssd1306_flush();
```

- **CHECK:** the OLED shows `SCOPE TEST` at the top, a dotted grid below, and a square wave jumping between the bottom row and row 37, with a period of 32 pixels (2 divisions).

STEP 6 · finish (no TODO)

- Add `test_m3_03` to `tests.active`, `make test-active`, commit `M3-03 done`.

Verify

Do this	Expect
`make t T=m3_03`	`6 Tests 0 Failures 0 Ignored`, `OK`
Flash the demo	text on top, grid, square wave of 4 pulses across the screen
`plot_wave(demo, 128, 500)`	the high level is clipped to row 16 (the top of the plot area)
`plot_wave(demo, 128, 250)`	same clipping, high level on row 16

If it fails, look here	Likely cause
wave is upside down	y = rows - 63 instead of 63 - rows
wave covers the text	`clamp_i` top is 0, not 16
edges are not vertical	the join loop is missing
text mirrored	rows and columns swapped in the loops
text garbled	`font5x7[c]` instead of `font5x7[c - 32]`
grid hides the trace	dots only: step by 4 and by 2, not 1

Self-check (answer aloud)

1. Which row is 0 V? Which row is 3.3 V at 1 V/div?
2. Why does `plot_wave` clamp?
3. What does bit 0 of a font byte mean?
4. Which card does the counts to rows scaling use?

Hint ladder (open one at a time)

1. STEP 4: do the two `mul_div` lines first; they are card C5.
2. Pattern: `mv = mul_div(sample, A, B); rows = mul_div(mv, C, vdiv_mv); y = clamp_i(63 - rows, TOP, BOTTOM);`
3. Template with the values: `<A>` = 3300, `<B>` = 4095, `<C>` = 8, `<TOP>` = 16, `<BOTTOM>` = 63.

Milestone M4: Trigger and controls

Trigger, scale tables, buttons and the UI state machine.

Title: M4-01 trig_find: the edge trigger
Milestone: M4 Trigger and controls
Labels: m4, blank-level-4, host-tests

M4-01 trig_find: the edge trigger

START HERE

1. Type `make t T=m4_01`. Red tests.
2. Open `src/trigger.c`. Read the recipe below, then start with `@TODO(M4-01/1)` (`mv_to_counts`, ONE line).
3. Do `trig_find` (STEP 2) only after reading the `first_above` example.

Starts from / Ends with

Starts from	Ends with
M3-03: drawing works, `mul_div`, `clamp_i`.	`mv_to_counts` (card C5).
	`trig_find` for rising and falling edges.
	`trig_start` (auto mode).

Recipe (what, in order, with its data)

#	Function	Input data	Output
1	`mv_to_counts(mv)`	trigger level in mV	ADC counts: `mv * 4095 / 3300` (card C5)
2	`trig_find(s, n_scan, level, rising)`	capture, how many samples to scan, level in counts, edge	index of the crossing (>= 1) or -1
3	`trig_start(idx)`	the index from step 2	first sample drawn: idx, or 0 if idx < 0

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `clamp_i`, `i2c_ccr`, `fb_index`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

What a trigger does. A scope shows a repeating signal as a still picture because every sweep starts at the same point of the wave: where it crosses a chosen level going up (rising) or going down (falling). Without it the picture slides around.

Detecting a crossing. Look at two neighbours. Rising: the earlier sample is below the level and the later one is at or above it. Falling: the earlier sample is at or above the level, the later one below. So the loop starts at index 1 (it needs a "previous" sample).

Why only the first 128 samples? The screen shows 128 samples starting at the trigger. The buffer has 256. If the trigger is found in the first 128, the window `[idx, idx + 127]` always fits.

No trigger = auto mode. If no crossing exists (a flat line, or a signal that never reaches the level), start at sample 0 so you still see something.

Level in counts. The user sets the level in millivolts (default 1650 mV). Samples are counts, so convert once: `mv * 4095 / 3300`, using card C5.

Worked example: `first_above()`

```
/* WORKED EXAMPLE (M4-01): first index where the value goes above a threshold, or -1. */
#include <stdint.h>
```

```
int first_above(const uint16_t *a, unsigned n, uint16_t thr)
{
    for (unsigned i = 0; i < n; i++) {
        if (a[i] > thr) return (int)i;     /* found: stop right here   */
    }
    return -1;                             /* looked at everything: none */
}
```

`trig_find` is the same loop with a two-sample test, starting at 1, and an extra `rising` switch.

Cheat sheet

Item	Value
Counts per mV	4095 / 3300
`mv_to_counts` examples	0 -> 0, 1000 -> 1240, 1650 -> 2047, 3300 -> 4095
Rising crossing at i	`s[i-1] < level && s[i] >= level`
Falling crossing at i	`s[i-1] >= level && s[i] < level`
Loop	`for (unsigned i = 1; i < n_scan; i++)`
No crossing	return -1
`trig_start`	negative index -> 0, otherwise the index itself
In the scope	scan 128 samples (`n_scan` = 128), level = `mv_to_counts(ui.level_mv)`

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M3-03. `make t T=m4_01`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M4-01/1 · `mv_to_counts` · card C5 · [src: M4-01 step 1, card C5]

- **WHAT:** Millivolts to ADC counts.
- **INGREDIENTS:** `mul_div`.
- **HOW (recipe):**
  1. Call `mul_div` with three numbers: the level in mV, then 4095, then 3300.
  2. Cast the result to `uint16_t` and return it.

mv	result
0	0
1000	1240
1650	2047
3300	4095

This is card C5 again with (4095, 3300): the mirror of `adc_to_mv`.

- **WHERE:** `src/trigger.c`, `mv_to_counts`.
- **CHECK:** `make t T=m4_01` -> `test_mv_to_counts:PASS`.

STEP 2 · TODO M4-01/2 · `trig_find` · [src: M4-01 step 2, worked example first_above]

- **WHAT:** Find the first rising or falling crossing.
- **INGREDIENTS:** the two-neighbour tests from the cheat sheet.
- **HOW (recipe):**
  1. Loop `i` from 1 up to (not including) `n_scan`.
  2. If `rising` is true: when the previous sample (`i - 1`) is below `level` AND sample `i` is at or above it, return `i`.
  3. Otherwise (falling): when the previous sample is at or above `level` AND sample `i` is below it, return `i`.
  4. After the loop ends with no crossing, return -1.

example	result
`{0,0,100,200,300,...}`, level 150, rising	3
`{300,200,100,0,...}`, level 150, falling	2
`{200,300,400,...}`, level 150, rising	-1 (starts above)
crossing at index 5 but `n_scan` = 5	-1

- **WHERE:** `src/trigger.c`, `trig_find`. Delete the stub lines.
- **CHECK:** `make t T=m4_01` -> `test_rising`, `test_falling`, `test_none`, `test_equal_level` all `:PASS`.

STEP 3 · TODO M4-01/3 · `trig_start` · [src: M4-01 step 3]

- **WHAT:** Where does the drawn window start?
- **INGREDIENTS:** `idx` (-1 = no trigger).
- **HOW (recipe):** if `idx` is negative return 0, otherwise return `idx` as an unsigned number.

idx	result
-1	0
7	7

- **WHERE:** `src/trigger.c`, `trig_start`.
- **CHECK:** `make t T=m4_01` -> `6 Tests 0 Failures`, `OK`.

STEP 4 · finish (no TODO)

- Add `test_m4_01` to `tests.active`, `make test-active`, commit `M4-01 done`.
- **Optional board demo:** in `main`, after a capture of the 1 kHz PWM at 32000 Hz, do `int t = trig_find(g_cap, 128u, mv_to_counts(1650u), 1); unsigned st = trig_start(t);` then `plot_wave(&g_cap[st], 128u, 1000u)` and flush. The pulse's rising edge now sits at the left edge, every refresh.

Verify

Do this	Expect
`make t T=m4_01`	`6 Tests 0 Failures 0 Ignored`, `OK`
Optional board demo, PA1 to PA2, 25 % PWM	the picture does not slide: the rising edge is at column 0
Same demo with `trig_start(-1)` forced	the picture slides from one refresh to the next

If it fails, look here	Likely cause
`test_equal_level` red	`>=` / `<` swapped between rising and falling
`test_none` red on the 5-sample case	loop bound uses 8 instead of `n_scan`
crash or wrong value for i = 0	the loop must start at 1
`mv_to_counts(1650)` gives 2048	rounding: this kit truncates, expect 2047

Self-check (answer aloud)

1. Why does the loop start at 1?
2. What does it mean when `trig_find` returns -1, and what does the scope do then?
3. Why scan only the first 128 samples?
4. Which card converts mV to counts?

Hint ladder (open one at a time)

1. It is `first_above` with two samples instead of one, and a rising/falling switch.
2. Pattern: `if (prev OP level && cur OP2 level) return i;`
3. Template with the values: rising `prev <` and `cur >=`; falling `prev >=` and `cur <`. Loop `i` from 1 up to (not including) `n_scan`.

Title: M4-02 Time/div and volts/div tables, sample-rate math

Milestone: M4 Trigger and controls

Labels: m4, blank-level-4, host-tests

M4-02 Time/div and volts/div tables, sample-rate math

START HERE

1. Type `make t T=m4_02`. Red tests.
2. Open `src/ui.c`. Find `@TODO(M4-02/1)`: the table of 8 numbers. Copy them from the cheat sheet.
3. Read the baud-rate table example first.

Starts from / Ends with

Starts from	Ends with
M4-01: trigger works. `sampler_timing` (M2-02), `mul_div`, `clamp_i`.	Two lookup tables: `tdiv_us` (8 values), `vdiv_mv` (3 values).
	`tdiv_to_fs`: the sample rate for each time/div.
	`ui_init` defaults and `step_index`.

Recipe (what, in order, with its data)

#	Item	Data	Result
1	`tdiv_us[8]`	100, 200, 500, 1000, 2000, 5000, 10000, 20000	array of microseconds per division
2	`vdiv_mv[3]`	1000, 500, 250	array of millivolts per division
3	`tdiv_to_fs(idx)`	`16 * 1000000 / tdiv_us[idx]` (card C5)	160000, 80000, 32000, 16000, 8000, 3200, 1600, 800
4	`ui_init(u)`	mode `M_TDIV`, tdiv 3, vdiv 0, level 1650, rising 1	defaults
5	`step_index(idx, delta, max)`	`clamp_i(idx + delta, 0, max)`	new index, never outside 0..max

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `mul_div`, `clamp_i`, `smpr_field`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

Table-driven settings. A scope has a few fixed choices: 100 us, 200 us, 500 us, ... per division. Store the choices in an array. The UI only keeps an index. Buttons change the index. Everything else is looked up: `tdiv_us[idx]`. No big `switch`, no magic numbers scattered around.

Where the sample rate comes from. The screen shows 8 divisions across 128 pixels, so each division is 16 pixels wide, and each pixel is one sample. If one division lasts `t` microseconds, 16 samples must fit in `t`, so `fs = 16 / t`. In integer maths: `fs = 16 * 1,000,000 / t_us` (card C5).

time/div	samples per division	sample rate
100 us	16	160 kHz
1 ms	16	16 kHz
20 ms	16	800 Hz

Why these rates? All eight divide 72 MHz exactly, so the timer makes them without rounding error (your `sampler_timing` from M2-02 proves that, and the test re-checks it).

Volts per division. The plot area has 6 divisions of 8 pixels. At 1 V/div it spans 6 V (you only use 3.3 V of it). At 500 mV/div it spans 3.0 V: a 3.3 V signal clips. At 250 mV/div it spans 1.5 V.

Stepping safely. UP and DOWN change the index by +1 or -1. It must stop at the ends: that is `clamp_i`.

Aliasing warning. At slow rates (800 Hz) a 1 kHz signal is sampled less than twice per period: the picture and frequency are wrong. Remember this for M6.

Worked example: a table-driven setting (baud rates)

```
/* WORKED EXAMPLE (M4-02): a table-driven setting. The index is the setting; the table holds the numbers. */
#include <stdint.h>
#define BAUD_COUNT 4
const uint32_t baud_tab[BAUD_COUNT] = { 9600, 19200, 38400, 115200 };
const char *const baud_label[BAUD_COUNT] = { "9600", "19k2", "38k4", "115k" };

static int clampi(int v, int lo, int hi) { return v < lo ? lo : (v > hi ? hi : v); }

int baud_step(int idx, int delta) { return clampi(idx + delta, 0, BAUD_COUNT - 1); }   /* UP / DOWN button */

uint32_t brr_from_idx(int idx, uint32_t pclk)       /* the table feeds a formula */
{
    return (pclk + baud_tab[idx] / 2u) / baud_tab[idx];
}
```

Your tables are the same idea with different numbers, and your formula is `16 * 1000000 / tdiv_us[idx]` instead of the BRR formula.

Cheat sheet

Index	tdiv_us	tdiv_label	fs (Hz)
0	100	100us	160000
1	200	200us	80000
2	500	500us	32000
3	1000	1ms	16000
4	2000	2ms	8000
5	5000	5ms	3200
6	10000	10ms	1600
7	20000	20ms	800

Index	vdiv_mv	vdiv_label
0	1000	1V
1	500	500mV
2	250	250mV

Item	Value
Labels	already given in `ui.c` (`tdiv_label`, `vdiv_label`)
Defaults	`mode = M_TDIV`, `tdiv_idx = 3`, `vdiv_idx = 0`, `level_mv = 1650`, `rising = 1`
`ui_t` fields	`mode`, `tdiv_idx`, `vdiv_idx`, `level_mv`, `rising`
`tdiv_to_fs`	`mul_div(16u, 1000000u, tdiv_us[tdiv_idx])`
`step_index`	`clamp_i(idx + delta, 0, max)`
Sizes	`TDIV_COUNT` = 8, `VDIV_COUNT` = 3

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M4-01. `make t T=m4_02`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M4-02/1 · `tdiv_us` · [src: M4-02 step 1]

- **WHAT:** Fill the time/div table.
- **INGREDIENTS:** the first table in the cheat sheet.
- **HOW:** `const uint32_t tdiv_us[TDIV_COUNT] = { <8 NUMBERS, comma separated> };`

index	0	1	2	3	4	5	6	7
value	100	200	500	1000	2000	5000	10000	20000

- **WHERE:** `src/ui.c`, replace the stub line `{ 1 }`.
- **CHECK:** `make t T=m4_02` -> `test_tables` still fails (STEP 2 next).

STEP 2 · TODO M4-02/2 · `vdiv_mv` · [src: M4-02 step 2]

- **WHAT:** Fill the volts/div table.
- **INGREDIENTS:** the second table in the cheat sheet: 1000, 500, 250.
- **HOW:** `const uint32_t vdiv_mv[VDIV_COUNT] = { <3 NUMBERS> };`

index	0	1	2
value	1000	500	250

- **WHERE:** `src/ui.c`.
- **CHECK:** `make t T=m4_02` -> `test_tables:PASS` and `test_labels:PASS`.

STEP 3 · TODO M4-02/3 · `tdiv_to_fs` · card C5 · [src: M4-02 step 3, card C5]

- **WHAT:** Sample rate for a time/div index.
- **INGREDIENTS:** 16 pixels per division, 1,000,000 microseconds per second.
- **HOW:** `return mul_div(<PIXELS>, <US PER S>, tdiv_us[<IDX>]);`

name	value
PIXELS / US PER S	`16u` / `1000000u`

idx	tdiv_us	result
0	100	160000
3	1000	16000
7	20000	800

This is card C5 again with (16, 1000000, tdiv_us).

- **WHERE:** `src/ui.c`, `tdiv_to_fs`.
- **CHECK:** `make t T=m4_02` -> `test_tdiv_to_fs:PASS`, `test_rates_fit_timer:PASS`.

STEP 4 · TODO M4-02/4 · `ui_init` · [src: M4-02 step 4]

- **WHAT:** Start with the default settings.
- **INGREDIENTS:** the "Defaults" row of the cheat sheet; the enum name `M_TDIV` (given).
- **HOW:** one assignment per field through the pointer: `u->mode = ...;`

field	value
`u->mode`	`M_TDIV`
`u->tdiv_idx`	`3`
`u->vdiv_idx`	`0`
`u->level_mv`	`1650`
`u->rising`	`1`

`u->mode` means "the field `mode` of the struct that `u` points to".

- **WHERE:** `src/ui.c`, `ui_init`.
- **CHECK:** `make t T=m4_02` -> `test_ui_init:PASS`.

STEP 5 · TODO M4-02/5 · `step_index` · [src: M4-02 step 5, drill clamp_i]

- **WHAT:** Move an index by one step, never outside 0..max.
- **INGREDIENTS:** `clamp_i`.
- **HOW:** `return clamp_i(<IDX> + <DELTA>, <LO>, <MAX>);` with `<LO>` = 0.

idx	delta	max	result
3	+1	7	4
7	+1	7	7
0	-1	7	0

idx	delta	max	result
2	+1	2	2

- **WHERE:** `src/ui.c`, `step_index`. Delete the stub `return idx;`.
- **CHECK:** `make t T=m4_02` -> `6 Tests 0 Failures`, `OK`.

STEP 6 · finish (no TODO)

- Add `test_m4_02` to `tests.active`, `make test-active`, commit `M4-02 done`.

Verify

Do this	Expect
`make t T=m4_02`	`6 Tests 0 Failures 0 Ignored`, `OK`
Work out by hand: 16 x 1,000,000 / 5000	3200
Check by hand: 72,000,000 / 3200	22500 (a whole number: no rounding error)

If it fails, look here	Likely cause
`test_tables` fails at element 0	stub `{ 1 }` still in place
`test_tdiv_to_fs` gives 0	divided first: use `mul_div` (card C5)
`test_rates_fit_timer` fails	a table value is mistyped
`test_ui_init` red	the struct field name is misspelled (`tdiv_idx`, not `tdiv`)

Self-check (answer aloud)

1. Why 16 samples per division?
2. What does the UI store: the setting, or an index into a table? Why?
3. At 500 mV/div, what is the largest voltage on screen?
4. What does `step_index(7, +1, 7)` return, and why?

Hint ladder (open one at a time)

1. STEP 3 is card C5 with the numbers from the recipe.
2. Pattern: `mul_div(a, b, c)`.
3. Template with the values: `mul_div(<A>, <B>, tdiv_us[tdiv_idx])` with `<A>` = 16u and `<B>` = 1000000u.

Title: M4-03 Button debouncer and the table-driven UI state machine

Milestone: M4 Trigger and controls

Labels: m4, blank-level-4, hardware, host-tests

M4-03 Button debouncer and the table-driven UI state machine

START HERE

1. Type `make t T=m4_03`. Red tests.
2. Open `src/ui.c`, find `@TODO(M4-03/1)`: `debounce_update`. Read the traffic-light example first.
3. The state table `ui_table` is given. You write the action functions and the loop that reads the table.

Starts from / Ends with

Starts from	Ends with
M4-02: tables, `ui_init`, `step_index`, `reached`/`elapsed` (M1-04).	`debounce_update`: one clean event per press.
	The seven small action functions.
	`ui_handle`: a loop over `ui_table`.
	`buttons_init` and `buttons_read` for PB12, PB13, PB14.

Recipe (what, in order, with its data)

#	Function	Data	Behaviour
1	`debounce_update(d, raw, now)`	`raw` 1 = pressed; 20 ms (`DEBOUNCE_MS`)	returns 1 exactly once per press
2	seven `act_...` functions	`step_index`, `clamp_i`, ranges below	change one field of `ui_t`
3	`ui_handle(u, ev)`	`ui_table` rows `{mode, event, action, next_mode}`	find the row, run its action, switch mode
4	`buttons_init()`	PB12, PB13, PB14 nibble 0x8; BSRR bits 12..14	inputs with pull-up
5	`buttons_read()`	`((~IDR) >> 12) & 7`	bit 0 MODE, bit 1 UP, bit 2 DOWN; 1 = pressed

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `reached`, `elapsed`, `clamp_i`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

Buttons bounce. A metal contact does not close cleanly: it chatters for a few milliseconds. Read it raw and one press looks like 5 presses.

Debounce rule. Only believe a new state once the pin has held the same value for 20 ms. Keep three things per button: `stable` (what you believe), `last` (the latest raw reading) and `t_change` (when `last` last changed). Each call:

1. If raw differs from `last`: remember it and the time. Not stable yet. Return 0.
2. Else if raw differs from `stable` and the pin has held for 20 ms (`elapsed(now, t_change) >= 20`): accept it: `stable = raw`. Return `raw` (so you get 1 only for a press).
3. Otherwise return 0.

That uses card C6 (`elapsed`) so it survives the millis wrap.

Active-low buttons. The button connects the pin to GND. The internal pull-up holds it HIGH when open. So pressed = 0. `buttons_read` inverts the pins so that 1 = pressed.

Pull-up recipe on the F103. Nibble 0x8 means "input with pull". The ODR bit then chooses: ODR = 1 gives pull-UP. You set ODR bits through BSRR.

State machine in a table. Each row says: "if I am in MODE m and event e happens, run action a, then go to mode n." One loop finds the row. Adding a feature = adding a row, not rewriting code.

mode	MODE button	UP	DOWN
M_TDIV (T)	go to M_VDIV	tdiv index +1	tdiv index -1
M_VDIV (V)	go to M_LEVEL	vdiv index +1	vdiv index -1
M_LEVEL (L)	go to M_EDGE	level +100 mV	level -100 mV
M_EDGE (E)	go to M_TDIV	toggle rising/falling	toggle rising/falling

Worked example: a one-button debouncer plus a traffic-light state table

```
/* WORKED EXAMPLE (M4-03): a one-button debouncer plus a traffic-light state table. */
#include <stdint.h>
#include <stddef.h>

/* ---- debouncer: a press counts once the pin has been steady for 20 ms ---- */
typedef struct { uint8_t stable, last; uint32_t t_change; } deb_t;
static uint32_t elapsed(uint32_t now, uint32_t start) { return now - start; }

int deb_update(deb_t *d, int raw, uint32_t now)
{
    raw = (raw != 0);
    if (raw != d->last) { d->last = (uint8_t)raw; d->t_change = now; return 0; }   /* it moved: restart the
        timer */
    if (raw != d->stable && elapsed(now, d->t_change) >= 20u) { d->stable = (uint8_t)raw; return raw; }
    return 0;
}

/* ---- state table: {current state, event, next state} ---- */
typedef enum { RED, GREEN, YELLOW } light_t;
typedef enum { EV_PRESS, EV_TIMEOUT } ev_t;
typedef struct { light_t state; ev_t ev; light_t next; } row_t;
static const row_t table[] = {
    { RED,    EV_PRESS,   GREEN  },
    { GREEN,  EV_TIMEOUT, YELLOW },
    { YELLOW, EV_TIMEOUT, RED    },
};

light_t light_handle(light_t s, ev_t ev)
{
    for (unsigned i = 0; i < sizeof table / sizeof table[0]; i++) {
        if (table[i].state == s && table[i].ev == ev) return table[i].next;   /* found the row */
    }
    return s;                                                                 /* no row: stay  */
}
```

`deb_update` is your `debounce_update`. `table` and `light_handle` are your `ui_table` and `ui_handle`: loop over rows, match, return the next state.

Cheat sheet

Item	Value
Debounce time	`DEBOUNCE_MS` = 20
`debounce_t`	`{ uint8_t stable; uint8_t last; uint32_t t_change; }`
Events	`EV_NONE` 0, `EV_MODE` 1, `EV_UP` 2, `EV_DOWN` 3
Modes	`M_TDIV`, `M_VDIV`, `M_LEVEL`, `M_EDGE`
Ranges	tdiv index 0..7 (`TDIV_COUNT - 1`), vdiv index 0..2 (`VDIV_COUNT - 1`), level 0..3300 mV in steps of 100
Action rules	tdiv and vdiv: `step_index(current, +1 or -1, max)`. Level: `clamp_i(level +/- 100, 0, 3300)`. Edge: flip `rising`
Table row type	`ui_row_t { mode, ev, action, next }` (given)
Table	`ui_table[]` (given), 12 rows
Buttons	PB12 MODE, PB13 UP, PB14 DOWN. GPIOB clock = APB2ENR bit 3 (`RCC_APB2ENR_IOPBEN`). PB12 is CRH shift 16, PB13 shift 20, PB14 shift 24

Item	Value
Nibble	0x8 = input with pull-up/pull-down. BSRR bits 12, 13, 14 = pull-UP
GPIOB	`GPIOB->IDR` offset 0x08, `GPIOB->BSRR` offset 0x10
`buttons_read`	read IDR, invert, shift right 12, mask with 7

Card C6: `elapsed = now - start;` and card C2: `gpio_cfg(GPIOx, pin, NIBBLE)`.

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M4-02. `make t T=m4_03`. **CHECK:** `8 Tests`, failures.

STEP 1 · TODO M4-03/1 · `debounce_update` · card C6 · [src: M4-03 step 1, card C6, worked example deb_update]

- **WHAT:** Turn a bouncing pin into one clean press event.
- **INGREDIENTS:** `elapsed`, `DEBOUNCE_MS`, the three fields of `debounce_t`.
- **HOW (recipe):**
  1. Turn `raw` into exactly 0 or 1.
  2. If `raw` differs from the `last` field: store it in `last`, store `now_ms` in `t_change`, return 0.
  3. Else, if `raw` differs from the `stable` field AND `elapsed(now_ms, t_change)` is at least `DEBOUNCE_MS`: store `raw` in `stable` and return `raw`.
  4. Otherwise return 0.

time (ms)	raw	returns
0	1	0
10	1	0
20	1	1 (the press)
30	1	0

This is card C6 again (`elapsed`) with a 20 ms limit. `d->last` means "the field `last` of the struct that `d` points to".

- **WHERE:** `src/ui.c`, `debounce_update`.
- **CHECK:** `make t T=m4_03` -> `test_debounce_clean_press`, `test_debounce_bounce`, `test_debounce_wrap` all `:PASS`.

STEP 2 · TODO M4-03/2 · the seven actions · [src: M4-03 step 2, drill clamp_i]

- **WHAT:** One line in each action function.
- **INGREDIENTS:** `step_index`, `clamp_i`; ranges from the cheat sheet.
- **HOW (recipe):** each action changes exactly one field of `*u`. The first four reuse `step_index` (M4-02), the next two reuse `clamp_i` (M3-03).

function	field	change	range
`act_tdiv_up` / `act_tdiv_down`	`tdiv_idx`	+1 / -1	0 to `TDIV_COUNT - 1`
`act_vdiv_up` / `act_vdiv_down`	`vdiv_idx`	+1 / -1	0 to `VDIV_COUNT - 1`
`act_level_up` / `act_level_down`	`level_mv`	+100 / -100	0 to 3300
`act_edge_toggle`	`rising`	becomes the opposite (1 to 0, 0 to 1)	0 or 1

The seven are the same pattern with a different field, step and range. Remove the `(void)u;` in each.

- **WHERE:** `src/ui.c`, the `static void act_...` functions.
- **CHECK:** compile only: `make t T=m4_03` still builds; the UI tests need STEP 3.

STEP 3 · TODO M4-03/3 · `ui_handle` · [src: M4-03 step 3, worked example light_handle]

- **WHAT:** Find the row, run its action, change mode.
- **INGREDIENTS:** `ui_table`, `ui_row_t`.
- **HOW (recipe):**
  1. Loop `i` over the rows of `ui_table` (row count = `sizeof ui_table / sizeof ui_table[0]`, which is 12).
  2. Find the row whose `mode` equals `u->mode` AND whose `ev` equals the argument `ev`.
  3. In that row: if `action` is not NULL, call it with `u`. Then set `u->mode` to the row's `next`. Stop (return).
  4. If no row matches, change nothing.

mode	event	action	next
M_TDIV	EV_UP	act_tdiv_up	M_TDIV
M_TDIV	EV_MODE	none (NULL)	M_VDIV

ui_table[i].action(u) calls the function stored in the row (a function pointer), as light_handle in the example returns the stored next state.

- **WHERE:** src/ui.c, ui_handle.
- **CHECK:** make t T=m4_03 -> test_ui_tdiv, test_ui_mode_cycle, test_ui_vdiv_level_edge, test_ui_none_event all :PASS.

STEP 4 · TODO M4-03/4 · buttons_init · cards C2 + C4 · [src: M4-03 step 4, cards C2+C4]

- **WHAT:** Make PB12, PB13, PB14 inputs with pull-up.
- **INGREDIENTS:** gpio_cfg, nibble 0x8, BSRR bits.
- **HOW:**

```
RCC->APB2ENR |= <CLOCK>;
gpio_cfg(GPIOB, <PIN1>, <NIBBLE>);
gpio_cfg(GPIOB, <PIN2>, <NIBBLE>);
gpio_cfg(GPIOB, <PIN3>, <NIBBLE>);
GPIOB->BSRR = BIT(<PIN1>) | BIT(<PIN2>) | BIT(<PIN3>);
```

name	value
CLOCK	RCC_APB2ENR_IOPBEN
PIN1, PIN2, PIN3	12u, 13u, 14u
NIBBLE	0x8u

This is card C2 three times (pins 12, 13, 14; all in CRH).

- **WHERE:** src/ui.c, buttons_init.
- **CHECK:** make t T=m4_03 -> test_buttons_code fails later, at the IDR line.

STEP 5 · TODO M4-03/5 · buttons_read · [src: M4-03 step 5]

- **WHAT:** Read the three buttons as a 3-bit mask (1 = pressed).
- **INGREDIENTS:** GPIOB->IDR, inversion ~, shift 12, mask 7.
- **HOW (recipe):**
  1. Read GPIOB->IDR.
  2. Invert it (pressed pins read 0).
  3. Shift right by 12 so PB12 becomes bit 0.
  4. Keep only the lowest 3 bits (mask 7).
  5. Return that as uint8_t.

pins (PB14 PB13 PB12)	IDR bits	result
none pressed	111	0
MODE pressed	110	1
UP pressed	101	2
DOWN pressed	011	4

- **WHERE:** src/ui.c, buttons_read. Delete the stub return 0;.
- **CHECK:** make t T=m4_03 -> 8 Tests 0 Failures, OK.

STEP 6 · finish (no TODO)

- Add test_m4_03 to tests.active, make test-active, commit M4-03 done.
- **Optional board check:** wire a button between PB12 and GND, poll buttons_read() with debounce_update in main, toggle the LED on each press event.

Verify

Do this	Expect
make t T=m4_03	8 Tests 0 Failures 0 Ignored, OK
Optional board check: press MODE once	the LED toggles once, even if the press feels rough

Do this	Expect
Hold the button down	no repeat toggles

If it fails, look here	Likely cause
bounce test fires early	missing the "raw != last" restart branch
wrap test fails	compared `now - t >= 20` with plain `>=` on times: use `elapsed`
UI test: level goes below 0	`clamp_i` missing
`buttons_read` is always 7	forgot `~`: pressed pins read 0
buttons always read as pressed	pull-up missing: BSRR bits 12..14, nibble 0x8

Self-check (answer aloud)

1. Why do buttons bounce, and what does a 20 ms hold prove?
2. What is stored in `last`, `stable` and `t_change`?
3. Why is the UI a table and not a long `if` chain?
4. Why does `buttons_read` use `~`?

Hint ladder (open one at a time)

1. STEP 1 is `deb_update` from the worked example with your field names.
2. Pattern: restart on change, accept after the hold time, else 0.
3. Template (fill each [X]): line A `if (raw != d->[A]) { d->[A] = raw; d->[B] = now_ms; return 0; }`, line B `if (raw != d->[C] && elapsed(now_ms, d->[B]) >= [D]) { d->[C] = raw; return raw; }`, line C `return 0;`. Values: [A] = `last`, [B] = `t_change`, [C] = `stable`, [D] = `DEBOUNCE_MS`.

Milestone M5: Measurements

Min, max, Vpp, average, frequency, text formatting.

Title: M5-01 min, max, Vpp and average

Milestone: M5 Measurements

Labels: m5, blank-level-4, host-tests

M5-01 min, max, Vpp and average

START HERE

1. Type `make t T=m5_01`. Red tests.
2. Open `src/measure.c`, find `@TODO(M5-01/1)`: `measure_stats`, one loop. Read the temperature example first.
3. `stats_vpp_mv` and `stats_avg_mv` (STEP 2) reuse card C5.

Starts from / Ends with

Starts from	Ends with
M4-03: UI works. `mul_div` (M2-01).	`measure_stats`: min, max and average in counts.
	`stats_vpp_mv`, `stats_avg_mv`: millivolts.

Recipe (what, in order, with its data)

#	Function	Data	Result
1	`measure_stats(s, n, st)`	`n` samples (1 to 256)	`st->min`, `st->max`, `st->avg` in counts. The sum needs 32 bits.
2	`stats_vpp_mv(st)`	`(max - min)` counts	millivolts: `mul_div(max - min, 3300, 4095)`
3	`stats_avg_mv(st)`	`avg` counts	millivolts: `mul_div(avg, 3300, 4095)`

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `mul_div`, `fb_bit`, `gpio_cfg`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

Three numbers describe a repeating signal: the lowest value (min), the highest (max) and the mean (average).

Vpp = peak-to-peak = max minus min. A signal that swings 0 V to 3.3 V has Vpp 3.3 V.

Average = sum of all samples divided by how many there are. A 25 % duty pulse of 3.3 V averages about 0.825 V. That is how you check your PWM without a multimeter.

Overflow trap. 256 samples of up to 4095 add up to 1,048,320. That does not fit in 16 bits but fits easily in 32. Use a `uint32_t` sum. Divide LAST.

Starting values. Start min and max from the FIRST sample, not from 0 or 4095. A starting min of 0 would never be beaten.

Unit conversion at the end (card C5): counts to millivolts with `mul_div`. Do the maths on counts first, convert once.

Worked example: statistics over a temperature array

```
/* WORKED EXAMPLE (M5-01): min, max, average over an array of temperatures in tenths of a degree. */
#include <stdint.h>
typedef struct { int16_t min, max; int32_t avg; } tstats_t;

void temp_stats(const int16_t *t, unsigned n, tstats_t *out)
{
    int16_t mn = t[0], mx = t[0];       /* start from the FIRST value, not from 0 */
```

```
    int32_t sum = 0;
    for (unsigned i = 0; i < n; i++) {
        if (t[i] < mn) mn = t[i];
        if (t[i] > mx) mx = t[i];
        sum += t[i];
    }
    out->min = mn;
    out->max = mx;
    out->avg = sum / (int32_t)n;        /* divide LAST */
}
```

Same loop with different data: tenths of a degree instead of ADC counts.

Cheat sheet

Item	Value
`stats_t`	`{ uint16_t min; uint16_t max; uint32_t avg; }` in counts
Loop	over all `n` samples, index `i` from 0
Start	`mn` and `mx` from the first sample; `sum` is a 32-bit variable starting at 0
Update	smaller than `mn`: replace it; larger than `mx`: replace it; always add to `sum`
End	store `mn`, `mx`, and `sum` divided by `n` (divide LAST)
`stats_vpp_mv`	(max - min) counts x 3300 / 4095 with `mul_div`
`stats_avg_mv`	avg counts x 3300 / 4095 with `mul_div`
Examples	min 0, max 4095, avg 1023: Vpp 3300 mV, avg 824 mV. Difference 2048 counts = 1650 mV
PWM test signal	real captures show min 0 to about 10, max about 4085 to 4095, avg about 1023 (824 mV)

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M4-03. `make t T=m5_01`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M5-01/1 · `measure_stats` · [src: M5-01 step 1, worked example temp_stats]

- **WHAT:** One pass: min, max, average.
- **INGREDIENTS:** the loop from the example.
- **HOW (recipe):**
  1. Start `mn` and `mx` from the FIRST sample `s[0]`. Start `sum` (a `uint32_t`) at 0.
  2. Loop `i` over all `n` samples. Each pass: if the sample is below `mn` it becomes the new `mn`; if above `mx` it becomes the new `mx`; add it to `sum`.
  3. After the loop store `mn` in `st->min`, `mx` in `st->max`, and `sum / n` in `st->avg`.

input	min	max	avg
10, 20, 30, 40	10	40	25
500, 700, 100, 900, 300	100	900	500
256 x 4095	4095	4095	4095

`st->min` stores into the struct the caller passed by pointer.

- **WHERE:** `src/measure.c`, `measure_stats`. Delete the stub lines.
- **CHECK:** `make t T=m5_01` -> `test_stats_small`, `test_stats_min_not_first`, `test_stats_single`, `test_sum_does_not_overflow`, `test_pwm_capture_numbers` all `:PASS`.

STEP 2 · TODO M5-01/2 · `stats_vpp_mv`, `stats_avg_mv` · card C5 · [src: M5-01 step 2, card C5]

- **WHAT:** Convert to millivolts.
- **INGREDIENTS:** `mul_div`, 3300, 4095.
- **HOW (recipe):** call `mul_div` with (the counts, 3300, 4095) and return the result.

function	the counts are
`stats_vpp_mv`	`max - min` (cast to `uint32_t`)
`stats_avg_mv`	`avg`

stats	Vpp mV	avg mV
min 0, max 4095, avg 1023	3300	824
min 1000, max 3048, avg 2048	1650	1650

This is card C5 again with (counts, 3300, 4095).

- **WHERE:** `src/measure.c`.
- **CHECK:** `make t T=m5_01` -> `6 Tests 0 Failures`, `OK`.

STEP 3 · finish (no TODO)

- Add `test_m5_01` to `tests.active`, `make test-active`, commit `M5-01 done`.
- **Optional board check:** with PA1 to PA2 at 25 %, capture 256 samples at 32000, run `measure_stats` and print `min`, `max`, `avg` (counts) over UART.

Verify

Do this	Expect
`make t T=m5_01`	`6 Tests 0 Failures 0 Ignored`, `OK`
Optional board check, PWM loop at 25 %	min 0 to about 10, max about 4085 to 4095, avg about 1000 to 1050 (about 0.82 V)
Same with PWM at 50 %	avg about 2047
PA2 on GND	min, max, avg all 0 to about 10

If it fails, look here	Likely cause
min is always 0	min started at 0 instead of `s[0]`
avg is wrong or negative	sum is a `uint16_t` (overflow): use `uint32_t`
Vpp is 0	you subtracted the wrong way round, or used avg
avg off by one	integer division truncates: that is allowed (the test uses a tolerance only for the PWM case)

Self-check (answer aloud)

1. Why must the sum be 32 bits?
2. Why start min and max from `s[0]`?
3. What Vpp does a 0 to 3.3 V square wave have?
4. Why convert to millivolts at the end?

Hint ladder (open one at a time)

1. It is the temperature example with a `uint16_t` array and a `stats_t` pointer.
2. Pattern: init from the first sample, one loop with three updates, divide last.
3. Template (fill each [X]): `uint16_t mn = s[[A]], mx = s[[A]];` with [A] = 0; loop `i` from [A] to `n`; the final line `st->avg = sum / [B];` with [B] = `n`.

Title: M5-02 Frequency from rising crossings, fmt_mv and fmt_hz

Milestone: M5 Measurements

Labels: m5, blank-level-4, host-tests

M5-02 Frequency from rising crossings, fmt_mv and fmt_hz

START HERE

1. Type `make t T=m5_02`. Red tests.
2. Open `src/measure.c`, find `@TODO(M5-02/1)`: `measure_freq`. Read the zero-crossing example first.
3. Then `src/fmt.c`: `fmt_mv` (STEP 2), `fmt_hz` (STEP 3).

Starts from / Ends with

Starts from	Ends with
M5-01: stats. `fmt_u32` (M1-05), `mul_div`.	`measure_freq`: Hz from sample data.
	`fmt_mv`: "3.30V".
	`fmt_hz`: "1.00kHz" or "800Hz".

Recipe (what, in order, with its data)

#	Function	Data	Result
1	`measure_freq(s, n, level, fs_hz)`	rising crossings of `level`; first index, last index, count	`fs_hz * (count - 1) / (last - first)` with `mul_div`, or 0 if fewer than 2 crossings
2	`fmt_mv(out, mv)`	`v = (mv + 5) / 10`	text like `3.30V`, returns the length
3	`fmt_hz(out, hz)`	below 1000: number + `Hz`; else `v = (hz + 5) / 10`	text like `1.00kHz`, returns the length

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `mul_div`, `i2c_ccr`, `wait_flag`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

Frequency = how many periods per second. You measure the time between repeats. Pick a repeating event: the signal crossing a level going UP. The distance between two such crossings is one period.

Use all the crossings. One period measured from two neighbours is rough (one sample of error is a big fraction). Instead take the first crossing and the last crossing: `last - first` samples cover `count - 1` periods. Average period = `(last - first) / (count - 1)` samples. Frequency = `fs / average period = fs * (count - 1) / (last - first)`. Multiply first, divide last (card C5): `mul_div(fs, count - 1, last - first)`.

Example. PWM 1 kHz sampled at 32 kHz: a period is 32 samples. 8 periods fit in 256 samples. Crossings might be at 27, 59, ..., 251 (8 of them). `fs * 7 / 224 = 32000 * 7 / 224 = 1000`.

What level? The scope uses the midpoint `(min + max) / 2`, so any signal crosses it.

Fewer than two crossings: answer 0.

Text without printf. `fmt_mv` shows volts with two decimals, rounded to the nearest 10 mV: `v = (mv + 5) / 10` hundredths; whole = `v / 100`, then `.`, then two digits `(v % 100) / 10` and `v % 10`, then `V`. `fmt_hz` below 1000 prints the number and `Hz`; from 1000 prints kHz with two decimals the same way. A helper `put_hundredths(out, v)` is already given in `fmt.c`: it writes `whole.dd` and returns the length.

Worked example: a zero-crossing period

```
/* WORKED EXAMPLE (M5-02): average period (in samples) of a signal from its upward zero crossings. */
#include <stdint.h>
/* Returns the period in samples times 16 (4 fractional bits), or 0 if there are fewer than 2 crossings. */
uint32_t zc_period_x16(const int16_t *s, unsigned n)
{
    unsigned first = 0, last = 0, count = 0;
    for (unsigned i = 1; i < n; i++) {
        if (s[i - 1] < 0 && s[i] >= 0) {          /* an upward crossing           */
            if (count == 0) first = i;            /* remember the first one       */
            last = i;                             /* always remember the latest   */
            count++;
        }
    }
    if (count < 2) return 0;
    return (uint32_t)(((uint64_t)(last - first) * 16u) / (count - 1u));   /* multiply first, divide last */
}
```

Yours is the same scan, except the crossing test uses `level` (not zero), the result is Hz, and there is no fixed-point fraction.

Cheat sheet

Item	Value
Rising crossing at i	`s[i-1] < level && s[i] >= level` (i from 1)
Remember	`first` (index of crossing #1), `last` (latest), `count`
Result	`count < 2` -> 0; else `mul_div(fs_hz, count - 1u, last - first)`
`fmt_mv`	hundredths = (mv + 5) / 10; `put_hundredths`, then the letter `V`, then a 0 byte; return the length
`fmt_mv` examples	3300 -> `3.30V`; 824 -> `0.82V`; 1650 -> `1.65V`; 5 -> `0.01V`; 4 -> `0.00V`; 0 -> `0.00V`
`fmt_hz` below 1000	number with `fmt_u32`, then `Hz`
`fmt_hz` from 1000	hundredths = (hz + 5) / 10; `put_hundredths`, then `kHz`
`fmt_hz` examples	1000 -> `1.00kHz`; 999 -> `999Hz`; 800 -> `800Hz`; 0 -> `0Hz`; 1006 -> `1.01kHz`; 40000 -> `40.00kHz`; 12345 -> `12.35kHz`
Given helpers	`put_str(out, "text")` returns the length; `put_hundredths(out, v)` writes `v` hundredths and returns the length; `fmt_u32(out, v)`
Return value	the number of characters written (not counting the 0 byte)

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M5-01. `make t T=m5_02`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M5-02/1 · `measure_freq` · card C5 · [src: M5-02 step 1, card C5, worked example zc_period_x16]

- **WHAT:** Frequency from the spacing of rising crossings.
- **INGREDIENTS:** the crossing test, `first`, `last`, `count`, `mul_div`.
- **HOW (recipe):**
  1. Keep three counters: `first`, `last`, `count`, all 0 at the start.
  2. Loop `i` from 1 to `n - 1`. A rising crossing is: sample `i - 1` below `level` AND sample `i` at or above `level`.
  3. At a crossing: if `count` is 0 then `first = i`. Always set `last = i`. Add 1 to `count`.
  4. After the loop: if `count` is less than 2 return 0.
  5. Otherwise return `mul_div(fs_hz, count - 1, last - first)`.

signal	crossings	result
period 32 samples, 8 periods, fs 32000	7 or 8	1000
period 16 samples, fs 16000	15	1000
flat line	0	0
one step up	1	0

This is card C5 again, with (fs, count - 1, last - first).

- **WHERE:** `src/measure.c`, `measure_freq`.
- **CHECK:** `make t T=m5_02` -> `test_freq_phase0`, `test_freq_phase5`, `test_freq_other_rate`, `test_freq_none` all `:PASS`.

STEP 2 · TODO M5-02/2 · `fmt_mv` · [src: M5-02 step 2]

- **WHAT:** Millivolts as `d.dd V` text.
- **INGREDIENTS:** `put_hundredths` (given).
- **HOW (recipe):**
  1. Compute `v = (mv + 5) / 10` (hundredths of a volt, rounded to the nearest 10 mV).
  2. Call `put_hundredths(out, v)`; it returns how many characters it wrote: keep that in `n`.
  3. Write the letter `V` at `out[n]`, add 1 to `n`, then write a 0 byte at `out[n]`.
  4. Return `n`.

mv	v	text
3300	330	`3.30V`
824	82	`0.82V`
5	1	`0.01V`
4	0	`0.00V`

- **WHERE:** `src/fmt.c`, `fmt_mv`. Delete the stub lines.
- **CHECK:** `make t T=m5_02` -> `test_fmt_mv:PASS`.

STEP 3 · TODO M5-02/3 · `fmt_hz` · [src: M5-02 step 3]

- **WHAT:** Hertz as `800Hz` or `1.00kHz`.
- **INGREDIENTS:** `fmt_u32`, `put_str`, `put_hundredths`.
- **HOW (recipe):**
  1. If `hz` is below 1000: write the number with `fmt_u32`, then append `Hz` with `put_str(out + n, ...)`, and return the total length.
  2. Otherwise: `v = (hz + 5) / 10`, `put_hundredths(out, v)`, then append `kHz` with `put_str`, and return the total length.

hz	v	text
999	(not used)	`999Hz`
1000	100	`1.00kHz`
12345	1235	`12.35kHz`

`out + n` is "the address `n` characters after `out`", so each piece lands after the previous one.

- **WHERE:** `src/fmt.c`, `fmt_hz`.
- **CHECK:** `make t T=m5_02` -> `6 Tests 0 Failures`, `OK`.

STEP 4 · finish (no TODO)

- Add `test_m5_02` to `tests.active`, `make test-active`, commit `M5-02 done`.

Verify

Do this	Expect
`make t T=m5_02`	`6 Tests 0 Failures 0 Ignored`, `OK`
Optional board check: PWM loop, capture at 32000, `measure_freq(g_cap, 256, 2047, 32000)` printed	1000 (allow 980 to 1020: within 2 %)
Same capture, `fmt_hz` of that value	`1.00kHz` (or `0.98kHz` to `1.02kHz`)

If it fails, look here	Likely cause
frequency twice too high	counted both edges: only rising crossings
frequency 0 on a good signal	the level is outside the signal range
result a little off	used `count` instead of `count - 1`
`fmt_mv(5)` shows `0.00V`	the `+ 5` rounding is missing
`fmt_hz(999)` shows `1.00kHz`	the `< 1000` branch is missing

Self-check (answer aloud)

1. Why `count - 1` periods between `count` crossings?
2. Why use the first and last crossing, not two neighbours?
3. What does `measure_freq` return with one crossing?
4. Why add 5 before dividing by 10?

Hint ladder (open one at a time)

1. STEP 1 is the zero-crossing example with `level` and a final `mul_div`.
2. Pattern: scan, remember first/last/count, `if (count < 2) return 0;`, `mul_div(fs, count - 1, last - first)`.
3. Template (fill each [X]): `mul_div([A], count - [B], last - first)` with [A] = `fs_hz` and [B] = 1; and the test `s[i - 1] < [C] && s[i] >= [C]` with [C] = `level`.

Milestone M6: Ship

Super loop, calibration, README, CI badge, tag v1.0.

Title: M6-01 scope_process, scope_draw and the main super loop

Milestone: M6 Ship

Labels: m6, blank-level-4, hardware, host-tests

M6-01 scope_process, scope_draw and the main super loop

START HERE

1. Type `make t T=m6_01`. Red tests.
2. Open `src/scope.c`, find `@TODO(M6-01/1)`: `scope_process`. It only calls functions you already wrote.
3. Read the Recipe table below, then the flag-driven loop example.

Starts from / Ends with

Starts from	Ends with
M5-02: stats, frequency, formatting. Everything from M1 to M5.	`scope_process`: trigger + stats + frequency.
	`scope_header` and `scope_draw`: text rows and plot.
	`main`: the super loop with buttons, capture, draw, flush.
	A working scope on the OLED.

Recipe (what, in order, with its data)

`scope_process(sc)` (data: `sc->cap` 256 samples, `sc->ui`):

#	Step	Data	Result
1	(calibration is added in M6-02 step 3; skip it now)	`g_cal`	nothing yet
2	`level = mv_to_counts(ui.level_mv)`	`level_mv`	counts
3	`trig = trig_find(cap, 128, level, ui.rising)`; `start = trig_start(trig)`		index or -1; window start
4	`measure_stats(cap, 256, &st)`		min, max, avg
5	`mid = (min + max) / 2`; if `max - min >= 100`: `freq_hz = measure_freq(cap, 256, mid, tdiv_to_fs(ui.tdiv_idx))` else 0		frequency

`scope_header(sc, line1, line2)`: line 1 = `tdiv_label` + space + `vdiv_label` + space + `fmt_hz(freq)` + space + `R` or `F`. Line 2 = mode letter from `"TVLE"` + space + `p` + `fmt_mv(vpp)` + space + `a` + `fmt_mv(avg)`.

`scope_draw(sc)`: `fb_clear`, `text_draw(0,0,line1)`, `text_draw(0,8,line2)`, `draw_grid`, `plot_wave(cap + start, 128, vdiv_mv[vdiv_idx])`. No flush.

`main`: init everything; loop: read buttons, debounce, `ui_handle`, restart capture on a change; when the capture is done: process, draw, flush, restart.

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `field_set`, `brr_for`, `fb_index`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

The super loop. There is no operating system. `main` ends in `for (;;)`. Inside, every pass checks "is there something to do?" and does it quickly. Slow waiting is forbidden. Interrupts only set flags (like SysTick advancing `millis`); the loop reads them.

Your loop is built from pieces you own.

1. Buttons: for each of the three buttons, `debounce_update` on the raw bit; a press returns 1: call `ui_handle(&ui, event)`. A change in settings means the old capture is stale: restart it with the new rate.
2. Capture: `sampler_wait(1)` asks "done yet?" with one try, so the loop never sits waiting. If done: `sampler_stop`, `scope_process`, `scope_draw`, `ssd1306_flush`, `sampler_start`.
3. LED heartbeat: toggle every 500 ms with `reached` (card C6). It tells you the loop is alive.

Event numbers. Button 0 = MODE = `EV_MODE` (1), button 1 = UP = `EV_UP` (2), button 2 = DOWN = `EV_DOWN` (3). So event = button index + 1.

Three small functions, one big job. `scope_process` finds numbers. `scope_header` makes the text. `scope_draw` paints the frame. `main` glues them. Keep each short.

Honest limits. At 8 kHz and slower (2 ms/div and up) the 1 kHz test signal is sampled fewer than 10 times per period. At 1.6 kHz and 800 Hz it is below the sampling limit: the picture and the frequency are wrong (aliasing). The scope is not broken: the signal is faster than the rate.

Worked example: a flag-driven super loop

```
/* WORKED EXAMPLE (M6-01): a flag-driven super loop. The interrupt only sets a flag; the loop does the work. */
#include <stdint.h>

static volatile uint8_t tick_flag;            /* volatile: an interrupt changes it behind the loop's back */
void SysTick_Handler(void) { tick_flag = 1; } /* ISR: set the flag and leave. Nothing else.               */

extern void read_sensor(void);
extern void update_screen(void);

int main(void)
{
    unsigned ticks = 0;
    for (;;) {                                /* the super loop never ends */
        if (tick_flag) {
            tick_flag = 0;                    /* clear first, then work    */
            ticks++;
            read_sensor();
            if (ticks % 100u == 0u) update_screen();
        }
    }
}
```

Your `main` has the same shape: the loop checks conditions, does bounded work, never blocks.

Cheat sheet

Item	Value
`scope_t`	`{ ui_t ui; uint16_t *cap; int trig; unsigned start; stats_t st; uint32_t freq_hz; }`
`scope_init`	given
Window	128 samples from `cap + start`; `start` is at most 127, so it always fits in 256
Trigger scan	the first 128 samples (`128u`), using `sc->ui.rising`
Level	`mv_to_counts` of `sc->ui.level_mv`
Frequency level	(min + max) / 2
Minimum swing for a frequency	`max - min >= 100` counts
Sample rate	`tdiv_to_fs(sc->ui.tdiv_idx)`
Labels	`tdiv_label[idx]`, `vdiv_label[idx]` (strings); mode letter `"TVLE"[sc->ui.mode]`
Text helpers	`fmt_hz(out, hz)`, `fmt_mv(out, mv)` return the length; `line1[n++] = ' ';` adds one character
Expected text (default UI, 1 kHz 25 % PWM at 1 ms/div)	line 1 `1ms 1V 1.00kHz R`; line 2 `T p3.29V a0.83V` (the real signal swings 3 to about 4090 counts)
Draw	clear, two text lines, grid, plot (see STEP 3)
Event from button i	button index + 1, cast to `ui_event_t`
`main` order of init	`clock_init`, `led_init`, `uart_init` (+ banner), `adc_init`, `sampler_init`, `pwm_init` + `pwm_set_duty(25u)`, `i2c_init`, `ssd1306_init(i2c_write)`, `buttons_init`, `scope_init(&g_scope)`
Capture start	`sampler_start` with `tdiv_to_fs(ui.tdiv_idx)`

Item	Value
Done check	`sampler_wait(1u)` returns 0
Globals in `main.c`	`static scope_t g_scope; static debounce_t g_deb[3];` (and a `start_capture` helper)

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M5-02. `make t T=m6_01`. **CHECK:** `6 Tests`, failures.

STEP 1 · TODO M6-01/1 · `scope_process` · [src: M6-01 step 1]

- **WHAT:** From 256 samples, work out trigger, stats and frequency.
- **INGREDIENTS:** `mv_to_counts`, `trig_find`, `trig_start`, `measure_stats`, `measure_freq`, `tdiv_to_fs`.
- **HOW (recipe):** (leave calibration out for now: M6-02 adds it)
  1. `level` = `mv_to_counts` of `ui.level_mv`.
  2. `trig` = `trig_find` over the first 128 samples with that level and the `ui.rising` edge. `start` = `trig_start(trig)`.
  3. `measure_stats` over all `SAMPLE_COUNT` samples into `sc->st`.
  4. `mid` = (min + max) / 2.
  5. If `max - min` is at least 100 counts: `freq_hz` = `measure_freq` over all samples with level `mid` and sample rate `tdiv_to_fs(ui.tdiv_idx)`. Else `freq_hz` = 0.

test signal (1 kHz, 1 ms/div)	result
rising crossing of 1650 mV first at index 16	`trig` 16, `start` 16
flat line	`trig` -1, `start` 0, `freq_hz` 0

- **WHERE:** `src/scope.c`, `scope_process`. Replace the stub `(void)sc;`. Leave the M6-02 comment about calibration alone for now.
- **CHECK:** `make t T=m6_01` -> `test_process_default:PASS`, `test_process_falling_and_auto:PASS`.

STEP 2 · TODO M6-01/2 · `scope_header` · [src: M6-01 step 2]

- **WHAT:** Build the two text lines.
- **INGREDIENTS:** `tdiv_label`, `vdiv_label`, `fmt_hz`, `fmt_mv`, `stats_vpp_mv`, `stats_avg_mv`.
- **HOW (recipe):** build two strings character by character, keeping a position `n`. Every `fmt_` call returns a length: add it to `n`.
  - Line 1, in order: the characters of `tdiv_label[ui.tdiv_idx]`; a space; the characters of `vdiv_label[ui.vdiv_idx]`; a space; `fmt_hz` of `freq_hz`; a space; `R` if `ui.rising` else `F`; the 0 byte.
  - Line 2, in order: the mode letter (index `ui.mode` into the text `"TVLE"`); a space; `p`; `fmt_mv` of `stats_vpp_mv(&sc->st)`; a space; `a`; `fmt_mv` of `stats_avg_mv(&sc->st)`. (`fmt_mv` already ends the string with a 0 byte.)

text	comes from
`1ms`	`tdiv_label[3]`
`1V`	`vdiv_label[0]`
`1.00kHz`	`fmt_hz(line1 + n, 1000)`
`R`	`ui.rising` is 1
`T`	`"TVLE"[M_TDIV]`
`p3.29V`	`p` then `fmt_mv(.., 3293)`

To copy a label: `for (p = label; *p; p++) line[n++] = *p;` where `p` is a `const char *`.

- **WHERE:** `src/scope.c`, `scope_header`.
- **CHECK:** `make t T=m6_01` -> `test_header:PASS`.

STEP 3 · TODO M6-01/3 · `scope_draw` · [src: M6-01 step 3]

- **WHAT:** Paint one frame into the framebuffer.
- **INGREDIENTS:** drawing functions from M3.
- **HOW (recipe):**

1. Declare two `char` arrays of 32 for the two lines.
2. `fb_clear()`.
3. Fill the lines with `scope_header`.
4. Draw line 1 at (0, 0) and line 2 at (0, 8) with `text_draw`.
5. `draw_grid()`.
6. `plot_wave` of 128 samples starting at `cap + start`, with `vdiv_mv[ui.vdiv_idx]`.
7. Do not flush here.

item	value
text positions	(0, 0) and (0, 8)
samples drawn	128
volts per division	`vdiv_mv[sc->ui.vdiv_idx]`

`sc->cap + sc->start` is "the address `start` samples after the buffer start".

- **WHERE:** `src/scope.c`, `scope_draw`.
- **CHECK:** `make t T=m6_01` -> `test_draw:PASS` and `test_draw_does_not_flush:PASS`.

STEP 4 · TODO M6-01/4 · `main` super loop · [src: M6-01 step 4]

- **WHAT:** Glue everything into the running scope.
- **INGREDIENTS:** the Recipe table for `main` and the cheat sheet. In `main.c`, the skeleton lists the old per-issue TODO comments: replace the body of `main` with the final version.
- **HOW (recipe):**
  1. At the top of the file: `static scope_t g_scope;`, `static debounce_t g_deb[3];` and a helper `start_capture` that calls `sampler_start` with `tdiv_to_fs(g_scope.ui.tdiv_idx)`.
  2. In `main`: call the init functions in the order from the cheat sheet. Then `start_capture()`. Remember `next_blink = millis() + 500`.
  3. Loop forever. Each pass: `now = millis()`; `raw = buttons_read()`.
  4. For each button `i` from 0 to 2: `debounce_update(&g_deb[i], bit i of raw, now)`. On a press: `ui_handle(&g_scope.ui, event)` where event = `i + 1` cast to `ui_event_t`, then `start_capture()`.
  5. If `sampler_wait(1u)` returns 0 (capture done): `sampler_stop()`, `scope_process`, `scope_draw`, `ssd1306_flush()`, `start_capture()`.
  6. If `reached(now, next_blink)`: `led_toggle()` and add 500 to `next_blink`.

button i	pin	event
0	PB12	`EV_MODE`
1	PB13	`EV_UP`
2	PB14	`EV_DOWN`

Bit `i` of `raw`: shift right by `i`, then keep the lowest bit.

- **WHERE:** `src/main.c`.
- **CHECK:** `make t T=m6_01` -> `test_main_loop_code:PASS` and `6 Tests 0 Failures`, `OK`. Then `make flash`.

STEP 5 · finish (no TODO)

- Add `test_m6_01` to `tests.active`, `make test-active`, commit `M6-01 done`, push: CI green.

Verify

Do this	Expect
`make t T=m6_01`	`6 Tests 0 Failures 0 Ignored`, `OK`
`make flash` with PA1 wired to PA2, OLED wired	a square wave (25 % high) on the screen, steady; line 1 about `1ms 1V 1.00kHz R`; line 2 about `T p3.29V a0.83V`
Press MODE four times	the letter in line 2 cycles T, V, L, E, T
Mode T, press UP	time/div goes up one step (`1ms` to `2ms`): more periods fit
Mode V, press UP to `500mV`	the 3.3 V high level is clipped at the top of the plot area
Mode E, press UP	line 1 ends in `F`: the picture is triggered on the falling edge
Switch to `500us/div`	frequency `1.00kHz` (32 kSPS: 32 samples per period); stays within 2 %

Do this	Expect
Switch to `20ms/div`	frequency reads wrong (aliasing): normal, see Limits
LED	blinks every 500 ms: the loop is alive

If it fails, look here	Likely cause
screen frozen	`start_capture` is never called again, or `sampler_wait` has the wrong tries
screen flickers or tears	`fb_clear` called before the flush ends: keep the order process, draw, flush
frequency shows `0Hz`	Vpp under 100 counts, or fewer than 2 crossings at this time/div
`p` shows a small number	`plot` is fine but the PWM jumper is missing
buttons do nothing	`buttons_init` missing; pull-up missing; event index off by one
line 2 shows `p0.00V`	`measure_stats` result not used: `scope_process` skipped
text runs off the screen	line 1 longer than 21 characters

Self-check (answer aloud)

1. Why can `main` never wait inside a loop?
2. Why does `sampler_wait(1u)` fit a super loop?
3. What happens to the capture when you press UP?
4. Why is the 1 kHz reading wrong at 20 ms/div?

Hint ladder (open one at a time)

1. `scope_process` is five calls you already wrote. Follow the Recipe table in order.
2. Pattern: `level` -> `trig_find` -> `trig_start` -> `measure_stats` -> `measure_freq`.
3. Template (fill each [X]): `sc->trig = trig_find(sc->cap, [A], level, sc->ui.[B]);` with [A] = 128u and [B] = `rising`; `sc->start = trig_start(sc->[C]);` with [C] = `trig`.

Title: M6-02 Calibration, README, CI badge, tag v1.0 (stretch: hold/freeze)

Milestone: M6 Ship

Labels: m6, blank-level-4, hardware, docs

M6-02 Calibration, README, CI badge, tag v1.0 (stretch: hold/freeze)

START HERE

1. Type `make t T=m6_02`. Red tests.
2. Open `src/cal.c`. Find `@TODO(M6-02/1)`: `cal_fit`, 4 lines. Then `cal_apply` (STEP 2).
3. Later: the calibration measurements on the board (STEPS 5 and 6), then README and tag.

Starts from / Ends with

Starts from	Ends with
M6-01: the scope runs on the OLED.	`cal_fit` and `cal_apply` (two-point calibration).
	`scope_process` runs samples through the calibration.
	Your own two calibration numbers in `g_cal`.
	README, CI badge, tag `v1.0`.

Recipe (what, in order, with its data)

#	Item	Data	Result
1	`cal_fit(c, raw_gnd, raw_3v3)`	counts at GND and at 3V3	`offset = raw_gnd`, `span = raw_3v3 - raw_gnd`; -1 if `raw_3v3 < raw_gnd + 2000`
2	`cal_apply(c, raw)`	`raw`	0 if `raw <= offset`; else `mul_div(raw - offset, 4095, span)` kept in 0..4095
3	hook in `scope_process`	`g_cal`, 256 samples	each sample corrected in place
4	README	measurements, wiring	filled sections
5	numbers into `g_cal`	your GND and 3V3 counts	`{ offset, span }`
6	tag	`git tag v1.0`	release

Warm-up (10 minutes, code only)

1. Save your work: `git add -A && git commit -m "before warm-up"`.
2. Open `src/drills.c`. In these three functions delete the BODY and keep the signature: `mul_div`, `clamp_i`, `reached`.
3. Rewrite each body from memory. (Names only from here on: you know the cards.)
4. Run `make test-active`. All green = done. Stuck? `git checkout src/drills.c` brings your old version back: read it once, then try again.

Learn (about 30 minutes)

No instrument is perfect. Two things drift: the ADC reads a few counts at 0 V (offset) and may read less than 4095 at 3.3 V (gain). A two-point calibration measures both ends and rescales between them.

You have no multimeter, but you have two exact voltages. GND is 0 V. The 3V3 pin is 3.3 V (as accurate as your regulator). Measure the ADC at each. Those two readings are your calibration points.

The rescale. `corrected = (raw - offset) * 4095 / span`, where `span = counts at 3V3 minus counts at GND`. That maps your two readings to exactly 0 and 4095. Use card C5, and keep the result inside 0..4095 with `clamp_i`. If `raw` is below the offset the subtraction would wrap around (unsigned): return 0 first.

Sanity rule. If the two readings are less than 2000 counts apart you wired something wrong. Reject.

Check with a known signal. Your PWM at 25 % should read an average near 0.82 V and a frequency of 1.00 kHz within 2 %.

Shipping. A project is done when a stranger can read the README, see CI green, and find a tag.

Stretch (optional): hold/freeze. Skip it if you are short of time.

Worked example

The two-point idea has no new hardware. Your worked example is the temperature-array code you already read in M5-01 plus card C5: first measure two reference values, then rescale. See `cal_apply`'s steps in the cheat sheet. (Pattern: `out = (in - low) * full / (high - low)`.)

Cheat sheet

Item	Value
`cal_t`	`{ uint16_t offset; uint16_t span; }`
Default	`{ 0, 4095 }` (does nothing: `cal_apply` returns the sample unchanged)
`cal_fit`	offset = reading at GND; span = reading at 3V3 minus reading at GND; reject if span is under 2000
`cal_apply`	corrected = (raw - offset) x 4095 / span; 0 if raw is at or below offset; clamp to 0..4095
Example	`cal_t c = {5, 4085}`: `cal_apply(c, 5)` = 0; `4090` = 4095; `4095` = 4095 (clamped); `2047` = 2046
Hook	first thing in `scope_process`: every sample goes through `cal_apply` with `g_cal`
Measuring the two points	PA2 on GND: capture 256 samples at 32000 Hz, print the average (`measure_stats`). PA2 on 3V3: the same. Unplug USB before moving the wire
Known good readings	GND: 0 to about 10. 3V3: about 4080 to 4095
Check	PWM loop, 25 %: average about 0.82 V; frequency `1.00kHz` within 2 %; Vpp near 3.30 V
Git tag	`git tag v1.0` then `git push --tags`
CI badge	markdown: `![ci](https://github.com/YOUR-NAME/scope-fw/actions/workflows/ci.yml/badge.svg)` (the README skeleton already has it: replace `YOUR-NAME`)
README sections	Hardware, Wiring, Build and test and flash, How it works, Measurements, Calibration, Limits
Stretch: hold	a `hold` flag in `scope_t`; a long press of MODE (held 1000 ms, measured with card C6 `elapsed`) toggles it; while `hold` is 1 skip `scope_process` and `scope_draw` but keep the last frame on the OLED

Card C5: `mul_div(a, b, c)`. Card C6: `elapsed = now - start`.

Cookbook steps

STEP 0 · set up (no TODO)

- Commit M6-01. `make t T=m6_02`. **CHECK:** `4 Tests`, failures.

STEP 1 · TODO M6-02/1 · `cal_fit` · [src: M6-02 step 1]

- **WHAT:** Turn two readings into offset and span.
- **INGREDIENTS:** the sanity rule (2000 counts).
- **HOW (recipe):**
  1. If `raw_3v3` is less than `raw_gnd + 2000`: return -1 and leave `*c` unchanged.
  2. Store `raw_gnd` in `c->offset`.
  3. Store `raw_3v3 - raw_gnd` in `c->span` (cast to `uint16_t`).
  4. Return 0.

raw_gnd	raw_3v3	result
5	4090	offset 5, span 4085, returns 0
4000	4090	returns -1, `c` unchanged

- **WHERE:** `src/cal.c`, `cal_fit`. Delete the stub lines.
- **CHECK:** `make t T=m6_02` -> `test_fit:PASS`.

STEP 2 · TODO M6-02/2 · `cal_apply` · card C5 · [src: M6-02 step 2, card C5, drill clamp_i]

- **WHAT:** Rescale one raw sample.
- **INGREDIENTS:** `mul_div`, `clamp_i`.

- **HOW (recipe):**
  1. If `raw` is at or below `c->offset`: return 0 (this avoids an unsigned wrap-around).
  2. Otherwise compute `mul_div(raw - offset, 4095, span)`.
  3. Keep the result inside 0..4095 with `clamp_i`.
  4. Return it as `uint16_t`.

c	raw	result
{5, 4085}	2	0
{5, 4085}	4090	4095
{5, 4085}	4095	4095 (clamped)
{0, 4095}	1234	1234

This is card C5 again, with (raw - offset, 4095, span).

- **WHERE:** `src/cal.c`, `cal_apply`.
- **CHECK:** `make t T=m6_02` -> `test_apply:PASS` and `test_identity:PASS`.

STEP 3 · TODO M6-02/3 · calibration hook · [src: M6-02 step 3]

- **WHAT:** Make the scope use the calibration.
- **INGREDIENTS:** `cal_apply`, `g_cal`, `SAMPLE_COUNT`.
- **HOW (recipe):** at the very start of `scope_process`, loop `i` over all `SAMPLE_COUNT` samples. Replace each `sc->cap[i]` by `cal_apply` of the global calibration `g_cal` and that sample. (`&g_cal` passes the address of the calibration struct.)

item	value
loop count	`SAMPLE_COUNT` (256)
calibration struct	`g_cal`

- **WHERE:** `src/scope.c`, `scope_process`, before the trigger line.
- **CHECK:** `make t T=m6_02` -> `test_scope_uses_cal:PASS`, `4 Tests 0 Failures`, `OK`.

STEP 4 · TODO M6-02/4 · README · [src: M6-02 step 4]

- **WHAT:** Write the README.
- **INGREDIENTS:** your wiring, build commands, results.
- **HOW:** fill each heading of `README.md`:

section	write
top paragraph	2 sentences: what the project is and what it measures
Hardware	the parts list
Wiring	a table: signal, pin, wire to (use the pin map in the spec sheet)
Build, test, flash	`make test-active`, `make firmware`, `make flash`
How it works	one paragraph: TIM3 triggers the ADC, DMA fills 256 samples, trigger, stats, draw
Measurements	your numbers from STEP 6
Calibration	your two `g_cal` numbers and how you got them
Limits	0 to 3.3 V only; aliasing at slow time/div; no input protection

Replace `YOUR-NAME` in the badge line with your GitHub name.

- **WHERE:** `README.md`. Remove the HTML comment that says TODO.
- **CHECK:** open the README on GitHub after `git push`: all headings have text and the badge shows `passing`.

STEP 5 · TODO M6-02/5 · measure your two points · [src: M6-02 step 5]

- **WHAT:** Get your offset and span.
- **INGREDIENTS:** a jumper, `measure_stats` (M5-01), UART.
- **HOW:** (hardware, no new code) in `main` temporarily capture 256 samples at 32000 Hz and print `st.avg`:

wire	do this	write down
PA2 on GND	unplug USB, move the wire, power up, read the printed average	`raw_gnd` (expect 0 to about 10)
PA2 on 3V3	unplug USB, move the wire, power up, read the printed average	`raw_3v3` (expect about 4080 to 4095)

Then replace the numbers in `cal_t g_cal = { 0, 4095 };` with `{ raw_gnd, raw_3v3 - raw_gnd }` (for example `{ 5, 4085 }`).

- **WHERE:** `src/cal.c`, the line under the TODO comment. Remove the temporary print code from `main` afterwards.
- **CHECK:** with PA2 on GND the scope's average reads `0.00V`; on 3V3 it reads `3.30V`.

STEP 6 · measure the PWM with the calibrated scope (no TODO)

- Wire PA1 to PA2, PWM 25 %, time/div `500us`.
- **CHECK (fill the Measurements section of your README):**

reading	expect
frequency (line 1)	`1.00kHz` (within 2 %: `0.98kHz` to `1.02kHz`)
Vpp (`p`)	near `3.30V`
average (`a`)	near `0.82V`

STEP 7 · finish and tag (no TODO)

- Add `test_m6_02` to `tests.active`, run `make test-active` (all 18 tests listed). Commit `M6-02 done`, `git push`, watch CI go green (both jobs).
- Tag: `git tag v1.0`, then `git push --tags`.

STEP 8 · stretch: hold/freeze (optional, no TODO, not tested)

- Add `int hold;` to `scope_t`. In `main`: on `EV_MODE` held for 1000 ms (card C6: remember the press time, then `elapsed(now, t0) >= 1000`), toggle `hold`. When `hold` is 1, skip `scope_process` and `scope_draw` (do not flush a new frame). Show `H` in line 2.
- **CHECK:** while holding, the picture stays still when you change the signal.

Verify

Do this	Expect
`make t T=m6_02`	`4 Tests 0 Failures 0 Ignored`, `OK`
`make test-active`	all 18 tests green
PA2 on GND, scope running	average `0.00V`
PA2 on 3V3	average `3.30V`
PA1 to PA2, 25 %, 500us/div	`1.00kHz` within 2 %, Vpp near `3.30V`, average near `0.82V`
`git tag`	lists `v1.0`
GitHub	README filled, badge green, tag `v1.0` visible

If it fails, look here	Likely cause
`test_apply` red at 2047	the subtraction is on the wrong side: `(raw - offset)` first
everything reads 0 V	`span` is 0 or wrong; check your `g_cal` numbers
signals look compressed	`span` computed from a reading that was not 3V3
`cal_fit` returns -1	the wires were not moved between the two captures
frequency wrong at slow rates	aliasing, not calibration
badge shows `failing`	open the Actions tab: a test you added to `tests.active` is red

Self-check (answer aloud)

1. What two errors does a two-point calibration remove?
2. Why return 0 when `raw <= offset`?
3. Which two known voltages do you have without a multimeter?
4. What does a green CI badge tell a stranger?

Hint ladder (open one at a time)

1. STEP 2 is card C5 with a subtraction inside the first argument.
2. Pattern: `mul_div(raw - offset, 4095, span)` then `clamp_i(..., 0, 4095)`.

3. Template (fill each [X]): `if (raw <= c->[A]) return 0;` and `mul_div((uint32_t)(raw - c->[A]), [B], c->[C])` with [A] = `offset`, [B] = `4095u`, [C] = `span`.

Print list (placeholder links)

Print these with Ctrl+P BEFORE you go offline. The kit works without them. They are for extra reading only. The links below are placeholders: replace each one before printing.

Topic	Placeholder link	Used in
ARM Cortex-M3 startup and vector table explained	<example.website/cortex-m3-startup>	M1-02
GNU linker script guide	<example.website/ld-scripts>	M1-02
GDB cheat sheet	<example.website/gdb-cheat-sheet>	Toolchain
OpenOCD and ST-Link guide	<example.website/openocd-stlink>	M1-03
SysTick and clock trees	<example.website/stm32-clocks>	M1-04
UART framing and baud rates	<example.website/uart-basics>	M1-05
ADC basics: sampling and quantisation	<example.website/adc-basics>	M2-01
DMA basics	<example.website/dma-basics>	M2-02
PWM basics	<example.website/pwm-basics>	M2-03
I2C protocol explained	<example.website/i2c-basics>	M3-01
SSD1306 command overview	<example.website/ssd1306-commands>	M3-02
Oscilloscope triggering explained	<example.website/scope-trigger>	M4-01
Sampling theorem and aliasing	<example.website/nyquist-aliasing>	M4-02, M6-01
Debouncing buttons	<example.website/debounce>	M4-03
Fixed-point scaling in C	<example.website/fixed-point>	M5-01
Unity test framework guide	<example.website/unity-guide>	Everywhere
GitHub Actions basics	<example.website/github-actions>	M1-03